

## 《陋室铭》

卡不在大，有灵则名。

码不在繁，有智则灵。

斯是陋室，惟吾德馨。

毛玻璃浮影，圆角映光荧。

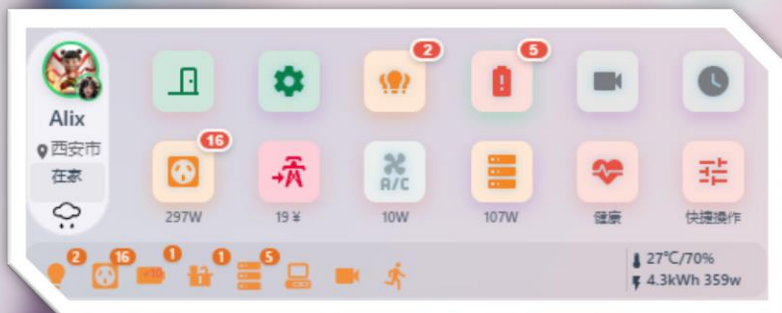
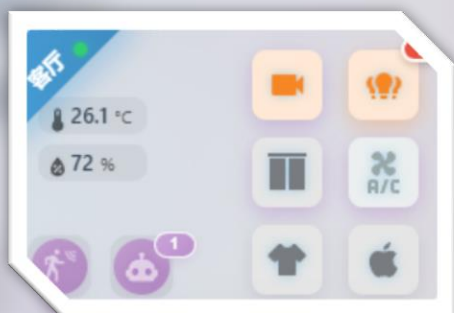
谈笑有主题，往来无滞停。

可以调光色、控媒体、启空调、管插屏。

无喧嚣之乱耳，无繁琐之劳形。

亮暗随晨昏，黑白自切换。

铭者云：何陋之有？



# 目录

|     |                                 |    |
|-----|---------------------------------|----|
| 一、  | 更新日志 .....                      | 6  |
| 二、  | 整体介绍 .....                      | 8  |
| 三、  | 安装 .....                        | 12 |
| 四、  | 外部组件 .....                      | 13 |
| 五、  | 基础配置结构 .....                    | 15 |
| 六、  | 专属配置项 .....                     | 19 |
| 1.  | 头部模式专属配置项 .....                 | 19 |
| (1) | 布局方式 .....                      | 19 |
| (2) | 概览 .....                        | 21 |
| (3) | 用户 (type: user) .....           | 35 |
| (4) | 公告 (v3.1.2 新增功能) .....          | 41 |
| 2.  | 普通模式专属配置项 .....                 | 47 |
| 3.  | 程序扩展坞 (Dock) v5.0.0 新增 .....    | 54 |
| 4.  | 头部多屏模式 v5.0.0 新增 .....          | 58 |
| 5.  | 其他配置项 .....                     | 60 |
| (1) | 禁止首页滚动 .....                    | 60 |
| (2) | 性能模式 (performance_mode) .....   | 61 |
| (3) | 外部 API 配置 v4.0.0 新增 .....       | 62 |
| 七、  | 通用配置 .....                      | 65 |
| 1.  | 房间/头部 .....                     | 65 |
| 2.  | 按钮 (核心功能) .....                 | 65 |
| (1) | 灯光组 (type: lights) .....        | 66 |
| (2) | 单个灯光 .....                      | 71 |
| (3) | 空调 (type: ac) v3.1.3 增强功能 ..... | 73 |
| (4) | 插座/开关 (type: socket) .....      | 76 |

|      |                                   |     |
|------|-----------------------------------|-----|
| (5)  | 耗材 (type: consumables)            | 79  |
| (6)  | 快捷操作 (scene_mode)                 | 81  |
| (7)  | 动态图标 (type: dynamic_icon)         | 94  |
| (8)  | 晾衣架 (type: clothes_dryer)         | 100 |
| (9)  | 无类型按钮                             | 103 |
| 3.   | 卡片 (核心功能)                         | 107 |
| (1)  | 传感器 (type: sensor)                | 110 |
| (2)  | 开关 (type: switch)                 | 111 |
| (3)  | 选择 (type: select)                 | 112 |
| (4)  | 数值 (type: number)                 | 114 |
| (5)  | 文本输入 (type: text)                 | 115 |
| (6)  | 按钮 (type: button)                 | 116 |
| (7)  | 卡片 (type: card)                   | 118 |
| (8)  | 图表 (type: chart_line)             | 119 |
| (9)  | 媒体 (type: media) v5.0.0 重新设计      | 120 |
| (10) | 图片 (type: picture)                | 123 |
| (11) | 时间轴 (type: timeline)              | 125 |
| (12) | 单选 (type: radio)                  | 127 |
| (13) | 窗帘 (type: curtain)                | 128 |
| (14) | 仪表图 (type: chart_gauge)           | 136 |
| (15) | 进度条图 (type: chart_progress)       | 137 |
| (16) | 统计型柱状图 (type: chart_bar)          | 138 |
| (17) | 环形图 (type: chart_pie)             | 140 |
| (18) | 网页卡片 (type: html)                 | 141 |
| (19) | 日历热力图 (type: chart_heatmap)       | 144 |
| (20) | 日历图 (type: chart_calendar)        | 146 |
| (21) | 混合图 (type: chart_mixed)           | 148 |
| (22) | 南丁格尔玫瑰图 (type: chart_nightingale) | 150 |
| (23) | 完整饼图 (type: chart_pie_full)       | 151 |
| (24) | 条件卡片 (type: conditional_tabs)     | 152 |
| (25) | 按钮组 (type: button_group)          | 156 |
| (26) | 快捷操作 (type: action)               | 162 |

|      |                                           |     |
|------|-------------------------------------------|-----|
| (27) | 健康 (type: health) v4.0.0 新增 .....         | 166 |
| (28) | 天气卡片 (type: weather) v5.0.0 新增 .....      | 171 |
| (29) | 调试卡片 (debug_card) v5.6.4 新增 .....         | 173 |
| (30) | 自动化卡片 (type: automations) v5.5.0 新增 ..... | 176 |
| (31) | 最近使用卡片 (recently-used) v5.4.0 新增 .....    | 180 |
| (32) | 使用量日历卡片 (usage_calendar) 新增 .....         | 183 |
| (33) | 打印机用量统计卡片 (type: printer) 新增 .....        | 192 |
| (34) | 爱快路由管理卡片 (type: ikuai) .....              | 194 |
| (35) | 设备用电统计卡片 (type: device_usage) .....       | 201 |
| 4.   | 通用动作系统 (tap_action) .....                 | 204 |
| (1)  | toggle — 切换开关 .....                       | 205 |
| (2)  | set_value — 设置目标值 (最灵活) .....             | 205 |
| (3)  | quick-action — 快捷操作 .....                 | 212 |
| (4)  | more-info — 详细信息弹窗 .....                  | 214 |
| (5)  | navigate — 页面导航 .....                     | 214 |
| (6)  | call-service — 调用 HA 服务 .....             | 215 |
| (7)  | popup_card — 弹出自定义卡片 .....                | 215 |
| (8)  | card — 弹出设备内置控制弹窗 .....                   | 216 |
| (9)  | entities 多实体批量操作 .....                    | 219 |
| (10) | 自动弹窗 (auto_open_entity) .....             | 220 |
| (11) | 循环赋值 (loop_assign) .....                  | 224 |
| 5.   | 弹窗系统 .....                                | 226 |
| (4)  | 弹窗架构概览 .....                              | 226 |
| (5)  | 弹窗类型一览 .....                              | 229 |
| (6)  | 弹窗动画 .....                                | 233 |
| (7)  | 弹窗宽度 (width) .....                        | 234 |
| (8)  | 自定义卡片弹窗 (popup_card) .....                | 235 |
| (9)  | 自由布局弹窗 (show_free_layout_popup) .....     | 237 |
| 6.   | 样式 .....                                  | 263 |
| 7.   | Way 计算引擎 .....                            | 265 |
| 8.   | <b>primary</b> 文本 (v3.1.3 增强功能) .....     | 271 |
| 9.   | 配置共享 (多个卡片共用配置) .....                     | 275 |



|     |                                                                                                                                                                          |     |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 10. | 外部 API 历史数据 .....                                                                                                                                                        | 286 |
| 八、  | 独立卡片模式 (standalone) v5.0.0 新增 .....                                                                                                                                      | 287 |
| 1.  | 适用场景 .....                                                                                                                                                               | 287 |
| 2.  | 基本配置 .....                                                                                                                                                               | 288 |
| 3.  | 支持的设备类型 .....                                                                                                                                                            | 288 |
| 4.  | 配置示例 .....                                                                                                                                                               | 290 |
| 九、  | 配置速查表 .....                                                                                                                                                              | 300 |
| 1.  | 顶层配置速查表 .....                                                                                                                                                            | 300 |
| 2.  | 按钮配置速查表 .....                                                                                                                                                            | 302 |
| 3.  | 图表配置速查表 .....                                                                                                                                                            | 304 |
| 4.  | 配置共享速查表 .....                                                                                                                                                            | 305 |
| 十、  | 外部组件及文件 .....                                                                                                                                                            | 306 |
|     | <div style="text-align: center;"> <br/> <b>pobaby_package</b><br/> <b>.rar</b> </div> |     |
| 1.  | 外部组件及其他文件 .....                                                                                                                                                          | 306 |
| 十一、 | 配置示例 .....                                                                                                                                                               | 307 |
| 十二、 | 配置成果 .....                                                                                                                                                               | 308 |

## 一、 更新日志

2026 年 6 月 15 日更新 **v5.0.0**

- 头部模式新增多屏显示；
- 新增程序扩展坞（dock）；
- 部分卡片开放独立使用功能；
- 动态图标新增轮播功能、预设天气和空调动态图标显示、图标右上角增加显示文本功能（支持 jinja2 模板）；
- 本地支持完整 jinja2 模板；
- 新增天气卡片；
- 修改媒体播放卡片支持本地列表播放、脱离卡片后端列表播放。

## 2026 年 6 月 3 日更新 v4.0.0

- 修复空调卡片未实时更新的 bug;
- 公告栏支持 html 的 css 样式设置、新增弹出窗口配置（见 6.1.4);
- 卡片使用的资源（空调图片资源、外部 js、和风天气的图标文件、户型图的坐标）全部移动到 www\pobaby\_package 文件夹中，文件见附件;
- 历史记录全面适配新制作的“HA 数据统一存储系统”，即 HA 没有历史记录时会自动到 API 处拉取历史记录（见 6.3.3);
- 健康卡片全面可用，数据走“HA 数据统一存储系统”（见 7.3.27);
- 插座弹出窗口可配置不显示确认对话框（见 7.2.4);
- 修复其他 bug;

注：我制作的其他卡片（温湿度、回放、）全面适配“HA 数据统一存储系统”，欢迎使用。

## 二、 整体介绍

Room Elves Card（房间精灵卡片）是一款功能极为强大的 Home Assistant 自定义卡片，旨在将一个房间（或全屋）的所有设备状态与控制能力集中到一张卡片中，实现"一卡掌控全家"的智能家居体验。

### 核心特性：

- 20+ 种设备类型支持：灯光（单灯/灯组）、空调、插座/开关、耗材/电池、暖气/新风机、媒体设备、通用设备（门磁/窗磁/人体传感器）、扫地机/吸尘器、晾衣架、窗帘（单层/双层）、风扇、动态图标、时间轴、情景模式、快捷操作、按钮组、用户卡片、话费、晾衣架等。
- 丰富的弹出控制面板：每种设备类型都有专属弹窗——灯光面板支持亮度/色温调节、按房间分组、批量开关；空调面板可调温度/模式/风速；窗帘面板支持百分比滑块控制、单双层切换；晾衣架面板带机械仿真拖拽和收藏位置；情景模式支持执行进度展示和状态验证。
- 两种显示模式：
  - 普通模式：左侧显示房间名称和传感器数据，右侧显示设备操作按钮，适合单房间展示。
  - 头部模式（head）：所有按钮铺满卡片，顶部可显示概览栏（灯光/空调/插座/耗材/环境/人员/能耗统计），

适合放在页面顶部作为全屋控制中心。

- 通用动作系统 (tap\_action): 统一的动作引擎覆盖所有可点击位置, 支持 toggle (自动推断服务切换开关)、set\_value (智能推断服务设置目标值)、quick-action (快捷操作/情景模式)、more-info、navigate、call-service、popup\_card、card (弹出设备内置控制弹窗) 等 10 种动作类型, toggle 和 set\_value 还支持多实体批量操作。
- 选项卡系统: 弹窗内支持选项卡分组, 具备图标、高亮、自动跳转、条件过滤 (display\_only)、禁用、角标、分组归属 (belong) 等高级功能, 支持按楼层/区域分组展示。
- 平面户型图: 支持 JSON 坐标数据的平面户型图渲染, 实时显示各房间人员占用状态, 拖动时间轴可回溯历史活动。
- 人员/天气/导航: 用户卡片支持多人头像布局 (当前登录用户大头像居中)、在/离家状态检测、实时定位、高德地图导航路线规划、多地区天气预报。
- 情景模式: 一键执行预定义操作列表, 支持气泡选择/直接执行模式、延时执行、执行进度实时展示、状态验证、执行历史记录, 服务调用自动推断 (覆盖 light/climate/cover/fan/media\_player/vacuum/lock/automation/script 等所有域)。还可作为统一卡片类型

(type: action) 或 tap\_action: quick-action 在弹窗、title\_entities 等任意位置使用。

- 自动弹窗 (auto\_open\_entity): 当指定实体变为 on 时自动打开弹窗, 支持延迟弹出和自动关闭, 适合告警联动场景。
- 自定义外观: 支持自定义卡片尺寸、背景色、圆角、主题切换 (亮/暗), 按钮图标/颜色/动画均可配置, CSS 变量覆盖扩展。
- 动画与交互: 图标支持 shake/rotate/blink/breathe/jump 等动画效果, 点击反馈、状态过渡动画、光晕特效等。

详细信息请前往: Room Elves Card — 出鞘即封喉, 此剑一出谁与争锋【原《陋室铭》】

<https://bbs.hassbian.com/thread-32410-1-1.html>







### 三、 安装

- 1、 将文件 `room-elves-card.js` 放入 Home Assistant 配置目录下的 `www` 文件夹中。
- 2、 在 HA 侧边栏的"配置" → "仪表盘" → 右上角三点菜单 → "资源"中，添加一个 JavaScript 模块资源，路径填 `/local/room-elves-card.js`。添加后刷新页面即可。

## 四、 外部组件

为了获得更好的体验，将外部组件（js 等其他资源）可以放在 `www\pobaby_package` 文件夹中（v4.x.x 及后续版本都会从此处获取外部资源），优先从本地加载，当本地文件不可用时会从 CDN 拉取对应的 js。

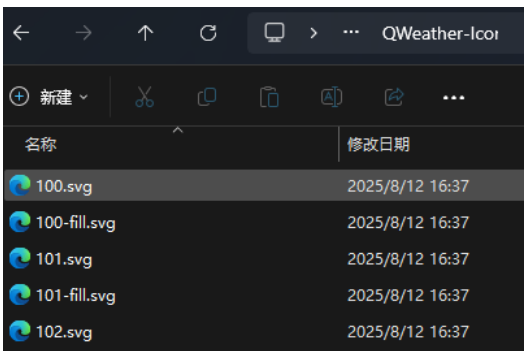
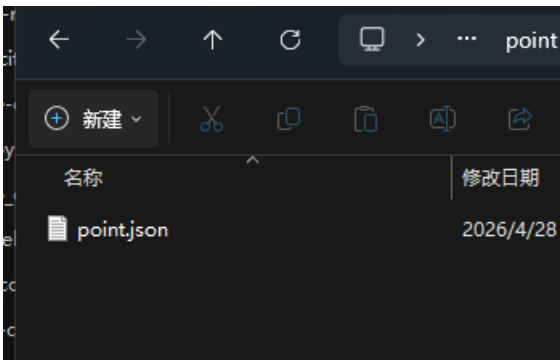
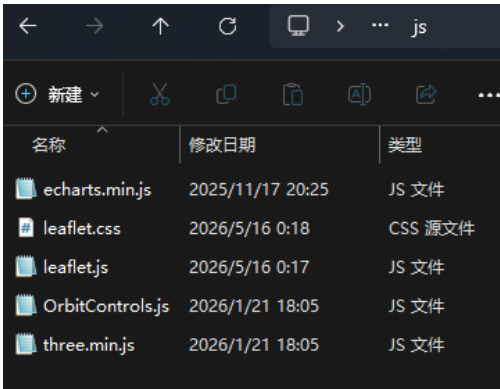
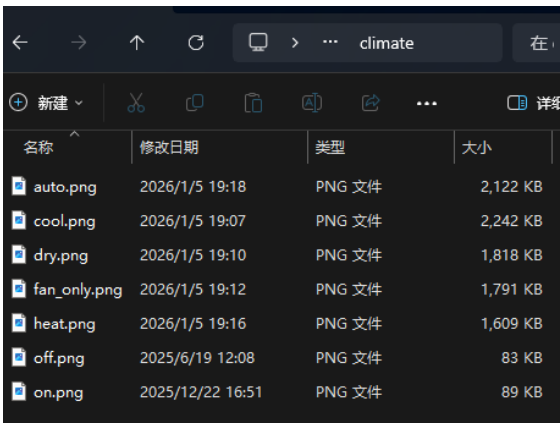
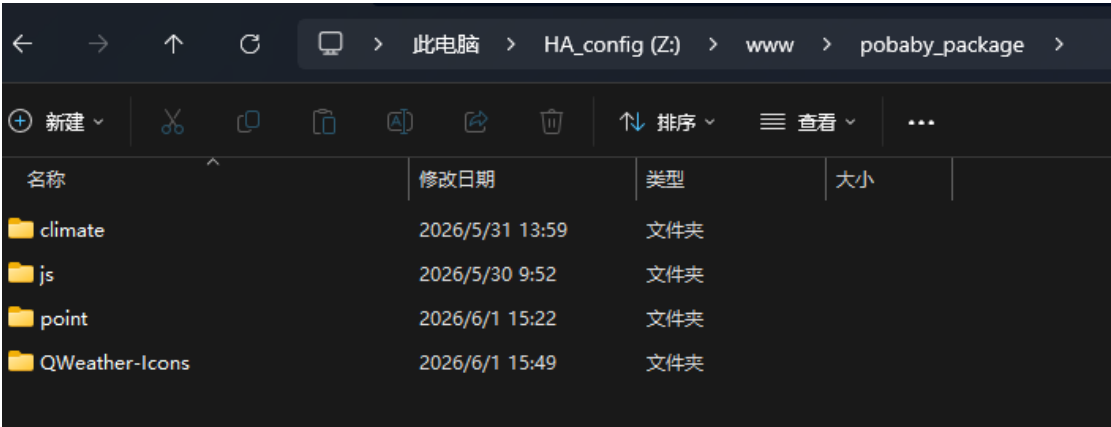
主要涉及图表（`echarts.min.js`）、地图（`leaflet.js`、`leaflet.css`）、3D 温湿度（`OrbitControls.js`、`three.min.js`）、和风天气的天气图标文件、户型图坐标等。

- 图表（`echarts.min.js`）：主要用来绘制各类图表：  
`chart_line`、`chart_pie`、`chart_pie_full`、  
`chart_nightingale`、`chart_heatmap`、  
`chart_calendar`、`chart_mixed`，以及插座桑基图。
- 地图（`leaflet.js`、`leaflet.css`）：主要是在用户卡片（`type: user`）中渲染高德地图。
- 3D 温湿度（`OrbitControls.js`、`three.min.js`）：主要为温湿度卡片 3D 显示使用。
- 和风天气的天气图标文件：只要用来显示和风天气的天气图标，本地加载，速度远高于网络获取，该文件必须放置，不放则天气预报不显示图标。

**注意：**目前新版本的 Room Elves Card（房间精灵卡片）全面摒弃了高德地图的 js，全面使用 leaflet，大幅度降低内

存、cpu 占用率，完全可以放心使用。

涉及到的文件在 word 最后的内置文件中，下载下解压后放置在 `www\pobaby_package` 文件夹下即可。



## 五、 基础配置结构

配置采用层级结构，最外层定义卡片的全局行为和外观，内层通过 `style`、`heartbeat_packet`、`entities`、`person`、`automation`、`overview`（头部模式才可配置该项）、`buttons` 等字段组织具体功能。

### 卡片的两大模式

卡片的布局由 `head` 字段决定，两种模式在视觉和功能上有显著差异：

| 特性   | 普通模式 (head: false) | 头部模式 (head: true) |
|------|--------------------|-------------------|
| 布局   | 左右分栏：左侧传感器 + 右侧按钮  | 按钮铺满整张卡片          |
| 传感器  | 显示在卡片左侧 (entities) | 不显示左侧传感器区域        |
| 概览栏  | ❌ 不支持              | ✅ 顶部显示全屋态势概览      |
| 适用场景 | 单房间卡片，展示该房间传感器和设备  | 全屋控制中心，放在页面顶部总览全局 |

💡 **选择建议：** 如果你只想管理一个房间，使用普通模式即可；如果你想在页面顶部放一张“全屋控制中心”卡片，同时总览所有房间的灯光、空调、人员、能耗等状态，请使用头部模式。

### 配置结构：

```
type: custom:room-elves-card # 固定写法，告诉 HA 使用这张卡片
room_name: 客厅 # 房间名称，会显示在卡片上
mode: grouped # 显示模式，保持 grouped 即可
head: false # 是否启用头部模式 (true=启用, false=普通模式)
theme: light # 主题设置 (见后文)
show_animation: true # 是否让按钮上的图标有动画效果
style: # 自定义卡片外观 (尺寸、颜色等)
-
```

```

entities: # 传感器列表（在普通模式下显示在左侧）
-
heartbeat_packet #心跳包，后面详解
-
person: # 人员在传感器（卡片左下角，点击弹出活动弹窗）
-
automation: # 自动化开关（卡片底部右侧，点击弹出控制面板）
-
overview: # 概览栏（! 仅 head 模式可用，非 head 模式无效）
-
buttons: # 按钮配置（这是卡片最重要的部分）
-

```

## 顶层配置项说明

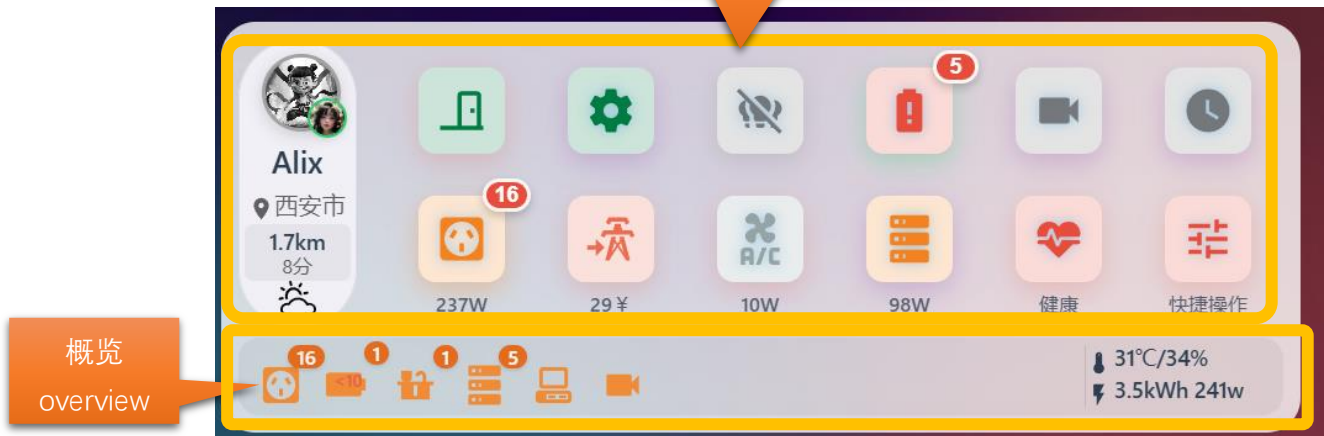
| 配置项            | 必填 | 类型      | 默认值     | 说明                                                                   |
|----------------|----|---------|---------|----------------------------------------------------------------------|
| type           | ✓  | string  | —       | 固定为 custom:room-elves-card                                           |
| room_name      | ✗  | string  | 无       | 房间名称，显示在卡片左上角。头部模式下也会作为卡片标识                                          |
| mode           | ✗  | string  | grouped | 卡片布局模式，保持 grouped 即可                                                 |
| head           | ✗  | boolean | false   | 是否启用头部模式。false = 普通模式（左侧传感器 + 右侧按钮），true = 头部模式（按钮铺满，顶部显示概览栏）        |
| theme          | ✗  | string  | light   | 主题风格，可选 light / dark 或自定义主题名                                         |
| show_animation | ✗  | boolean | true    | 是否启用按钮图标动画效果（如空调旋转、灯光呼吸等）                                            |
| style          | ✗  | object  | 无       | 自定义卡片外观，支持 width、height、background、border-radius 等 CSS 属性            |
| entities       | ✗  | array   | []      | 传感器数据列表，普通模式下显示在卡片左侧，支持温度、湿度、CO2、PM2.5 等任意传感器实体                      |
| buttons        | ✗  | array   | []      | 按钮配置列表，卡片核心功能区域。支持 20+ 种按钮类型（灯光、空调、插座、耗材、窗帘、晾衣架、情景模式等），每种类型有专属弹窗控制面板 |

|            |   |        |    |                                                                                                                                |
|------------|---|--------|----|--------------------------------------------------------------------------------------------------------------------------------|
| person     | ✗ | array  | [] | 人员在传感器配置，卡片左下角显示人体感应小按钮，点击弹出活动弹窗（含实时状态、时间轴、活动统计、平面户型图）                                                                         |
| automation | ✗ | array  | [] | 自动化开关列表，卡片底部右侧显示小图标，点击弹出控制面板统一管理所有自动化的启用/禁用                                                                                    |
| overview   | ✗ | array  | [] | ⚠ 仅 head 模式可用，非 head 模式下配置无效。全屋设备态势概览栏，显示在卡片顶部，支持环境（温湿度）、人员（在家状态）、能耗（用电/功率/余额）、天气（多城市预报）等类型。灯光/空调/插座/耗材概览在 head 模式下自动统计，无需手动配置 |
| notice     | ✗ | object | 无  | ⚠ 仅 head 模式可用，公告栏配置。显示在概览栏上方，支持模板文本和实体状态两种内容类型，多条内容时支持自动滚动<br><br>V3.1.2 新增功能                                                  |

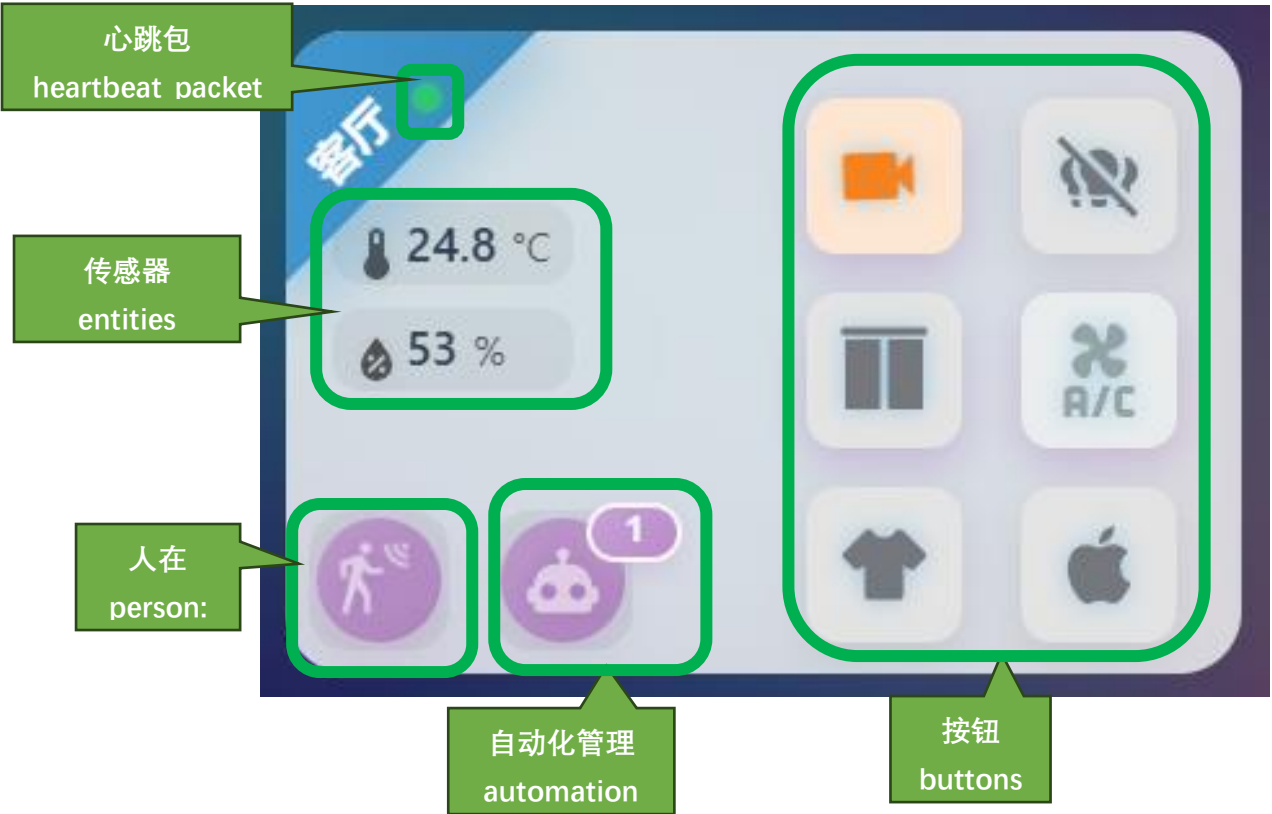


头部模式:

按钮 buttons



普通模式:



## 六、 专属配置项

不管采用何种模式，所有类型的按钮都可以在两种模式中布置。下面先对每种模式的专属配置做一详解。

### 1. 头部模式专属配置项

适合放在页面顶部作为全屋控制中心，可以显示全屋设备、耗材、人员等等情况。专属配置主要有**布局方式**和**概览**。

#### (1) 布局方式

头部模式 (head: true)，支持混合按钮布局，卡片采用 CSS Grid 网格布局，按钮铺满整个卡片区域，支持灵活的跨行跨列控制。

通过 head\_columns 和 head\_rows 控制网格的列数和行数：

|                 |                      |
|-----------------|----------------------|
| head: true      |                      |
| head_columns: 6 | # 网格列数，默认 6 列        |
| head_rows: auto | # 网格行数，默认 auto（自动撑开） |

| 配置项          | 类型     | 默认值  | 说明                     |
|--------------|--------|------|------------------------|
| head_columns | number | 6    | 网格列数，决定一行最多放几个按钮       |
| head_rows    | string | auto | 网格行数，auto 表示根据按钮数量自动撑开 |

#### 按钮跨行跨列 (row\_column)

每个按钮可以通过 row\_column 字段指定它在网格中占几行几列，格式为 “序号, 行数-列数”：



```
head: true
head_columns: 6
buttons:
  - type: lights
    card:
      - entity: light.living_ceiling
        row_column: '1,2-3'          # 序号 1, 占 2 行 3 列 (大按钮)
  - type: ac
    card:
      - entity: climate.living
        row_column: '2,1-2'          # 序号 2, 占 1 行 2 列
  - type: media
    entity: media_player.tv
    row_column: '3,1-4'              # 序号 3, 占 1 行 4 列 (横条)
  - entity: switch.plug              # 没有 row_column 会自动排列 (占 1 行 1 列)
```

row\_column 格式说明:

| 格式        | 含义        | 示例                      |
|-----------|-----------|-------------------------|
| 序号, 行数-列数 | 指定按钮占几行几列 | 1,2-3 = 序号 1, 占 2 行 3 列 |

常见布局示例:

| row_column | 效果         |
|------------|------------|
| 1,1-1      | 1×1 标准小按钮  |
| 1,2-2      | 2×2 方形大按钮  |
| 1,2-3      | 2×3 宽高大按钮  |
| 1,1-2      | 1×2 横向双宽   |
| 1,1-4      | 1×4 横条     |
| 不配置        | 自动排列 (1×1) |

序号只决定排列顺序，不决定最终位置。系统会自动检测碰撞，优化布局。

(2) 概览

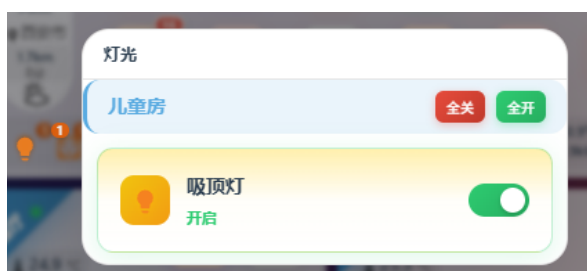


当 head: true 时，可以在卡片顶部显示一条全屋设备态势概览栏。它会自动统计各设备类型的运行状态数量，点击可弹出对应的控制面板。

支持**灯光概览、空调概览、插座概览**、耗材概览、环境概览、人在概览、能耗概览、天气概览、自定义设备概览，其中**灯光概览、空调概览、插座概览**无需配置，会自动读取全屋设备后构建概览页面和弹出窗口页面，剩余的概览需要配置，也可以直接引用其他卡片一配置的数据作为数据源（见……）

## ● 灯光概览

自动从 type: lights 按钮中配置的数据作为数据源，无需额外配置，点击会自动弹出已开启的列表。



## ● 空调概览

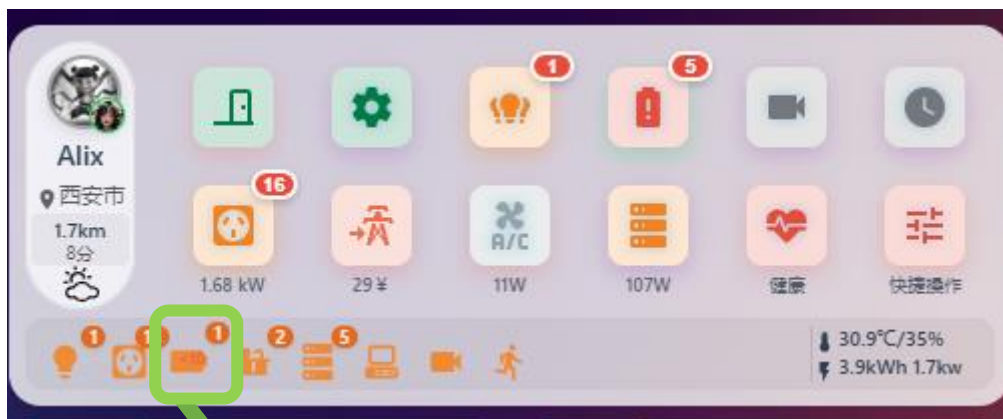
自动从 type: ac 按钮中配置的数据作为数据源，无需额外配置，点击会自动弹出已开启的列表。

## ● 插座概览

自动从 type: socket 按钮中配置的数据作为数据源，无需额外配置，点击会自动弹出已开启的列表。

## ● 耗材概览

自动从 type: consumables 按钮中配置的数据作为数据源，仅需要配置报警值，点击会自动弹出低于设置值的列表。



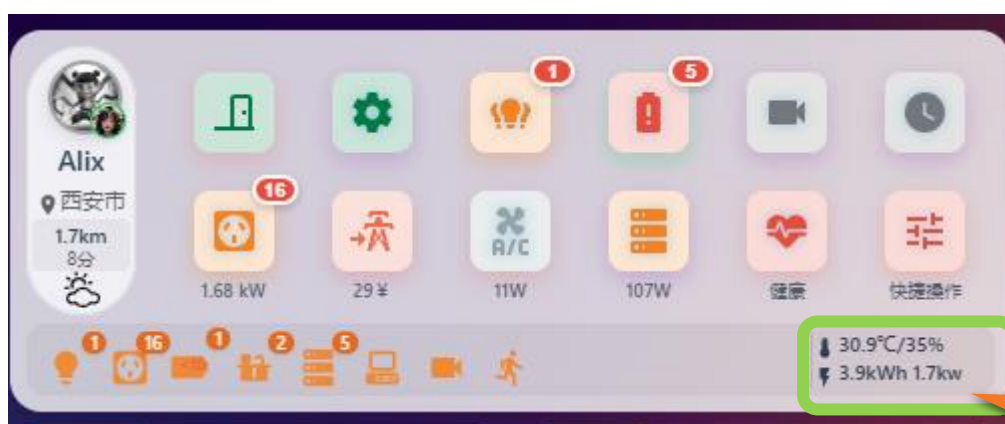
```

- type: consumables
  name: 耗材
  icon: mdi:battery
  condition:
    - <10

```

## ● 环境概览

显示室内外温湿度数据，点击会有气泡显示曲线图。同时支持自定义显示内容和弹出窗口。



默认方式：

```

- type: environment
  name: 环境
  icon: mdi:thermometer-water
  rooms:
    户外:
      temperature: sensor.ketingwaiyangmian_wendu
      humidity: sensor.ketingwaiyangmian_shidu
    客厅:
      temperature: sensor.keting_wendu
      humidity: sensor.keting_shidu

```

自定义方式：

一行一个 content; icon / name 均可省略。未配置时完全走现行模式：「户外温湿度 + 用电量」两段代码原样保留。

```

overview:
- type: right-col
  items:
    - icon: mdi:thermometer
      name: 客厅

```

```

content: >-
  {{ states('sensor.keting_wendu') }}°C
/({{ states('sensor.keting_shidu') }})%
- icon: mdi:flash
name: 今日
content: >-
  {{ states('sensor.today_energy') | float(0) | round(1) }}<span
style="color:#e67e22">kWh</span>
tap_action:
  action: navigate
  navigation_path: /lovelace/energy

```

点击以上空白区域弹出来页面：



## ● 人员概览

在房间平面图中显示在家人员、各房间占用状态。

点击房间弹出该房间的人员使用时长，点击人员显示该人员的在家时间段、时长，彩色表示进行中。同时历史记录支持选择时间。



```

- type: person
  name: 人在
  icon: mdi:run
  rooms:
    客厅: switch.giot_cn_1126348886_v64ksm_on_p_10_1    #房间的人在传感器
    餐厅: binary_sensor.canting_renzai
    厨房: binary_sensor.chufang_you_ren
    儿童房: binary_sensor.ertongfang_you_ren
    主卧: binary_sensor.zhuwo_renzai
    次卧: binary_sensor.ciwo_renzai
    大卫生间: binary_sensor.daweishengjian_renzai
    小卫生间: binary_sensor.xiaoweishengjian_renzai
  person:
    - name: Alix
      entity: device_tracker.ip15_94                #确定在家的实体
    - name: SPP
      entity: device_tracker.spp_39
  room_point_from: /local/point.json                #房间平面图的坐标数据

```

room\_point\_from 表示远程加载房间坐标点，房间坐标点以 yaml 的形式存放在 json 文件中，可以使用我制作的温湿度卡片（温湿度赋 <https://bbs.hassbian.com/thread-31171-1-1.html>）提供的房间绘制工具来绘制后导出自定义格式即可，格式如下。

坐标点示例：

```
Z: > www > {} pointjson
1  rooms:
2    - id: 次卧
3      name: 次卧
4      points: 14.5000,151.6875 129.5000,151.6875 129.5000,278.6875 14.5000,278.6875
5      labelX: "72"
6      labelY: "185"
7      dataX: "72"
8      dataY: "220"
9      room-data_font_size_round: 60,1
10     room-label_font_size: 55
11     room-unit_display:
12       temperature: true
13       humidity: true
14   - id: 厨房
15     name: 厨房
16     points: >-
17       136.5000,136.6875 245.5000,136.6875 245.5000,199.6875
18       136.5000,199.6875
19     labelX: "191"
20     labelY: "138"
21     dataX: "191"
22     dataY: "185"
23     room-data_font_size_round: 60,1
24     room-label_font_size: 55
25     room-unit_display:
26       temperature: true
27       humidity: true
```



● 能耗概览

显示用电量、功率、电费、水费、天然气使用量以及余额等信息，点击可以弹出对应的图表。

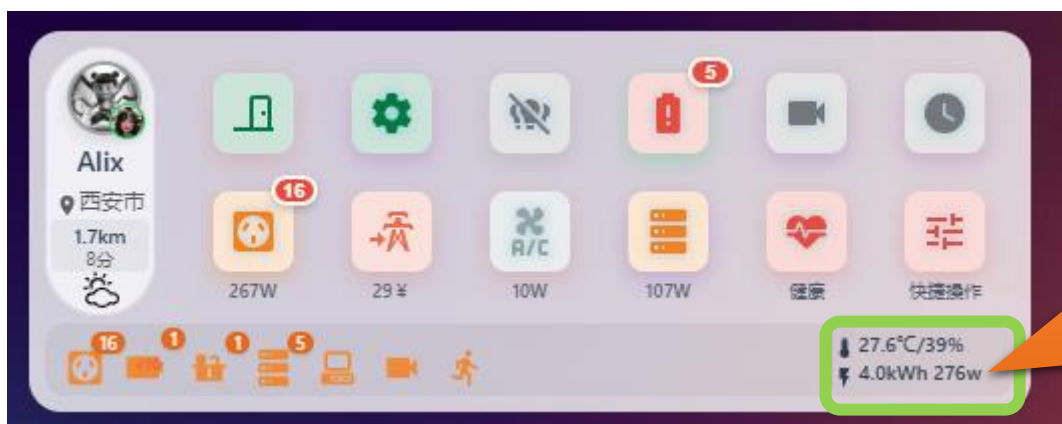


配置代码：

```
overview:
- type: energy
  name: 能耗
  icon: mdi:gas-burner
  utilities:
    天然气:
      entity: sensor.gas_day                #天然气日用量
      icon: mdi:gas-burner
      cost_entity: sensor.tian_ran_qi_2_new_2  #天然气余额
    电力:
      entity: sensor.ele_6103231959710        #电力
      tap_action:
        card_config:
          from_config_id: ele      #弹出窗口引用其他弹出窗口，后续详解
          get_type: all
          icon: mdi:transmission-tower-import
          power: sensor.quanwu_zongglv      #全屋功率
          power_onsumption: sensor.quan_wu_ri_yong_dian_liang  #全屋用电量
```

本处只配置了天然气，自来水配置方法一样。

power（全屋功率）、power\_onsumption（全屋用电量）的值会在首页的概览条中显示实体值，如下图所示。



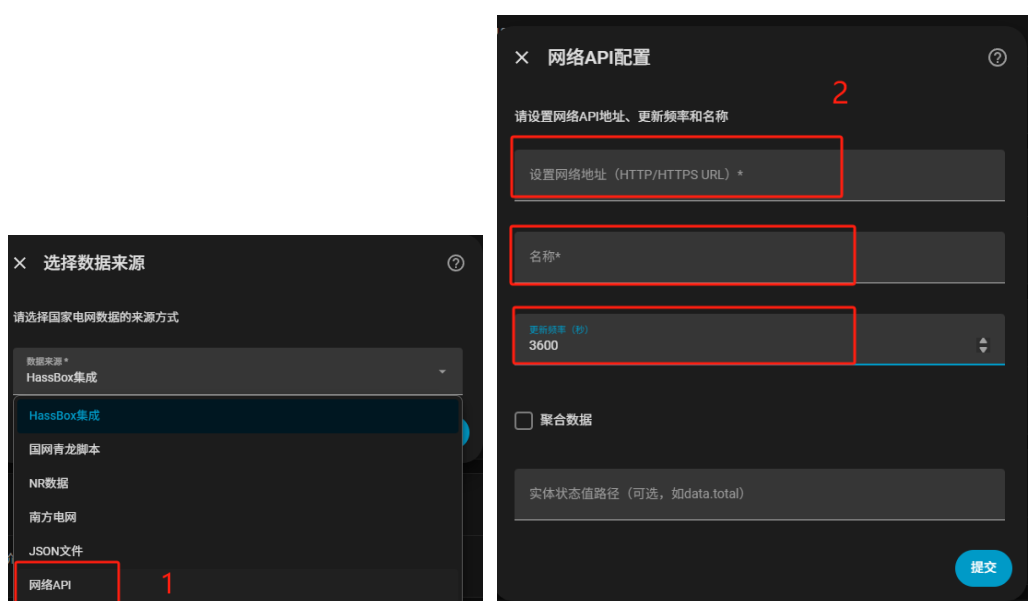
## ● 天气概览

主要显示天气预报，数据来源为和风天气的 7 天天气预报，支持从实体处获取数据和网络 API 处获取数据。

从实体处获取需要安装我在“消逝”制作的国网集成辅助基础上魔改的的另一个集成

([https://github.com/chjspp520/state\\_grid\\_info\\_enhance](https://github.com/chjspp520/state_grid_info_enhance))，该集成会把网络 api 转换为实体，并按照设置的更新频率来更新数据，建议使用此种方式，该方式在体验和 api 请求数量方面有较大优势。

集成安装自行百度，使用方法如下：

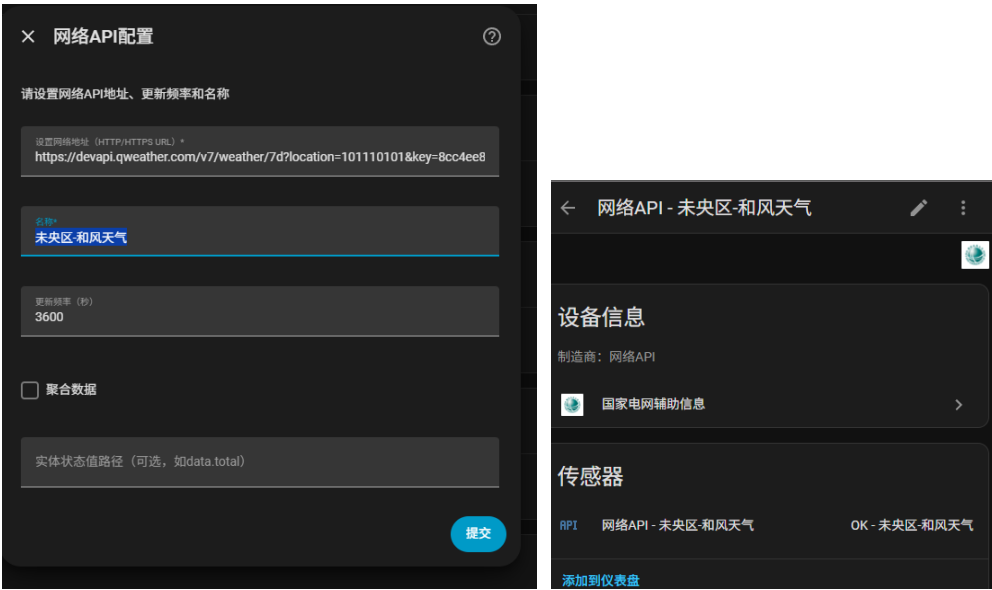


网络 api 需要在和风天气的官方网站中申请，api 地址为 <https://devapi.qweather.com/v7/weather/7d?location=城市代码&key=和风天气的key>。其中城市代码在 <https://github.com/qwd/LocationList> 中 China-City-List-

latest.csv 文件中查找后填入即可。

将你的 api 地址填入第 2 步的网络地址栏，并设置好名称，如“未央区-和风天气”，如下：

集成支持同时增加多个城市的天气预报（网络 api 转实体）。



## 实体模式：

```
overview:
- type: weather
  weather:
    - name: 未央区
      entity: sensor.weather_wei_yang_qu #刚才的通过网络 api 转换的实体 ID
    - name: 西乡县
      entity: sensor.weather_xi_xiang_xian
```

## API 模式：

```
overview:
- type: weather
  weather:
    - name: 西安市
      entity: sensor.he_feng_xi_an
      api_base_url:
        https://devapi.qweather.com/v7/weather/7d?location=101110101&key=xxx
    - name: 汉中市
      api_base_url:
        https://devapi.qweather.com/v7/weather/7d?location=101110101&key=xxx
```

**数据源优先级：** entity > api\_base\_url。同时配置时优先用实体数据，实体无数据则降级到 API。

**API 数据缓存：** 从 API 获取的数据会缓存 30 分钟，避免频繁请求。打开气泡时如果缓存过期会自动刷新。

**无实体纯 API 模式：** 当没有 entity 只有 api\_base\_url 时，首次加载会显示"加载中"占位，API 数据获取后自动刷新为真实天气数据。

**注意：** 该处配置的天气会在首页的用户卡片中显示天气。

- 自定义设备概览

用户可以根据自己的设备情况创建适合自己的设备统计类别和相应的弹出窗口。

该类型通过 `type: overview_device` 来区分是否为自定义概览设备。



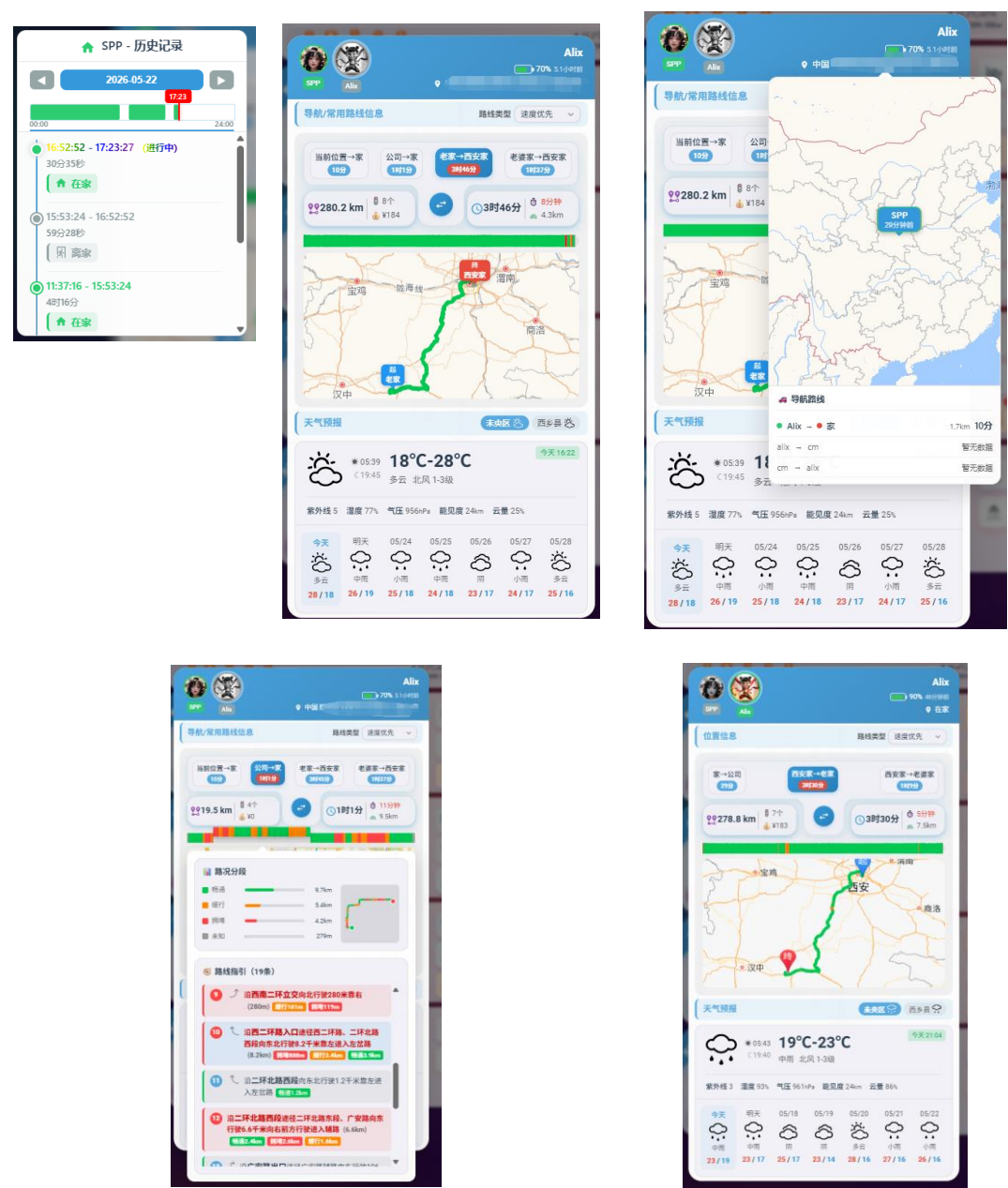
## 配置示例:

```
overview:
- type: overview_device
  icon: mdi:countertop
  show_card: true          # 概览弹窗是否显示
  how_badge: true          # 概览栏是否显示角标
  name: 厨具开启数量
  condition:               # 匹配条件
    - "on"
    - Delay
    - Keep Warm
    - Busy
  entities:                # 要统计的设备列表
    - entity: sensor.chunmi_cn_404489380_eh1_status_p_2_1
      name: 电饭锅
    - entity: switch.kongqizhaguo
      name: 空气炸锅
    - entity: switch.weibolu
      name: 微波炉
    - entity: switch.youyanji
      name: 油烟机
    - entity: switch.reshuiqi
      name: 热水器
    - entity: input_boolean.shao_shui_hu
      name: 烧水壶
    - entity: switch.canting_bingxiang_chazuo
      name: 冰箱
  tap_action:
    action: call-service
    service: modern_room_card.show_free_layout_popup
    service_data:
      title: 全部厨具
      width: 430px
      popup_position: center
      display_only:         # 仅显示那些状态的选项卡
        - "on"
        - Delay
        - Keep Warm
        - Busy
    auto_redirect: true
  tabs:
    - tabs_config:
        from_config_id:    # 组装引用自其他卡片中的配置的选项卡/弹窗作为
        选项卡, 后面详解
        - config_id: dian_fan_guo    #其他卡片配置的 id
          name: 电饭锅
        - config_id: kong_qi_zhaguo
          name: 空气炸锅
        - config_id: wei_bo_lu
          name: 微波炉
        - config_id: you_yan_ji
          name: 油烟机
        - config_id: re_shui_qi
          name: 热水器
        - config_id: shao_shui_hu
          name: 烧水壶
        - config_id: bingxiang
          name: 冰箱
```

### (3) 用户 (type: user)

用户 (type: user) 卡片主要显示用户在家/离家状态, 导航、路况、地图、天气情况。

最多配置 5 个家庭成员。卡片中当前登录 HA 的用户头像居中突出 (大头像), 其余成员以小头像按上/左/右/下环绕布局。





## 配置示例:

```
buttons:
- type: card
  row_column: 1,2-1
  card_config:
    type: user #用户卡片
    persons:
      - entity: person.alix #用户账号实体
        at_home: device_tracker.ip15_94 #用于判断是否是在家的实体
        location: sensor.l5pro_geocoded_location #位置实体
        phone_wifi: sensor.l5pro_ssid #wifi 实体
        phone_battery: sensor.l5pro_battery_state #电池实体
        phone_battery_level: sensor.l5pro_battery_level #电量实体
      - entity: person.spp
        at_home: device_tracker.spp_39
    bubble_map_show_navigation: #点击地址弹出的气泡
      - alix → cm #用户间的距离/时间
      - cm → alix
    location:
      show_location: input_boolean.show_local #是否显示地址，可以是实体
      #也可以是 true.false
    bubble_map_type: ampa #地图类型，请按此填写，开源地图国内无法访问
    weather: #天气
      - name: 未央区
        entity: sensor.weather_wei_yang_qu #和环境概览中的天气是同一个实体
      - name: 西乡县
        entity: sensor.weather_xi_xiang_xian
    navigation:
      auto_load_maps: input_boolean.auto_load_maps #是否自动打开地图
      amap_web_key: xxxxxxxxxxxx #高德 web_api
      amap_web_js_key: xxxxxxxxxxxx #高德 web_js_api
      home_coordinates: 109. xxxxxx,34. Xxxxxx #家庭坐标
      current_coordinates: 108. xxxxxx,34. Xxxxxx #默认目的地
      converted_coordinates: false #是否坐标转换
      frequent_routes: #自定义线路
        - name: 家→公司
          start: 109. xxxxxx,34. xxxxxx
          end: 108.902682,34.239788
          time: "12:00"
          reverse_entity: device_tracker.ip15_94 #反向实体
          reverse_entity_value: not_home #当反向实体为该值时反向
        - name: 老家→西安家
          start: 107. xxxxxx,32. xxxxxx
          end: 109. xxxxxx,34. xxxxxx
          reverse_entity: device_tracker.ip15_94
          reverse_entity_value: home
        - name: 老婆家→西安家
          start: 108. xxxxxx,34. xxxxxx
          end: 109. xxxxxx,34. xxxxxx
          reverse_entity: device_tracker.ip15_94
          reverse_entity_value: home
```

注意：导航相关的数据需要申请高德地图的 api，申请地址为 <https://console.amap.com/dev/key/app>，申请的类型有“web 服务”和“web 端 (js\_api)”，申请好后，将对应的 api

填写在配置中即可。

⊕ 为「HA」添加Key

\* Key名称:

支持汉字、数字、字母、下划线、中划线，不超过15个

命名规范

\* 服务平台:

Android平台

iOS平台

Web端(JS API)

Web服务

智能硬件

微信小程序

HarmonyOS NEXT 平台

可使用服务: [JavaScript API](#) [地图组件](#) [室内地图 JS API](#) [地铁图 JS API](#)

域名白名单:

非必填，留空表示无域名限制  
添加域名白名单后，只有白名单中的域名可访问服务  
域名填写格式，如：www.amap.com，多个域名请每行填写一条  
支持子域名通配符，如：\*.domain.com

☐ 阅读并同意

[高德地图开放平台服务协议](#) 和 [高德地图开放平台隐私权政策](#)

法人或非法人组织使用平台服务应事先购买[技术服务许可](#)以获取授权。若您未购买技术服务许可，平台向您提供的KEY和服务配额仅供您用于短期、少量的测试目的；若您超出前述范围使用平台服务（包括但不限于您开始向第三方或公众销售或提供您的应用、将您的应用于参与第三方投标、您的应用在应用程序分发平台上架或已经可以被公众获取使用、您的应用开始收费或发布广告或以其他方式获益、在组织内部使用的您的应用已上线运营、您的应用长期或大量调用平台服务等情形），构成未授权使用，平台有权采取相应措施。

取消

提交

HA 2026/3/18 创建

[编辑](#) [添加Key](#) [删除](#)

| Key 名称        | Key <a href="#">商用说明</a> | 安全密钥 (点击查看 <a href="#">安全密钥使用说明</a> ) | 绑定服务  | 操作 <a href="#">①</a>                                       |
|---------------|--------------------------|---------------------------------------|-------|------------------------------------------------------------|
| HA_Web_js_api | c8372d                   |                                       | Web端  | <a href="#">设置</a> <a href="#">删除</a>                      |
| HA_WEB        | b44ef34cee02             | —                                     | Web服务 | <a href="#">设置</a> <a href="#">查看配额</a> <a href="#">删除</a> |

另外，填写家庭坐标、目的地坐标等，需要从<https://lbs.amap.com/tools/picker>地址找到对应的地点，获取坐标，填写到需要的地方即可。



### 天气配置说明：

| 字段           | 必填 | 类型     | 说明                      |
|--------------|----|--------|-------------------------|
| name         | ✗  | string | 地区名称，显示在 Tab 切换按钮上      |
| entity       | ✗  | string | 和风天气 HA 实体 ID（优先使用）     |
| api_base_url | ✗  | string | 和风天气 API 地址（实体无数据时降级使用） |

**数据源优先级：** entity > api\_base\_url。同时配置时优先用实体数据，实体无数据则降级到 API。

**API 数据缓存：** 从 API 获取的数据会缓存 30 分钟，避免频繁请求。打开弹窗时如果缓存过期会自动刷新。

### 每个 person 成员字段说明：

| 字段                  | 必填 | 类型                            | 说明                                                    |
|---------------------|----|-------------------------------|-------------------------------------------------------|
| entity              | ☑  | person. xxx                   | 头像/姓名/user_id 均从此实体自动获取                               |
| at_home             | ☑  | device_tracker. xxx           | home → 在家 / 其他 → 离家。控制头像边框颜色 + 灰度滤镜                   |
| location            | ✗  | sensor. xxx_geocoded_location | HA 移动端自动解析的地址文本；坐标从 attributes.Location [lat, lon] 读取 |
| phone_wifi          | ✗  | sensor. xxx_ssid              | 手机当前连接的 WiFi 名（弹窗头部显示）                                |
| phone_battery       | ✗  | sensor. xxx_battery_state     | 手机电池状态（charging / discharging）                        |
| phone_battery_level | ✗  | sensor. xxx_battery_level     | 手机电池电量百分比（0-100）                                      |

at\_home 为必填。仅凭 person.state 无法判断是否在家（可能是 company 等 zone 名称），只有 device\_tracker.state == 'home' 能准确判断。

**位置坐标获取优先级（location 未配置时的降级）：**

1. location 实体 → attributes.Location 数组
2. at\_home (device\_tracker)  
→ attributes.latitude / longitude

3. person 实体的 attributes.device\_trackers →  
遍历其中各 device\_tracker

**卡片布局效果：** 当前登录用户头像居中（大头像），其余成员的小头像环绕四周。头像颜色反映在/离家状态： 绿=在家， 灰=离家（灰度滤镜）。

**弹窗布局效果：** 所有成员头像在头部并排显示，当前用户高亮（稍大+白色外圈），各自显示"在线/离线"状态胶囊。地址区域显示焦点用户的位置。地图上以不同颜色标记所有人的坐标，带导航路线规划。

**小头像点击：** 点击任意小头像，弹窗切换为该成员专属的定位/导航/详情页。

(4) 公告 (v3.1.2 新增功能)



公告栏显示在概览栏上方，用于展示通知、状态提示等信息。支持两种内容类型：

content：使用 Jinja2 模板语法（内置 `_TemplateEngine`），支持 `states()`、`state_attr()`、`is_state()`、`iif()`、`now()` 等函数，另外，由于卡片内置解析能力有限，如果使用比较复杂的 jinja2 语句，本地无法解析时，请服务端解析参数（`server_explain: true`）使用服务端解析即可。

entity：直接读取实体状态值 + 单位

两种类型可以混排在 `items` 数组中。

基本配置

```
notice:
  style:                                # 可选，自定义样式
    - width: 100%
    - height: 30px
  scroll: false                          # 是否自动滚动（仅多条时有效）
  items:
    - content: "{{iif(is_state('device_tracker.spp_39', 'not_home'), '不在家"

```

```

', '在家'))}"
- entity: sensor.keting_wendu          # 显示: 状态值 +
unit_of_measurement
- content: "{{now().hour}}时{{now().minute}}分"

```

## 自动滚动模式

当 `scroll: true` 时，多条内容会自动逐条向上滚动，每条停顿约 1.5 秒后继续，无缝循环。

```

notice:
  style:
    - width: 100%
    - height: 30px                    # 滚动模式下建议设固定高度
  scroll: true
  items:
    - content: "{{iif(is_state('device_tracker.spp_39', 'not_home'), '不在家', '在家')}}"
    - content: "{{iif(is_state('device_tracker.l5pro', 'not_home'), '不在家', '在家')}}"
    - entity: sensor.keting_wendu

```

**注意：**`scroll: true` 时需要设置 `style.height`（默认 35px），否则内容不会溢出也就不会滚动。

## 参数说明

| 参数         | 类型      | 默认值   | 说明                                                    |
|------------|---------|-------|-------------------------------------------------------|
| style      | array   | 无     | 自定义样式，格式同卡片 style，支持 width、height、background 等 CSS 属性 |
| scroll     | boolean | false | 是否自动滚动。true 时多条内容逐条向上滚动，无缝循环                          |
| items      | array   | []    | <b>必填</b> ，公告内容数组，每项为 content 或 entity                |
| tap_action | object  | 无     | 统一点击动作。该动作对所有未设置逐条 tap_action 的 item 生效。优先级低于         |

| 参数 | 类型 | 默认值 | 说明                              |
|----|----|-----|---------------------------------|
|    |    |     | items[].tap_action (详见下方"点击动作") |

## items 配置

| 类型   | 参数                   | 说明                                          |
|------|----------------------|---------------------------------------------|
| 模板文本 | content: "模板字符串"     | 使用 Jinja2 模板语法，支持所有 _TemplateEngine 内置函数    |
| 实体状态 | entity: entity_id    | 读取实体当前状态值，自动拼接 unit_of_measurement          |
| 后端渲染 | server_explain: true | 搭配 content 使用，将模板发送到 HA 后端渲染，支持完整 Jinja2 语法 |
| 点击动作 | tap_action           | object                                      |

## 模板函数参考

公告栏 content 中可使用以下模板函数

(由 \_TemplateEngine 提供):

| 函数            | 用法                                                     | 说明                         |
|---------------|--------------------------------------------------------|----------------------------|
| states()      | {{states('sensor.temp')}}}                             | 读取实体状态值                    |
| state_attr()  | {{state_attr('weather.x', 'forecast.0.temperature')}}} | 读取实体属性，支持点号路径导航和数组索引       |
| is_state()    | {{is_state('light.x', 'on')}}}                         | 判断实体状态是否为指定值，返回 true/false |
| state_default | {{state_default('sensor.x                              | 读取实体状态，                    |



| 函数      | 用法                                              | 说明                                                   |
|---------|-------------------------------------------------|------------------------------------------------------|
| lt()    | ', '离线'))}}                                     | unavailable/unknown 时返回默认值                           |
| float() | {{float(states('sensor.x'), 0)}}                | 转换为浮点数，可指定默认值                                        |
| int()   | {{int(states('sensor.x'), 0)}}                  | 转换为整数，可指定默认值                                         |
| iif()   | {{iif(is_state('light.x', 'on'), '开启', '关闭'))}} | 内联条件表达式                                              |
| now()   | {{now().hour}}:{{now().minute}}                 | 当前时间属性：<br>year/month/day/hour/minute/second/weekday |

也支持 `{% set %}`、`{% if %}`/`{% else %}`/`{% endif %}` 等控制语法。

## 嵌套属性读取

`state_attr()` 支持点号路径导航和数组索引，例如：

```
# 读取天气实体的预报数组第一项的温度
- content: "{{state_attr('weather.sha_yu_lei_de_jia', 'forecast.0.temperature')}}"°C"

# 读取嵌套属性
- content: "{{state_attr('sensor.x', 'data.0.name')}}" "
```

## 走后端解析模式

```
notice:
  items:
    - content: >-
      {% if states('sensor.temp') | float > 30 %}高温{% elif
states('sensor.temp') | float > 20 %}舒适{% else %}低温{% endif %}
      server_explain: true    # ← 加这个字段，走后端渲染
```

```
- content: "{{ states('sensor.humidity') }}" # 简单模板，无需后端
```

## 内联样式支持 (HTML) v4.0.0 新增

content 内容支持标准的 `<span style="...">` 标签实现内联样式渲染，适用于需要高亮显示部分信息的场景（如余额数字标色）。

- 只放行 `<span style="...">` 标签，其他 HTML 标签自动剥离；
- 支持嵌套 `<span>`
- 支持所有 CSS 属性：color、background、font-weight、font-size、border 等；
- 事件属性（onclick、onerror 等）自动过滤，安全无 XSS 风险；
- server\_explain: true 模式下也同样支持。

```
notice:
  style:
    - width: 100%
    - height: 25px
  scroll: true
  items:
    - content: >-
      {% set sensor = 'sensor.tian_ran_qi_2_new_2' %}
      {% set balance = states(sensor) | float(0) %}
      {% set daylist = state_attr(sensor, 'daylist') %}
      天然气余额: <span style="background: #006e54;
color:#fdeff2">{{ balance }} 元</span>,
      {%- if daylist and daylist | length > 0 %}
      {%- set e_gas_list = daylist | map(attribute='e_gas') |
map('float') | list %}
      {%- set avg_e_gas = e_gas_list | sum / e_gas_list | length %}
      {%- set days_left = (balance / avg_e_gas) | round(0) if
avg_e_gas > 0 else 0 %}
      可用 {{ days_left }} 天, 每天用气 {{ avg_e_gas | round(2) }} 方
```

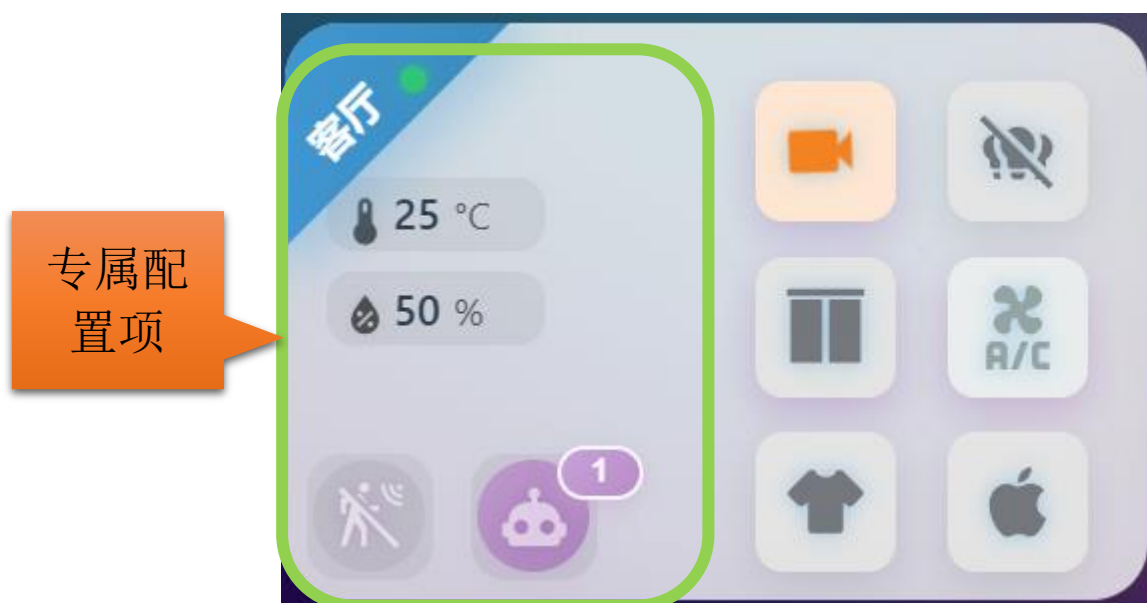
```
{%- else %}  
    暂无历史数据无法预测  
{%- endif %}  
server_explain: true
```



## 2. 普通模式专属配置项

普通模式（无 head: true）为左侧显示房间名称和传感器数据，右侧显示设备操作按钮，适合单房间展示。

专属配置项有：heartbeat\_packet（心跳包）、entities（传感器）、person（人在）、automation（自动化）。



### ● heartbeat\_packet

heartbeat\_packet 即“心跳包”，为可选配置项。

主要功能是当用户打开 HA 时，执行自动化，比如用户打开 HA 页面，自动打开某个开关。没有该需求请删除该配置项，如果要配置，仅需要在任意一张非头部卡片配置即可。

实现方式：定时向实体写入值（间隔 x 秒后实体值加 1），当配置了 heartbeat\_packet 时，在 room-name 后面显示一个小绿色呼吸点。写入失败时显示红色呼吸，成功时绿色呼

吸。

后端可以通过自动化实现很多好玩儿的功能，比如当用户打开 HA 页面，自动开启/关闭防火墙等操作，需要用户根据自己的想法来实现对应的功能。

### 配置示例：

```
heartbeat_packet:
  entity: sensor.room_card_heartbeat # 接收心跳值的实体（必须是数字型）
  interval: 10 # x 秒写入一次（建议 10-60 秒）
  whether_send: switch.heartbeat_enable # 开关实体控制是否发送（on=发送）
  show_breathe: true # 是否在房间名称旁边显示一个呼吸点
```

## ● person

person 即“人在”，为可选配置项。

以户型图、时间轴的方式显示人在每个房间的记录，可以通过拖动时间轴来查看历史时刻人在每个房间的详细记录，通过点击房间可以查看单个房间的详细信息。

其中房间坐标和温湿度公用一套坐标，与概览中的房间坐标是同一个文件。



红框中为人在图标，房间无人则为灰色，有人则为紫色，

点击该图标弹出窗口，点击每个房间弹出该房间的历史记录，点击时间轴也可以弹出该房间的历史活动记录，只是展示方式不一样。

### 配置示例：

```
person:
- main_entity: binary_sensor.ciwo_renzai      #主实体，会在顶部显示一个按钮
  room_point from: /local/point.json          #房间坐标点文件位置
  corner_radius: 8                            # 房间圆角大小
  room_name_font_size: 16                     # 房间名字体
  person icon:                                # 有人图标自定义（可选）
    size: 30px                                # 图标大小
    top: 20px                                 # 向下偏移
    left: 10px                                # 向右偏移
    color: rgb(39, 174, 96)                   # 图标颜色
  room_name_font_size: 15px
  popup_position: center                      #弹出窗口相当于克隆按钮的位置（通用参数详细讲解）
width: 360px
rooms:
  客厅: switch.giot_cn_1126348886_v64ksm_on_p_10_1
  餐厅: binary_sensor.canting_renzai
  厨房: binary_sensor.chufang_you_ren
  儿童房: binary_sensor.ertongfang_you_ren
not_show: 厨房,儿童房                      #不显示指定房间的活动记录
entities:                                    #人在传感器的设备的的其他设置
- entity: sensor.ciwo_renzai_juli
  name: 距离
  icon: mdi:ruler
  type: sensor
- entity: switch.ciwo_renzai_led
  name: LED
  icon: mdi:alarm-light
  type: switch
```

## ● automation

automation 即“自动化”，为可选配置项。

主要用来管理该房间的自动化任务，为可选配置项。



红色框内就是自动化管理的功能入口，点击会弹出自动化管理窗口（右图所示），配置代码如下：

### 配置示例：

```
automation:
- entity: automation.xxxxxxxx #自动化实体
  popup_position: clone_button_down #弹出窗口相当于克隆按钮的位置（通用参数详细讲解）
  name: API 安全网关：动态授权
  width: 400px
  icon: mdi:lan-connect
  on_icon: mdi:lan-check
  off_icon: mdi:lan-disconnect
  on_color: "#f58220"
  off_color: "#74787c"
  show_history: input boolean.show_history #是否弃用点击图标弹出历史记录，可以是实体，也可以是 true/false
```



## ● entities

entities 即“传感器”，为可选配置。

建议用来显示环境类数据显示，如温湿度、pm2.5 等，支持单个点击动作、整体定义点击动作。如下红框内为显示功能，点击弹出详细信息页面如右图。



注意：弹出窗口中的卡片是我单独制作的温湿度卡片，如有需要，请前往 <https://bbs.hassbian.com/thread-31171-1-1.html> 下载温湿度卡片，即可享受该卡片带给你的震撼的视觉体验（带有平面/3D 视角，同时支持 api 数据源），该卡片已适配房间卡的主题，同时该卡片中使用的坐标数据和概览中使用的是同一套坐标数据。

配置示例：

```

entities:
  - entity: sensor.keting_wendu          #
    type: temperature
    icon: mdi:thermometer
  - entity: sensor.keting_shidu
    type: humidity
    icon: mdi:water-percent
entities_tap_action:
  action: popup_card
  popup_position: clone_button_down    #弹出窗口相当于克隆按钮的位置（通用参
数详细讲解）
card:
  type: custom:wenshidu-card          #卡片配置参数见论坛
  room_point_from: /local/point.json
  show_3dtemperature_section: true
  show_temperature_section: true
  show_humidity_section: true
  layout: normal
  card_width: 440px
  popup_modal_width: 420px
  theme: input_select.theme
  dark_light_theme: dark,light
  default_main_room: 客厅
  show_title: true
  show_room_data: true
  show_room_label: false
  corner_radius: 5
  main_entities:
    客厅:
      temperature: sensor.keting_wendu

```

### 3. 程序扩展坞 (Dock) v5.0.0 新增

程序扩展坞是一个从屏幕右侧滑入的快捷启动栏，支持复用头部模式的全部按钮类型和动作系统。



## 基础配置

在卡片配置中添加 dock 数组：

```
type: custom:room-elves-card
dock:
  - name: api
    primary: API
    entity: input_boolean.api_gateway_man_close
    on_icon: mdi:lan-connect
    off_icon: mdi:lan-disconnect
    on_color: "#007b43"
    off_color: "#b94047"
    tap_action:
      action: toggle
  - name: HA-TCP
    primary: HA-TCP
    entity: switch.nps_server
    on_icon: mdi:lan-connect
    off_icon: mdi:lan-disconnect
    tap_action:
      action: toggle
```

## 支持的按钮类型

dock 条目支持头部模式的所有按钮类型，用法与 buttons 配置完全一致：

| 类型                 | 说明     | 示例           |
|--------------------|--------|--------------|
| 无 type             | 普通开关按钮 | 默认 toggle 动作 |
| type: light        | 灯光按钮   | 显示灯光颜色/状态    |
| type: curtain      | 窗帘按钮   | 显示窗帘开关状态     |
| type: sensor       | 传感器按钮  | 显示传感器数值      |
| type: scene_mode   | 情景模式按钮 | 一键切换场景       |
| type: dynamic_icon | 动态图标   | 支持规则匹配和动画    |

```
dock:
  - type: light
    name: 客厅灯
    entity: light.living_room
  - type: curtain
    name: 窗帘
    entity: cover.curtain
  - type: scene_mode
    name: 离家
    scenes:
      - name: 全部关闭
        actions:
          - service: light.turn_off
```

|                           |
|---------------------------|
| target:<br>entity_id: all |
|---------------------------|

### Primary 文本

dock 条目支持 primary 字段，在图标下方显示文本，支持 Jinja2 模板：

|                                                                                                                                       |
|---------------------------------------------------------------------------------------------------------------------------------------|
| dock:<br>- name: api<br>primary: "{{ states('input_boolean.api_gateway_man_close') }}"<br>entity: input_boolean.api_gateway_man_close |
|---------------------------------------------------------------------------------------------------------------------------------------|

### 右上角三角状态指示器

dock 条目会自动在按钮右上角显示一个三角状态指示器。

**自动模式：** 条目有 entity 时自动显示实体状态的缩写：

| 实际状态         | 三角显示    |
|--------------|---------|
| on           | 开       |
| off / closed | 关       |
| open         | 开       |
| home         | 家       |
| not_home     | 离       |
| unavailable  | 不显示     |
| 短文本 (≤4 字)   | 原文      |
| 长文本          | 截断加 ... |

**手动模式：**通过 `icon_text` 自定义显示内容，支持 Jinja2 模板：

```
dock:
- name: api
  entity: input_boolean.api_gateway_man_close
  icon_text: "{{ '开' if is_state('entity_id', 'on') else '关' }}"
```

三角颜色自动跟随实体的 `on_color` / `off_color`。

## 触发按钮位置

通过 `dock_trigger_position` 调整右侧触发按钮的垂直位置：

```
dock_trigger_position: center    # top / center (默认) / bottom, 或 "30%" 等百分比
dock:
- name: api
  ...
```

## 交互方式

- 点击触发按钮：打开/关闭扩展坞
- 点击扩展坞内按钮：执行动作后自动关闭
- 点击扩展坞外部区域：自动关闭扩展坞

## 4. 头部多屏模式 v5.0.0 新增

当 `head: true` 且按钮数量超过一页时，可以通过 `buttons_2`、`buttons_3`…… 等配置将按钮分到多个页面，通过横向滑动切换页面，底部圆点指示器显示当前页位置。



```
head: true
head_columns: 6                # 第 1 页列数
head_columns_2: 4              # 第 2 页列数 (可选, 默认继承 head_columns)

buttons:                        # 第 1 页按钮
- type: lights
  card:
    - entity: light.living_ceiling
- type: ac
  card:
    - entity: climate.living

buttons_2:                      # 第 2 页按钮
- type: curtain
  entity: cover.living_curtain
- type: media
  entity: media_player.tv
  row_column: '1,1-3'          # 第 2 页也支持 row_column
```

配置说明:

| 配置项            | 类型     | 默认值            | 说明                |
|----------------|--------|----------------|-------------------|
| buttons        | array  | 必填             | 第 1 页按钮列表         |
| buttons_2      | array  | 无              | 第 2 页按钮列表         |
| buttons_3      | array  | 无              | 第 3 页按钮列表（支持无限多页） |
| head_columns   | number | 6              | 第 1 页网格列数         |
| head_columns_2 | number | 同 head_columns | 第 2 页独立列数         |

### 多屏模式特性：

| 特性      | 说明                                   |
|---------|--------------------------------------|
| 滑动切换    | 横向整屏滑动，滚动捕捉（scroll-snap），支持触屏拖拽和鼠标滚轮 |
| 底部指示器   | 圆点显示当前页位置，点击圆点可直接跳转到对应页              |
| 每页独立列数  | 通过 head_columns_N 为每页设置不同网格列数        |
| 按钮排列    | 每页按钮独立排列，row_column 分别生效             |
| 概览栏/公告栏 | 仅在首页（Page 0）显示，切换页面时随首页一起滑动消失        |
| 页内按钮更新  | 实时状态更新覆盖所有页面，翻页后按钮状态已是最新             |



## 5. 其他配置项

主要是卡片级别的配置项目，常位于卡片的顶层。

### (1) 禁止首页滚动

锁定首页不滚动，下拉炒作三分之二屏幕时接触锁定。

```
prohibit_homepage_scroll: true # 始终锁定页面滚动  
prohibit_homepage_scroll: input boolean.lock_scroll # 用实体控制 (on=锁定)
```

(2) 性能模式 (performance\_mode)

控制卡片的渲染性能策略，适用于不同性能的设备。

```
performance_mode: auto      # 自动检测（默认）
performance_mode: desktop   # 强制桌面模式（全性能）
performance_mode: mobile    # 强制移动模式（低性能优化）
```

| 模式      | 说明                               |
|---------|----------------------------------|
| auto    | 自动检测设备类型。桌面设备全性能渲染，移动设备自动启用低性能优化 |
| desktop | 强制桌面模式，所有动画和渲染效果全部启用             |
| mobile  | 强制移动模式，减少动画帧数、禁用部分重渲染、优化滚动性能     |

### (3) 外部 API 配置 v4.0.0 新增

主要用来查询历史数据，即 HA 没有历史记录时会自动到 API 处拉取历史记录，此功能需要我制作的“HA 数据统一存储系统”支持。只需要在每张卡片顶层增加 api\_base\_url 和 key 配置项接口实现该卡片下的实体历史数据查询，涉及到历史数据的情况下，日期选择器的日历单元格中的“A”表示 api 中有数据，点击即可查看，“H”表示 HA 中有数据，点击会使用 HA 的历史数据。



```
type: custom:room-elves-card
room_name: 客厅
theme: input_select.theme
dark_light_theme: dark,light
show_animation: input_boolean.show_room_animation
prohibit_homepage_scroll: input_boolean.prohibit_homepage_scroll
api_base_url: /api/ha_data_store/ #api 地址，不能改变
key: 27d8e81de2f84xxxxxxxxxxxxxxxxxxxxxx #key，从集成中获取，集成会自动生成
style:
  - class: room-card
    height: 165px
    width: 100%
```

注意使用此功能会有数据泄露的风险，如 key 泄露、黑客

攻破等导致数据泄露由用户自行负责，与该功能的开发者无关，使用此功能即表示用户自行承担风险，由此造成的数据泄露等责任与开发这无关，请自行斟酌!!!

建议配合心跳包使用，这样就能自动控制只有在 app 打开时链路打开，当 app 关闭或者 app 后台后自动关闭链路，自动化代码如下：

```
alias: 0-安全网关：综合逻辑闭环控制（V8-瞬时同步）
description: >
  【逻辑摘要】 1. 瞬时联动：手动开关拨到 ON 立即开网关，拨到 OFF 立即关网关并清零。 2. 心跳维持：在 ON 状态下，持续的心跳包会重置 20s 关机倒计时。 3. 异常兜底：若手动为 ON 但长久无心跳，则自动关闭网关并清零。
triggers:
  - entity_id: input_number.api_gateway_heartbeat_counter
    trigger: state
  - entity_id: input_boolean.api_gateway_man_close
    trigger: state
actions:
  - choose:
    - alias: 手动关闭逻辑
      conditions:
        - condition: state
          entity_id: input_boolean.api_gateway_man_close
          state: "off"
      sequence:
        - target:
            entity_id: switch.hashu_ju_tong_yi_cun_chu_xi_tong_api_access
            action: switch.turn_off
        - if:
            - condition: template
              value_template: >-
                {{ states('input_number.api_gateway_heartbeat_counter') |
                  float > 0 }}
            then:
              - target:
                  entity_id: input_number.api_gateway_heartbeat_counter
                  data:
                    value: 0
                  action: input_number.set_value
    - alias: 手动开启与心跳监控逻辑
      conditions:
        - condition: state
          entity_id: input_boolean.api_gateway_man_close
          state: "on"
      sequence:
        - target:
            entity_id: switch.hashu_ju_tong_yi_cun_chu_xi_tong_api_access
            action: switch.turn_on
        - wait_for_trigger:
            - entity_id: input_number.api_gateway_heartbeat_counter
              trigger: state
            timeout: "00:00:20"
            continue_on_timeout: true
        - if:
            - condition: template
              value_template: "{{ wait.trigger == none }}"
```

```
    then:
      - target:
        entity_id:
switch.hashu_ju_tong_yi_cun_chu_xi_tong_api_access
        action: switch.turn_off
      - wait_for_trigger:
        - entity_id:
switch.hashu_ju_tong_yi_cun_chu_xi_tong_api_access
          to: "off"
          trigger: state
        timeout: "00:00:05"
      - if:
        - condition: state
          entity_id:
switch.hashu_ju_tong_yi_cun_chu_xi_tong_api_access
          state: "off"
        then:
          - target:
            entity_id: input_number.api_gateway_heartbeat_counter
            data:
              value: 0
            action: input_number.set_value
mode: restart
```

温馨提示：不管是不是自动控制，  
input\_boolean.api\_gateway\_man\_close 开关拥有最高权限，  
可以手动切断 api 链路。

再次提示：使用此功能会有数据泄露的风险，如 key 泄露、黑客攻破等导致数据泄露由用户自行负责，与该功能的开发者无关，使用此功能即表示用户自行承担风险，由此造成的数据泄露等责任与开发这无关。请自行斟酌!!!

## 七、通用配置

### 1. 房间/头部

房间/头部是所有功能的前置条件，是所有配置的最顶层。

配置结构如下：

```
type: custom:room-elves-card
primary_text_variant: minimal          #图标下方文字的样式
theme: input_select.theme             #主题配置
dark_light_theme: dark,light          #主题亮色和暗色对应的主题名称
head_columns: 7                       #每行几个按钮，头部模式专属
head: true                            #是否是头部模式
show_animation: input_boolean.show_animation #是否显示动画，支持 true/false
或者开关实体
style:
  - class: room-card                  #样式，功能及其强大，单独解释
    height: 190px
    width: 100%
buttons:
```

### 2. 按钮（核心功能）

按钮是整个卡片的核心，是所有功能的入口，可以配置在头部或者非头部区域。配置项位于 buttons 下，不管是房间卡片还是头部均支持第二页设置（左右滑动），新增一个 buttons\_2 即可，也可以支持 3 也甚至 4 页。

buttons 是卡片最重要的配置，每个按钮可以控制一种或一组设备。buttons 支持两种格式：

格式一：标准数组格式（推荐）

```
buttons:
  - type: lights
    card:
      - entity: light.living_room
        name: 主灯
  - type: ac
    entity: climate.living
```

## 格式二：对象格式（快捷写法）

当需要将多个实体按类型快速分组时，可以键名作为 type、值作为 card 数组：

```
buttons:
  lights:
    - entity: light.living_room      # 键名作为 type
      name: 客厅灯                  # 直接写 card 数组的内容
    - entity: light.bedroom
      name: 卧室灯
  socket:
    - entity: switch.plug1          # 多个 type 可混用
      name: 插头 1
```

两种格式可以混用，卡片会自动合并处理。

### （1） 灯光组（type: lights）

用于控制一组灯（如一个房间里的所有灯）。点击自动弹出灯光控制窗口，灯光会自动识别色温、亮度、颜色的调节功能显示对应的调节 UI，另外可以配置收藏、亮度、色温、颜色，以便于快速调节。



点击吊灯图标弹出左边的历史记录窗口，在历史记录窗口中

可以选择该设备的任意日期的历史记录，点击时间轴中的“竖线”会自动跳转到对应的时间段。

多楼层时，选项卡自动转换为按钮样式。





## 配置示例：

```
buttons:
  - type: lights
    width: 430px      #弹出窗口的宽度，不配置时，自适应
    per_line: 1      #一行显示几个灯具
    popup_position: clone_button_down #弹出窗口相当于克隆按钮的位置（通用参数详细讲解）
    show_duration: input_boolean.show_duration #是否显示最后一个状态到当前时间的时长，可以是 true/false，也可以从实体处获取
    on_icon: mdi:lightbulb-group
    off_icon: mdi:lightbulb-group-off
    group_lights_by_room: input_boolean.group_lights_by_room #按照房间分组显示，可选项，可以是 true/false，也可以从实体处获取
    show_history: input_boolean.show_history #是否允许点击图标弹出该实体的历史记录，可以是 true/false，也可以从实体处获取
    tabs: input_boolean.tabs #是否按照选项卡显示，可以是 true/false，也可以从实体处获取
    tabs_by: room #选项卡的分页字段
    table_per_line: 4 #一行几个按钮，可选项
    name: 全屋灯光
    on_color: "#f58220"
    off_color: "#74787c"
    card:
      - entity: light.living_ceiling # 灯的实体 ID
        name: 天花灯 # 灯的名称
        room: 客厅 # 所属房间（用于弹窗分组）
        global_exception: true # 全关/全开/取反时跳过此实体
        # —— 收藏快捷按钮（可选，在滑块下方显示快捷圆点） ——
        collect_brightness: # 收藏亮度（百分比）
          - name: 夜灯
            brightness: 20%
          - name: 阅读
            brightness: 50%
          - name: 明亮
            brightness: 80%
        collect_colour_temperature: # 收藏色温（Kelvin）
          - name: 暖光
            colour_temperature: 2000
          - name: 自然光
            colour_temperature: 5000
          - name: 冷光
            colour_temperature: 6500
        collect_colors: # 收藏颜色（HEX 色值）
          - name: 红
            color: '#ff0000'
          - name: 绿
            color: '#00ff00'
          - name: 蓝
            color: '#0000ff'
      - entity: light.living_floor_lamp
        name: 落地灯
        room: 客厅
```

## 灯光组专用配置项：

| 配置项                  | 类型      | 默认值   | 说明                             |
|----------------------|---------|-------|--------------------------------|
| group_lights_by_room | boolean | false | 弹窗中是否按 room 字段分组显示灯光，各房间独立区域展示 |

| 配置项           | 类型      | 默认值   | 说明           |
|---------------|---------|-------|--------------|
| show_duration | boolean | false | 按钮上是否显示已开启时长 |

### card 子项配置项：

| 配置项                        | 类型      | 默认值   | 说明                                                                                   |
|----------------------------|---------|-------|--------------------------------------------------------------------------------------|
| room                       | string  | 无     | 所属房间（用于弹窗按房间分组）                                                                      |
| global_exception           | boolean | false | true 时，在弹窗中使用"全关"/"全开"/"取反"等批量操作时 <b>跳过</b> 此实体。适用于不想被批量控制的设备（如安防灯、常亮夜灯等）            |
| collect_brightness         | array   | 无     | 收藏亮度快捷按钮列表，每个子项含 name（显示名称）和 brightness（亮度百分比，如 20% 或 50）                            |
| collect_colour_temperature | array   | 无     | 收藏色温快捷按钮列表，每个子项含 name（显示名称）和 colour_temperature（色温值，单位 Kelvin，如 2000 = 暖光，6500 = 冷光） |
| collect_colors             | array   | 无     | 收藏颜色快捷按钮列表，每个子项含 name（显示名称）和 color（HEX 色值，如 '#ff0000'）                               |

### 收藏快捷按钮说明：

收藏快捷按钮显示在灯光弹窗控制面板的滑块下方，以圆点形式展示，点击即可快速切换到预设值：

- **collect\_brightness**：收藏亮度，圆点内用黄色渐变填充百分比表示亮度高低。配置后即使设备不支持亮度

调节也会显示

- **collect\_colour\_temperature:** 收藏色温，圆点颜色根据色温值自动计算（暖光偏橙、冷光偏蓝）。配置后即使设备不支持色温调节也会显示

- **collect\_colors:** 收藏颜色，圆点直接使用配置的 HEX 色值填充

三个收藏属性均可配置在 type: lights 的 card 子项上（每盏灯独立配置），也可配置在 type: light 单灯的顶层。配置在 type: lights 顶层时，会自动向下透传到所有 card 子项（子项自身未配置时使用顶层配置）。

global\_exception 同样适用于 type: socket 的 card 子项。此配置项仅在灯光/插座等聚合按钮的弹窗批量操作中生效，不影响单个按钮的点按操作。

## (2) 单个灯光

单个灯按钮，一般适用于控制带有调色温/调亮度/调颜色类的灯光，普通灯光请勿使用（普通灯光使用无类型按钮即可），点击图标开灯同时自动弹出灯光控制界面，几秒后（可设置时长）自动关闭弹出窗口。如果用户操作（如调亮）则不自动关闭，点击外部手动关闭。当灯有调色、调亮度和情景模式时，会自动打开对应的 UI，无需手动配置。

当灯光在开启状态时，可以通过配置（on\_tap\_action）来确定点击动作（关灯/弹出控制界面）。无论 on\_tap\_action 配置如何，长按 light 类型按钮（500ms）始终弹出控制面板。



配置示例：

```
buttons:
  buttons:
    - type: light
      name: 餐厅灯光
      entity: light.canting_diaodeng
      on_tap_action: adjust #灯开着时点击按钮 → 弹出控制面板调参。
```

```
popup_position: clone_button_down
on_icon: mdi:ceiling-light
off_icon: mdi:ceiling-light-outline
on_color: "#f58220"
off_color: "#74787c"
close_time: 5
width: 420px
collect_colors:
  - name: 红
    color: "#ff0000"
  - name: 绿
    color: "#00ff00"
  - name: 蓝
    color: "#0000ff"
tap_action:
  action: toggle
```

配置项说明：

**close\_time:** 表示几秒自动关闭灯光控制界面，设置为 0 则表示永不关闭。

**popup\_position:** 弹出窗口相对于按钮（克隆按钮）的位置，参照通用参数说明。

**width:** 灯光控制界面宽度。

**on\_tap\_action:** adjust，可选参数，表示灯开着时点击按钮 → 弹出控制面板调参，需要在控制面版中点击关灯。

**collect\_colors:** 收藏颜色，当有调色功能时可配置

如果不配置 close\_time 和 width 参数时，关闭时长会使用卡片内部的默认参数 3 秒和自适应宽度。

(3) 空调 (type: ac) v3.1.3 增强功能

显示单个/多个空调的状态，点击弹出空调控制面板（可调温度、模式、风速）。同时当为 type 为 ac 时，图标为动态图标，会自动根据空调的状态显示不太的图标、颜色、动画。

该卡片支持独立使用，详见 8.4。

V3.1.2 新增功能：

- 适配了主题；
- 新增配置湿度功能 (humidity\_entity)；
- 新增点击温度弹出温度、湿度、目标温度、开启记录详细信息；





配置示例：

```
buttons:
  - type: ac
    name: 全屋空调
    popup_position: clone_button_down
    width: 430px
    primary: >
      {% set value = states('sensor.kong_diao_zong_gong_lu') | float(0) %}
      {%
        if value < 1000 %}
          {{ value | round(0)}}W
        {% endif %}
      }
```

```

    {% else %}
    {{ (value / 1000) | round(1) }}kW
    {% endif %}
  card:
    - type: ac
      entity: climate.lumi_cn_875402794_mcn02
      current_temperature: sensor.ciwo_wendu #室内温度，可选，不配置时使用空调自带的温度
      power_entity: sensor.ciwo_ac_energy_yearly #用电量，可选
      power_display_entity: sensor.ciwo_ac_gonglv #功率，可选
      name: 次卧空调
      width: 400px
      features: #扩展功能，可选
        - type: toggle
          switch_entity: null
          switch_name: null
          icon: mdi:power-sleep
    - type: ac
      entity: climate.zhuwo_ac_banlv
      humidity_entity: sensor.keting_shidu #v3.1.2 新增湿度配置功能
      current_temperature: sensor.zhuwo_wendu
      power_entity: sensor.zhuwo_ac_energy_yearly
      power_display_entity: sensor.zhuwo_ac_banlv_gonglv
      icon: mdi:power-sleep
      width: 400px
      name: 主卧空调
      features:

```

空调是将原来我制作的单独空调卡片移植进来的，所以扩展配置详见 <https://bbs.hassbian.com/thread-31109-1-1.html>。后期将不在维护独立空调卡片，只更新房间精灵卡片内部组件。

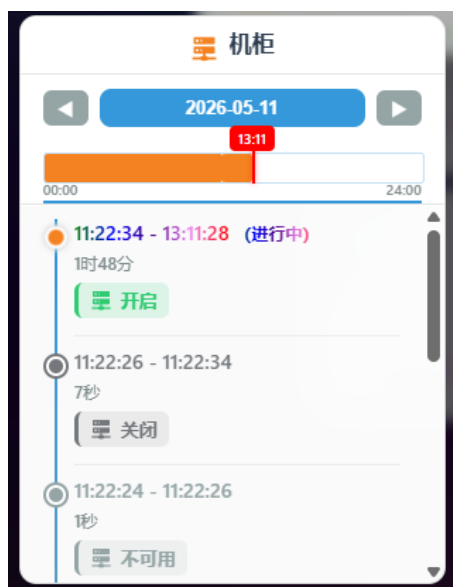


#### (4) 插座/开关 (type: socket)

插座聚合按钮，点击弹出插座控制窗口，可以选配功率实体（当配置该实体后点击该实体还会弹出插座的设置界面），同时点击图标还会弹出该设备的开启历史记录：



点击图标弹出历史记录：



点击弹出插座设置：



## 配置示例:

```
buttons:
- type: socket
  icon: mdi:power-socket-au
  width: 400px
  per_line: 1
  show_badge: true #是否显示角标（开启个数）
  popup_position: clone_button_down
  show_duration: input_boolean.show_duration
  group_sockets_by_room: true
  shwo_sankey: true
  show_history: input_boolean.show_history
  tabs: input_boolean.tabs
  tabs_by: room
  table_per_line: 7
  name: 全屋插座
  primary: >
    {% set value = states('sensor.cha_zuo_zong_gong_lv') | float(0) %} {%
if
  value < 1000 %}
    {{ value | round(0)}}W
  {% else %}
    {{ (value / 1000) | round(2) }}kW
  {% endif %}
on_color: "#f58220"
off_color: "#74787c"
card:
- entity: switch.giot_cn_1124641761_v6shsm_on_p_2_1
  name: 监控插座
  confirm: false #v4.0.0 新增，切换时不弹出确认对话框
  room: 客厅
  belong: 房间
  icon: mdi:video
  global_exception: true # 全关/全开/取反时跳过此实体
  tap_action:
    action: toggle
- entity: switch.cuco_cn_696791082_v3_on_p_2_1
  name: 投影
  room: 客厅
  belong: 房间
  icon: mdi:projector
  tap_action:
    action: toggle
entities:
- entity: sensor.cuco_cn_696791082_v3_electric_power_p_11_2
  type: sensor
  name: 功率
  icon: mdi:lightning-bolt
- row: 1-1 #点击功率弹出窗口的配置项
  title: 信息中心
  items:
    - entity: sensor.cuco_cn_696791082_v3_fault_p_2_3
      type: sensor
      name: 通知
      icon: mdi:information-variant-circle
    - entity: sensor.cuco_cn_696791082_v3_temperature_p_12_2
      type: sensor
      name: 温度
      icon: mdi:thermometer
    - entity: sensor.cuco_cn_696791082_v3_electric_power_p_11_2
      type: sensor
      name: 功率
```

```

        icon: mdi:lightning-bolt
- row: 2-1
  title: 用电中心
  items:
    - entity: sensor.touyingyi_day
      type: sensor
      name: 日用电
      icon: mdi:calendar-today
      layout: mini
    - entity: sensor.touyingyi_month
      type: sensor
      name: 月用电
      layout: mini
      icon: mdi:calendar-range
    - entity: sensor.touyingyi_year
      type: sensor
      name: 年用电
      layout: mini
      icon: mdi:calendar-month
- row: 3-1
  title: 控制中心
  items:
    - entity: light.cuco_cn_696791082_v3_s_13_indicator_light
      type: switch
      name: 指示灯
      icon:
        "on": mdi:alarm-light
        "off": mdi:alarm-light-off
    - entity:
switch.cuco_cn_696791082_v3_physical_controls_locked_p_7_1
      type: switch
      name: 童锁
      icon:
        "on": mdi:lock
        "off": mdi:lock-open-variant
- row: 4-1
  title: 实时功率
  items:
    - type: chart_line
      entity: sensor.cuco_cn_696791082_v3_electric_power_p_11_2
      height: 120px
      icon: mdi:cached
      name: 实时功率
      update_interval: 3
      line:
        min: 0
        max: 500
        hours_to_show: 24

```

## (5) 耗材 (type: consumables)

显示电池电量、滤芯寿命等耗材的剩余量。当低于设定阈值时自动告警，角标显示的是最低值。

电池图标会根据剩余量自动变化（满电→绿色，低电→红色），同时按钮上显示最低剩余量数字。



注意：如果是电量类，请在名称中包含“电”字，系统会根据名称来自动选取图标（动态图标，根据电量设置图标和图标颜色），首页的图标也是动态图标，无需设置，参数解释：

group：分组名称，系统会根据分组名称自动分组；

per\_line：每行几个按钮，自行设置；

popup\_position：可选，弹出窗口定位，不设置时自动选择合适的位置，popup\_position的具体参数，后面详细解释

width：可选，弹出窗口的宽度，自行设置，不设置时，自动根据内容设置宽度。

tabs\_by：选项卡字段

### 配置示例：

```
buttons:
  - type: consumables
    name: 全屋耗材
    popup_position: clone_button_down
    width: 400px
    per_line: 1
    tabs: input_boolean.tabs
    tabs_by: group
    card:
      - entity: sensor.zhuwo_wenshidu_dianliang
        name: 主卧
        group: 温度计电量
      - entity: sensor.keting_wenshidu_dianliang
        name: 客厅
        group: 温度计电量
      - entity: sensor.canting_wenshidu_dianliang
        name: 餐厅
        group: 温度计电量
```

## (6) 快捷操作 (scene\_mode)

快捷操作按钮用于一键执行一组预定义的操作（如"离家"关闭所有灯、"回家"打开指定灯），支持气泡选择、直接执行、执行进度展示和状态验证。

点击按钮后弹出气泡列表展示所有快捷操作，点击快捷操作执行对应的操作列表。执行过程中显示实时进度面板，验证每个操作是否成功。

该卡片可以独立使用，独立使用方法见“八”章节。



## 配置示例:

```
buttons:
- type: scene_mode
  primary: 快捷操作
  width: 400px
  entity: input_text.scene_mode
  icon: mdi:tune
  icon_color: "#9e579d"
  mode: bubble
  show_badge: false
  verify_timeout: 10
  scenes:
  - name: 离家模式
    icon: mdi:home-export-outline
    confirm: true
    fold: true
    expand_all: true
    actions:
    - entities:
      - 客厅_大灯: light.keting_dadeng
      - 客厅_玄关灯: light.keting_xuanguan
      - 客厅_灯带: light.keting_dengdai
      - 厨房_大灯: light.chufang
      - 主卧_吸顶灯: light.zhuwo_xidingdeng
      - 主卧_床头南灯: light.zhuwo_chuangtounan
      - 主卧_床头北灯: light.zhuwo_chuangtoubai
      - 次卧_吸顶灯: light.ciwo_xidingdeng
      - 儿童房_吸顶灯: light.ertongfang_xidingdeng
      value: "off"
    name: 全屋灯光
  - entities:
      - 儿童房_空调: climate.xiaomi_m6_00c5_air_conditioner
      - 次卧_空调: climate.lumi_mcn02_f61f_air_conditioner
      - 主卧_空调: climate.dc330d3430f6_climate
      - 客厅_空调: climate.keting_ac_keting_ac
      name: 全屋空调
      value: "off"
  - entities:
      - 小卫_插座: switch.xiaoweishengjian_chazuo
      - 儿童房_床头: switch.ertongfang_chuangtouchazuo
      - 次卧_床头: switch.ciwo_chuangtouchazuo
      - 厨房_烧水壶: switch.chufang_chazuo_shaoshuihu
      - 厨房_热水器: switch.chufang_reshuiqi
      - 厨房_空气炸锅: switch.chufang_kongqizhaguo
      - 厨房_油烟机: switch.chufang_youyanji
      - 厨房_电饭锅: switch.chufang_dianfangguo
      - 厨房_微波炉: switch.chufang_weibolu
      - 厨房_微波炉: switch.chufang_weibolu
      name: 插座
      value: "off"
```

## 顶层配置项

| 配置项     | 必填 | 类型     | 默认值  | 说明             |
|---------|----|--------|------|----------------|
| type    | ☑  | string | —    | 固定为 scene_mode |
| primary | ✗  | string | 情景模式 | 按钮下方文字         |

| 配置项            | 必填 | 类型      | 默认值      | 说明                                           |
|----------------|----|---------|----------|----------------------------------------------|
| icon           | ✗  | string  | mdi:tune | 按钮图标                                         |
| icon_color     | ✗  | string  | #3498db  | 按钮图标颜色，设置后按钮背景和光晕也会跟随变化                      |
| mode           | ✗  | string  | bubble   | 执行模式：bubble（气泡选择）/ direct（直接执行第一个情景）         |
| entity         | ✗  | string  | 无        | 执行历史记录实体（需为 input_text 类型，详见下方“执行历史”）        |
| show_badge     | ✗  | boolean | true     | 是否显示角标（情景数量）                                 |
| verify_timeout | ✗  | number  | 10       | 执行后验证状态的超时秒数                                 |
| width          | ✗  | string  | 400px    | 气泡选择面板和执行进度面板的宽度（如 400px、500px），移动端高度不超过 60% |
| fold           | ✗  | boolean | false    | 动作列表是否默认展开。true 时展开所有情景的动作列表，false 或不配置时折叠   |
| scenes         | ✓  | array   | []       | 情景列表                                         |

## 执行模式（mode）

| 模式     | 说明                                   |
|--------|--------------------------------------|
| bubble | 点击按钮弹出气泡列表，用户选择情景后执行。多个情景时默认使用此模式    |
| direct | 点击按钮直接执行第一个情景，不弹出气泡。适合只有一个情景的场景      |
| 不配置    | 自动判断：只有 1 个情景时为 direct，多个情景时为 bubble |

## scenes 子项配置

每个情景（scene）支持以下字段：

| 配置项  | 必填 | 类型     | 默认值                 | 说明                  |
|------|----|--------|---------------------|---------------------|
| name | ✓  | string | 情景                  | 情景名称（显示在气泡列表和进度面板中） |
| icon | ✗  | string | mdi:palette-outline | 情景图标（显示在气泡列表中）      |



| 配置项        | 必填 | 类型      | 默认值       | 说明                                                      |
|------------|----|---------|-----------|---------------------------------------------------------|
| icon_color | ✗  | string  | 无         | 情景图标颜色，设置后图标背景和阴影也会跟随变化。未配置时使用默认蓝色                      |
| confirm    | ✗  | boolean | false     | 执行前是否弹出确认对话框                                            |
| fold       | ✗  | boolean | 顶层 fold 值 | 动作列表是否默认展开。<br>true 时展开，false 时折叠；<br>未配置时回退到顶层 fold 设置 |
| actions    | ✓  | array   | []        | 操作列表                                                    |

## actions 子项配置

每个操作（action）支持两种模式：

**单实体模式：**控制一个实体

```
actions:
  - entity: light.bed_light      # 目标实体 ID
    value: "off"                 # 目标状态值（如 "on"、"off"、"25" 等）
    name: 床头灯                 # 操作名称（显示在进度面板中）
    delay: 3                     # 延时执行秒数（可选，默认 0=立即执行）
    service_data:                # 附加服务数据（可选，传递给 HA 服务调用）
      kelvin: 3000               value: "off"
```

**批量模式：**将多个实体设为同一状态，entities 支持三种格式：

**格式一：**对象格式（推荐） — 键名为自定义名称，在进度面板中替代 entity ID 显示。

```
actions:
  - entities:                    # 键名=自定义名称，键值=实体 ID
    客厅灯: light.keting_dadeng
    客厅玄关灯: light.keting_xuanguan
    客厅灯带: light.keting_dengdai
    value: "off"                 # 所有实体的目标状态值
    name: 全屋灯光               # 操作分组名称
```

**格式二：**列表式键值对格式 — YAML 语法，效果与对象格式相同。

```
actions:
  - entities:
    - 客厅灯: light.keting_dadeng
    - 客厅玄关灯: light.keting_xuanguan
    - 客厅灯带: light.keting_dengdai
    value: "off"
```

|            |
|------------|
| name: 全屋灯光 |
|------------|

**格式三：数组格式（向后兼容）** — 纯实体 ID 数组，进度面板中显示 entity ID 或 HA friendly\_name。

|                          |             |
|--------------------------|-------------|
| actions:                 |             |
| - entities:              | # 纯实体 ID 数组 |
| - light.ciwo_xidingd     |             |
| - light.ertongfang_xidin |             |
| value: "off"             |             |
| name: 其他灯光               | # 操作名称      |
| delay: 0                 | # 延时秒数（可选）  |

**进度面板显示效果：**

| entities 格式   | 进度面板中的名称显示                         | 示例                            |
|---------------|------------------------------------|-------------------------------|
| 对象格式 / 列表式键值对 | 使用键名（自定义名称）                        | 客厅灯 on→off 成功                 |
| 数组格式          | 使用 HA friendly_name，无则显示 entity ID | light.keting_dadeng on→off 成功 |
| 单实体模式（entity） | 用 name > friendly_name > entity ID | 床头灯 on→off 成功                 |

进度面板中每项显示格式为：**名称 + 当前状态→目标状态 + 执行结果**。例如 客厅灯 on→off 成功、客厅灯 on→off 已是关闭，无需执行。

| 配置项      | 必填 | 类型             | 默认值 | 说明                            |
|----------|----|----------------|-----|-------------------------------|
| entity   | ✗  | string         | 无   | 单实体模式的目标实体 ID（与 entities 二选一） |
| entities | ✗  | array / object | 无   | 批量模式的实体列表，支持三种格式（见上方）         |
| value    | ✓  | string         | 无   | 目标状态值（如 "on"、"off"、"25" 等）    |
| name     | ✗  | string         | 无   | 操作名称（显示在进度面板中，批量模式下为分组名称）     |
| delay    | ✗  | number         | 0   | 延时执行秒数，0 = 立即执行               |

| 配置项          | 必填 | 类型      | 默认值  | 说明                                          |
|--------------|----|---------|------|---------------------------------------------|
| service_data | ✗  | object  | 无    | 附加服务数据（传递给 HA 服务调用，如 kelvin、brightness 等）   |
| select       | ✗  | boolean | true | 动作复选框是否默认选中。false 时该动作默认不勾选（执行时跳过），不配置时默认选中 |

entity 和 entities 至少配置一个。对象格式 entities 的键名优先级高于 name，会作为每个实体在进度面板中的独立显示名称。

### select 参数详解：

每个 action 在气泡列表展开后会显示为复选框，用户可以取消勾选不想执行的动作。select 控制该复选框的默认状态：

- select: true（默认）— 复选框默认勾选，执行情景时会执行该动作
- select: false — 复选框默认不勾选，执行情景时跳过该动作（用户可手动勾选）

适用场景：某些动作是可选的（如"关闭空调"情景中，"关闭加湿器"是可选动作），设为 select: false 后默认不执行，但用户仍可在展开面板中勾选执行。

```

scenes:
- name: 离家
  actions:
    - entity: light.bed_light
      value: "off"
      name: 床头灯
      select: true # 默认勾选（或不配置，效果相同）
    - entity: fan.bedroom
      value: "off"
      name: 卧室风扇
      select: false # 默认不勾选，执行时跳过，用户可手动勾选

```

|                                                                                     |                 |
|-------------------------------------------------------------------------------------|-----------------|
| <pre> - entity: vacuum.roborock   value: "start"   name: 扫地机   select: false </pre> | # 默认不执行，离家时可选清扫 |
|-------------------------------------------------------------------------------------|-----------------|

select 仅控制复选框的默认状态，用户仍可在气泡展开面板中手动勾选/取消。如果情景配置了 mode: direct（直接执行，不弹出气泡），select: false 的动作将始终被跳过。

## 支持的实体类型与服务自动推断

卡片会根据实体 ID 的域前缀（如 light.、climate.）自动推断应调用的 HA 服务，无需手动指定服务名。以下是各实体类型支持的 value 值与对应服务：

| 实体域                    | value 值                                                | 调用服务                    | 说明                                                       |
|------------------------|--------------------------------------------------------|-------------------------|----------------------------------------------------------|
| light                  | "on" / "true"                                          | light.turn_on           | 开灯，service_data 可传 brightness、kelvin、color_temp、effect 等 |
| light                  | "off" / "false"                                        | light.turn_off          | 关灯，自动清理 kelvin/brightness 等参数                            |
| light                  | 其他值                                                    | light.turn_on           | 作为亮度等参数                                                  |
| switch / input_boolean | "on" / "off"                                           | turn_on / turn_off      | 开关切换                                                     |
| climate                | "off"                                                  | climate.set_hvac_mode   | 关闭空调 (hvac_mode: "off")，自动清理 temperature/fan_mode        |
| climate                | "on" / "true"                                          | climate.turn_on         | 开启空调                                                     |
| climate                | 数值（如 "26"）                                             | climate.set_temperature | 设置温度，temperature 取数值                                     |
| climate                | 模式字符串（如 "cool"、"heat"）<br>+ service_data 含 temperature | climate.set_temperature | 同时设置 hvac_mode + temperature                             |

| 实体域                   | value 值                  | 调用服务                            | 说明                                               |
|-----------------------|--------------------------|---------------------------------|--------------------------------------------------|
| climate               | 模式字符串<br>(无 temperature) | climate.set_hvac_mode           | 仅设置模式                                            |
| climate               | service_data 含 fan_mode  | 额外调用 climate.set_fan_mode       | fan_mode 自动拆分为独立服务调用；关闭模式 (value: "off") 时跳过风速设置 |
| cover                 | "open"                   | cover.open_cover                | 打开遮盖                                             |
| cover                 | "closed" / "close"       | cover.close_cover               | 关闭遮盖                                             |
| cover                 | 数值                       | cover.set_cover_position        | 设置位置百分比                                          |
| fan                   | "on" / "off"             | fan.turn_on / fan.turn_off      | 风扇开关                                             |
| fan                   | 数值                       | fan.set_percentage              | 设置风速百分比                                          |
| media_player          | "on" / "off"             | media_player.turn_on / turn_off | 媒体设备开关；toggle 时 idle/paused/standby 视为开启状态       |
| media_player          | "play" / "pause"         | media_play / media_pause        | 播放/暂停                                            |
| button / input_button | 任意                       | press                           | 按压按钮                                             |
| input_select          | 任意                       | input_select.select_option      | option = value                                   |
| select                | 任意                       | select.select_option            | option = value                                   |
| input_number          | 数值                       | input_number.set_value          | 设置数值                                             |
| automation            | "on" / "off"             | automation.turn_on / turn_off   | 自动化开关                                            |
| automation            | 其他                       | automation.trigger              | 触发自动化                                            |
| vacuum                | "on" / "start" / "clean" | vacuum.start                    | 开始清扫                                             |

| 实体域    | value 值                               | 调用服务                      | 说明   |
|--------|---------------------------------------|---------------------------|------|
| vacuum | "off" / "return"<br>/ "home" / "dock" | vacuum.return_<br>to_base | 返回充电 |
| vacuum | "stop"                                | vacuum.stop               | 停止清扫 |
| vacuum | "pause"                               | vacuum.pause              | 暂停清扫 |
| script | 任意                                    | script.{实体<br>名}          | 执行脚本 |
| lock   | "locked" / "lock<br>"                 | lock.lock                 | 上锁   |
| lock   | 其他                                    | lock.unlock               | 解锁   |
| 其他域    | "on" / "off"                          | turn_on / turn_<br>off    | 通用开关 |
| 其他域    | 其他                                    | homeassistant.<br>turn_on | 通用回退 |

### media\_player 状态判定说明:

media\_player 实体有多种状态

(on/off/idle/paused/playing/standby 等), 卡片对其做了  
统一处理:

| 场景                     | 判定逻辑                                       | 说明                                                                                     |
|------------------------|--------------------------------------------|----------------------------------------------------------------------------------------|
| toggle 切<br>换          | off/unavailable/unknown 之外的<br>状态均视为"开启"   | idle (空闲)、paused (暂<br>停)、playing (播放中)、<br>standby (待机) 都算开启,<br>toggle 时会执行 turn_off |
| 验<br>证 value:<br>"on"  | 状态<br>为 on/playing/idle/paused 之一<br>即视为成功 | standby 不视为开启                                                                          |
| 验<br>证 value:<br>"off" | 状态为 off 或 standby 均视为成功                    | 部分设备 (如 Sonos、<br>Chromecast) turn_off 后状态<br>变为 standby 而非 off, 同样<br>判定为关闭成功         |

### 空调 (climate) 配置示例:

```
actions:
  # 制冷模式 + 温度 + 风速 (自动拆分为 3 次服务调用)
  - entity: climate.xiaomi air conditioner
```

```

    value: cool
    service_data:
      temperature: "26"      # → climate.set_temperature { hvac_mode:
"cool", temperature: 26 }
      fan_mode: level6      # → climate.set_fan_mode { fan_mode:
"level6" }

    # 仅设置温度
    - entity: climate.xiaomi_air_conditioner
      value: "26"            # → climate.set_temperature { temperature:
26 }

    # 仅设置模式
    - entity: climate.xiaomi_air_conditioner
      value: heat            # → climate.set_hvac_mode { hvac_mode:
"heat" }

    # 选择器实体
    - entity: select.ac_mode
      value: 制冷            # → select.select_option { option: "制冷" }

    # 关闭空调（使用 set_hvac_mode 而非 turn_off，自动清理
temperature/fan_mode）
    - entity: climate.xiaomi_air_conditioner
      value: "off"          # → climate.set_hvac_mode { hvac_mode:
"off" }
    # 即使 service_data 包含 temperature/fan_mode，关闭时也会自动清理，不会
报错

```

透传模式：如果 `service_data` 中包含 `_service` 字段，则跳过自动推断，直接调用指定服务：

```

actions:
  - entity: climate.xiaomi_air_conditioner
    value: cool
    service_data:
      _service: climate.set_hvac_mode  # 强制使用此服务
      # 其他字段原样传递

```

## 执行流程

- ① 用户点击按钮 → 根据模式弹出气泡列表或直接执行
- ② 展开动作列表 → 点击情景右侧的展开按钮（▼），可查看该情景包含的所有动作（actions），以复选框形式展示
- ③ 取消勾选动作 → 取消勾选不想执行的动作，仅执行剩余勾选的动作
- ④ 选择情景 → 如果配置了 `confirm: true`，先弹出确认对话框

⑤ 逐个执行操作 → 仅执行勾选的 actions，按顺序执行，支持 delay 延时

⑥ 状态验证 → 所有服务调用完成后，轮询检查实体是否达到目标状态（超时由 verify\_timeout 控制）

⑦ 显示结果 → 进度面板中每个操作显示：名称 + 状态转换 + 结果

进度面板显示格式：每项显示为 名称 当前状态→目标状态 结果

| 阶段     | 显示示例                  |
|--------|-----------------------|
| 等待中    | 客厅灯 on→off 等待中        |
| 执行中    | 客厅灯 on→off 执行中        |
| 已发送    | 客厅灯 on→off 已发送        |
| 验证成功   | 客厅灯 on→off 成功         |
| 验证失败   | 客厅灯 on→off 未达到        |
| 调用失败   | 客厅灯 on→off 失败         |
| 已是目标状态 | 客厅灯 off→off 已是关闭，无需执行 |

**动作选择：**气泡列表中每个情景右侧有一个展开按钮（▼），点击后展开该情景的动作列表。每个动作以复选框形式显示（默认全选），取消勾选后执行情景时会跳过该动作。批量模式（entities）的动作取消后，该批次所有实体都会被跳过。

**设备展开：**包含多个设备的动作（entities 数组/对象）左侧会显示一个展开按钮（►），点击后在该动作下方展开显示



所有设备名称，再次点击折叠。这有助于查看批量操作中包含的具体设备。

**按房间自动分组：**当 entities 的键名（自定义名称）中包含下划线\_时，卡片会自动按下划线前的部分作为房间名进行分组显示。下划线前为房间名，下划线后为设备名。例如键名 客厅\_大灯 会显示为房间标签"客厅"下的设备"大灯"。不含下划线的键名保持原始平铺显示（向后兼容）。

```
# 使用下划线命名自动按房间分组
actions:
  - entities:
      客厅_大灯: light.keting_dadeng
      客厅_玄关灯: light.keting_xuanguan
      客厅_灯带: light.keting_dengdai
      厨房_大灯: light.chufang
      主卧_吸顶灯: light.zhuwo_xidingdeng
      主卧_床头南灯: light.zhuwo_chuangtounan
    value: "off"
    name: 全屋灯光
```

展开后效果：



执行历史记录 (entity)

配置 entity 后（推荐使用 input\_text 类型实体，长度 255），每次执行完成会自动将执行模式、时间和结果写入该实体，并在气泡列表中显示上次执行信息。

**实体数据格式**（JSON 存储在 input\_text 的 state 中）：

```
{
  "离家": { "time": "2026-05-20T10:30:00.000Z", "result": "success",
    "detail": "全部执行成功" },
```

```
"回家": { "time": "2026-05-20T08:00:00.000Z", "result": "partial",  
  "detail": "成功 2 项, 失败 1 项" }
```

气泡列表中每个情景项额外显示:

- 上次执行时间 (如"今天 10:30"、"昨天 08:00"、  
"5/19 18:00")
- 上次执行结果标签 (绿色=全部成功, 橙色=部分失败, 红色=全部失败)

注: Home Assistant 中创建记录实体 (input\_text.xx)  
的方法自行百度。

## (7) 动态图标(type: dynamic\_icon)

这是最灵活的按钮类型。可以根据实体的不同状态，自动切换图标、颜色和动画，v5.0.0 新增轮播模式、内置规制完整 jinja2 模板支持。多条规则同时匹配时可以循环显示。

### 基础模式

```
- type: dynamic_icon
  icon: mdi:bell
  color: '#74787c'
  rules:
    - condition:
        entity: binary_sensor.front_door
        operator: '=='
        value: 'on'
        icon: mdi:door-open
        color: '#2ecc71'
        animation: shake # 可用的动画: shake, rotate, blink, breathe, jump
    - condition:
        entity: sensor.temperature
        operator: '>'
        value: '30'
        icon: mdi:thermometer-alert
        color: '#e74c3c'
        animation: blink
  loop_display: true
  display_time: 3
  show_badge: true
```

# 默认图标（无规则匹配时显示）  
# 默认颜色  
# 规则列表  
# 规则 1: 门开了  
# 规则 2: 温度>30 度  
# 多条规则匹配时循环显示  
# 每条规则显示几秒  
# 显示匹配条件的规则数量

### 条件支持的操作符:

| 操作符      | 说明   | 示例                            |
|----------|------|-------------------------------|
| ==       | 等于   | value: 'on'                   |
| !=       | 不等于  | value: 'off'                  |
| >        | 大于   | value: '30'                   |
| <        | 小于   | value: '10'                   |
| >=       | 大于等于 | value: '60'                   |
| <=       | 小于等于 | value: '20'                   |
| contains | 包含   | value: '异常'                   |
| between  | 在范围内 | value: '20,30'（表示 20 到 30 之间） |

### 复合条件 (AND/OR):

```
- condition:
  logic: and
  conditions:
```

# 所有条件必须同时满足

```
- entity: binary_sensor.door
  operator: '=='
  value: 'on'
- entity: sensor.temperature
  operator: '>'
  value: '25'
icon: mdi:door-open
color: '#e74c3c'
```

支持的条件值也可以写成数组（相当于 OR）：

```
value: ['on', 'active', 'playing'] # 匹配其中任意一个就算满足
```

### 参数说明：

**show\_badge:** 是否显示右上角的角标，该角标是根据设备状态自动显示的。

**loop\_display:** 循环显示，为 true 时会循环显示符合要求（符合 rules）的设备的图标、动画、颜色。

**display\_time:** 每个图标显示的时长，单位为秒。

**animation:** 动画类型支持：

- ◆ rotate: spin 2s 线性无限循环 - 图标 360 度持续旋转
- ◆ blink/flash: dynamic-blink 1s 缓入缓出无限循环 - 图标透明度周期性变化
- ◆ breathe: dynamic-breathe 2s 缓入缓出无限循环 - 图标大小周期性缩放
- ◆ shake: shake 2s 缓入缓出无限循环 - 图标左右晃动
- ◆ jump: dynamic-jump 1.5s 缓入缓出无限循环 - 图标上下跳动
- ◆ random-move/random\_move: dynamic-random-move 3s 缓入缓出无限循环 - 图标在小范围内随机移动

**rules:** 规则，支持规则嵌套、多值匹配等。当 value 是数组时，实体状态匹配数组中任意一个值即满足条件（相当于多个 == 的 OR 组合）。

## 轮播模式 (primary: dynamic)

动态图标还支持**轮播模式**——不依赖条件评估，所有规则按顺序循环展示。每条规则独立显示图标、颜色、动画、primary 文本和右上角斜三角文字 (icon\_text)。适合需要轮播展示多条信息（如不同电表读数）的场景。

```
- type: dynamic_icon
  primary: dynamic
  display_time: 3
  tap_action:
    action: more-info
    entity: sensor.ele
  rules:
    - primary: "{{ states('sensor.ele') }}"
      icon: mdi:transmission-tower-import
      icon_text: "502"
      color: "#e60033"
      animation: blink
    - primary: "{{ states('sensor.gas') }}"
      icon: mdi:transmission-tower-import
      icon_text: "602"
      color: "#007b43"
```

## 轮播模式特点：

| 特点               | 说明                                       |
|------------------|------------------------------------------|
| condition 忽略     | 所有规则无条件轮流显示，不评估条件                        |
| 不显示 badge        | 轮播模式下自动隐藏角标                              |
| 始终为开启状态          | 按钮以 .on 样式显示，不受 onCount 影响               |
| icon_text        | 图标右上角斜三角区域显示文本，支持 Jinja2 模板，颜色跟随规则 color |
| primary 在规则内     | 每条规则独立定义按钮下方文字，跟随轮播实时刷新                  |
| tap_action 为顶层配置 | 点击整个按钮触发的动作，与当前轮播到哪条规则无关                 |

## 预设规则 (preset)

当 `dynamic_icon` 需要匹配的是一组已知状态（如天气状况、空调模式），可以不用手动写十几条 `rules`，而是使用预设：指定 `preset` 名称和 `entity`，系统自动生成 `rules`。用户仍可在 `rules` 中手写部分规则覆盖预设值。

 **重要：** 使用 `preset` 时必须同时指定 `entity`（预设用它生成每条规则的 `condition.entity`）。

```
- type: dynamic_icon
  entity: weather.qweather_pro_lian_hu_weather # 必填
  preset: qweather # 自动展开 16 条天气规则
  loop_display: false
  show_badge: false
  primary: |
    {{state_attr('weather.qweather_pro_lian_hu_weather', 'condition_cn')}}
  tap_action:
    action: popup_card
    card:
      type: custom:qweather-card
      entity: weather.qweather_pro_lian_hu_weather
```

等价于手写 16 条 `rules`（每条自动注入 `condition.entity`、`icon`、`icon_text`、`color`、`animation`）。

## 预设列表

| 预设名称                                     | 适用场景         | 条件键值                                                                                                                                                       | 说明                                                                                                        |
|------------------------------------------|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| qweather<br>/ weather<br>/ metno         | 天气           | clear-<br>day sunny cloudy partlycloudy fog hail lightn<br>ing lightning-<br>rainy pouring rainy snowy snowy-<br>rainy windy exceptional clear-night clear | 覆盖 HA 标准天气 +<br>qweather 扩展状态。<br><code>icon_text</code> 统一为单字<br>（晴/夜/阴/云/<br>雾...），适合右上角三角徽章            |
| ac / airc<br>on / clim<br>ate / hva<br>c | 空调<br>(HVAC) | off idle cool heat dry fan_only auto                                                                                                                       | 匹配 <code>climate</code> 实体的<br><code>hvac_mode</code> 状态。关闭<br>时图标为 <code>mdi:air-<br/>conditioner</code> |

## 用户自定义覆盖

如果预设中某个状态的图标/颜色不符合预期，可以手动写 rules 来覆盖——rules 有值时预设不会展开：

```
- type: dynamic_icon
  entity: climate.ke_ting
  preset: ac
  # 手动覆盖部分规则
  rules:
    - condition:
        value: cool
        icon: mdi:snowflake
        icon_text: ❄
        color: "#00bfff"
        animation: rotate
```



(8) 晾衣架 (type: clothes\_dryer)

控制晾衣架的升降位置，提供可视化的机械仿真拖拽交互、收藏位置快捷按钮和灯光控制。点击按钮弹出晾衣架控制面板。该卡片支持独立使用，详见 8.4。



|                                                                                                                                                                                                                                                                                                     |                                                                                                                                                         |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>- type: clothes_dryer   name: 晾衣架   entity: cover.clothes_dryer   light_entity: input_boolean.ce_shi   on_icon: mdi:hanger   off_icon: mdi:tshirt-crew   on_color: '#f58220'   off_color: '#74787c'   width: 400px   locker: false   collect_position:     - name: 经常       position: 70</pre> | <pre># 显示名称（显示在卡片顶部标题 # 晾衣架 cover 实体（用于升降 # 灯光实体（自动插入灯光开关卡 # 开启时图标 # 关闭时图标 # 开启时颜色 # 关闭时颜色 # 弹窗宽度 # 关闭锁定模式，启用拖拽和收 # 收藏位置快捷按钮（最多 3 # 目标位置百分比（0=顶部，</pre> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|

```

100=底部)
- name: 顶部
  position: 0
- name: 底部
  position: 100
button: # 功能开关卡片 (灯光开关已自动添加, 此处只需配其他功能)
- type: switch
  entity: switch.dryer_heat
  name: 恒温烘干
  icon: mdi:heart

```

## 配置字段说明:

| 配置项              | 必填 | 类型      | 说明                                                                                    |
|------------------|----|---------|---------------------------------------------------------------------------------------|
| entity           | ✓  | string  | 晾衣架 cover 实体 ID, 使用 current_position 属性获取/设置升降位置                                      |
| name             | ✗  | string  | 显示名称, 显示在弹窗顶部标题栏, 同时显示在线/离线状态标签                                                       |
| light_entity     | ✗  | string  | 灯光实体 ID (如 switch / input_boolean), 控制壳体灯光按钮和光晕效果; 配置后会自动在功能卡片首位插入灯光开关, 无需手动添加        |
| on_icon          | ✗  | string  | 开启时按钮图标, 默认 mdi:hanger                                                                |
| off_icon         | ✗  | string  | 关闭时按钮图标, 默认 mdi:tshirt-crew                                                           |
| on_color         | ✗  | string  | 开启时图标颜色, 默认 #2c7da0                                                                   |
| off_color        | ✗  | string  | 关闭时图标颜色, 默认 #95a5a6                                                                   |
| width            | ✗  | string  | 弹窗宽度, 默认 440px                                                                        |
| locker           | ✗  | boolean | 锁定模式, 默认 true。锁定时位置固定 75%, 隐藏拖拽手柄和收藏位置按钮, 仅通过上升/下降按钮控制电机方向; 设为 false 时启用拖拽交互和收藏位置快捷按钮 |
| collect_position | ✗  | array   | 收藏位置快捷按钮, 最多 3 个, 每个含 name 和 position 字段 (仅在 locker: false 时显示)                       |
| Button           | ✗  | array   | 功能开关卡片列表, 使用统一卡片系统渲染, 2 列网格布局                                                         |

## 交互功能说明:

1. **机械仿真拖拽：**弹窗中的晾衣架可视化区域支持触摸/鼠标拖拽，直接拖动晾杆到目标位置（locker: false 时可用）
2. **升降按钮：**提供上升/停止/下降三个按钮，长按持续移动
3. **收藏位置：**一键快速移动到预设位置，带平滑过渡动画，当前位置匹配时按钮高亮（locker: false 时显示）
4. **灯光控制：**
  - 。壳体右上角的灯光按钮可直接切换 light\_entity 开关
  - 。灯光开启时，壳体下方出现暖黄色倒置梯形光晕照射效果（上窄下宽）
  - 。光晕效果实时响应灯光状态变化
5. **功能卡片：**底部 2 列网格展示功能开关卡片；配置了 light\_entity 时，灯光开关会自动插入首位，无需在 card 中手动添加
6. **位置反转自动适配：**自动检测 cover 实体的 position\_reverse 属性，适配不同品牌晾衣架的正反转逻辑

#### collect\_position 子项字段：

| 字段       | 必填 | 类型     | 说明                      |
|----------|----|--------|-------------------------|
| name     | ✓  | string | 按钮显示名称                  |
| position | ✓  | number | 目标位置百分比，0 = 顶部，100 = 底部 |

## （9） 无类型按钮

无类型按钮也是该卡片的重要按钮，可作为弹出窗口的功能入口。

无类型按钮也可以根据实体的状态来动态显示图标，自定义指定点击动作，支持所有的动作（含卡片扩展的动作）。



```
- name: 冰箱
  entity: switch.canting_bingxiang_chazuo      #根据实体的状态动态显示图标
  on_icon: mdi:fridge
  off_icon: mdi:fridge
  on_color: "#f58220"
  off_color: "#74787c"
  tap action:
```

无类型按钮支持独立按钮的角标支持 badge 数组或 badge\_entity 配置，可以根据实体状态条件显示角标计数。当配置了 badge 或 badge\_entity 时，角标的值还决定了图标的显示状态：

角标值  $\geq 1 \rightarrow$  显示 on\_icon + on\_color（视为"开启"状态）

角标值  $< 1 \rightarrow$  显示 off\_icon + off\_color（视为"关闭"状态）

这意味着即使没有 entity，或 entity 状态与角标无关，只要配置了 badge / badge\_entity，图标和颜色就会跟随角标值变化。

```
- name: 电脑
  badge:
    - entity: binary_sensor.192_168_1_34
      condition: "on"
  entity: binary_sensor.192_168_1_34
  on_icon: mdi:desktop-classic
  off_icon: mdi:desktop-classic
  on_color: "#f58220"
  off_color: "#74787c"
  tap_action:
    action: toggle
```

上例中：binary\_sensor 为 on 时角标计数 = 1 ( $\geq 1$ )，图标显示橙色 (on\_color)；为 off 时角标计数 = 0 ( $< 1$ )，图标显示灰色 (off\_color)。

**多条件角标示例：**

```
- name: 电脑
  entity: binary_sensor.192_168_1_34
  on_icon: mdi:desktop-classic
  off_icon: mdi:desktop-classic
  on_color: "#f58220"
  off_color: "#74787c"
  badge:
    - entity: binary_sensor.192_168_1_34
      condition: on
    - entity: sensor.zimi_cn_1137225298_zncz01_electric_power_p_10_1
      condition:
        - <10
```

如果 binary\_sensor 为 on → 计数 +1；如果功率 < 10  
→ 计数 +1。两项都满足时角标显示 2，只满足一项显示 1，  
都不满足则角标不显示且图标为 off 状态。

badge\_entity 示例：

```
- name: 未读消息
  badge_entity: sensor.unread_count      # 实体值作为角标，≥1 时显示 on
  on_icon: mdi:bell-ring
  off_icon: mdi:bell-outline
  on_color: "#e74c3c"
  off_color: "#95a5a6"
```

condition 支持的格式：

| 格式     | 说明            | 示例                                                                |
|--------|---------------|-------------------------------------------------------------------|
| 字符串状态  | 实体状态等于该值      | condition: on                                                     |
| 字符串表达式 | 数值比较          | condition: <10、>=50、!=0                                           |
| 数组     | OR 语义，任一满足即可  | condition: ['<10', '>100']                                        |
| 对象     | 透传给条件引擎（高级用法） | condition: { operator: '>', value: 50, attribute: 'temperature' } |

| 格式 | 说明                                 | 示例           |
|----|------------------------------------|--------------|
| 缺省 | 实体非<br>unknown/unavailable 即为<br>真 | 不写 condition |

角标优先级： 配置了 badge 或 badge\_entity 时，角标值决定图标/颜色，entity 的状态不再影响图标显示。badge 与 badge\_entity 互斥，badge 优先级更高。

### 3. 卡片（核心功能）

控制/展示实体的状态，主要有以下几类，以下将对每个卡片进行简要介绍：

- 传感器（type: sensor） - 显示传感器数值
- 开关（type: switch） - 控制开关实体
- 选择（type: select） - 选择选项
- 数字（type: number） - 调节数值
- 文本（type: text） - 输入文本并提交
- 按钮（type: button） - 执行按钮操作
- 自定义（type: card） - 显示自定义内容
- 使用量（type: usage） - 显示使用量数据
- 曲线图表（type: chart\_line） - 显示历史数据图表  
（移除旧格式 type: chart，不在兼容该格式，使用 ECharts 渲染）
- 媒体（type: media） - 控制媒体播放器
- 图片（type: picture） - 根据实体状态显示不同图片
- 时间轴（type: timeline） - 显示活动记录时间轴
- 单选（type: radio） - 从选项列表选择一个值
- 用户（type: user） - 显示用户信息、位置、天气等
- 窗帘（type: curtain） - 显示窗帘
- 仪表图（type: chart\_gauge） - 仪表图类型
- 进度条图（type: chart\_progress） - 进度条图



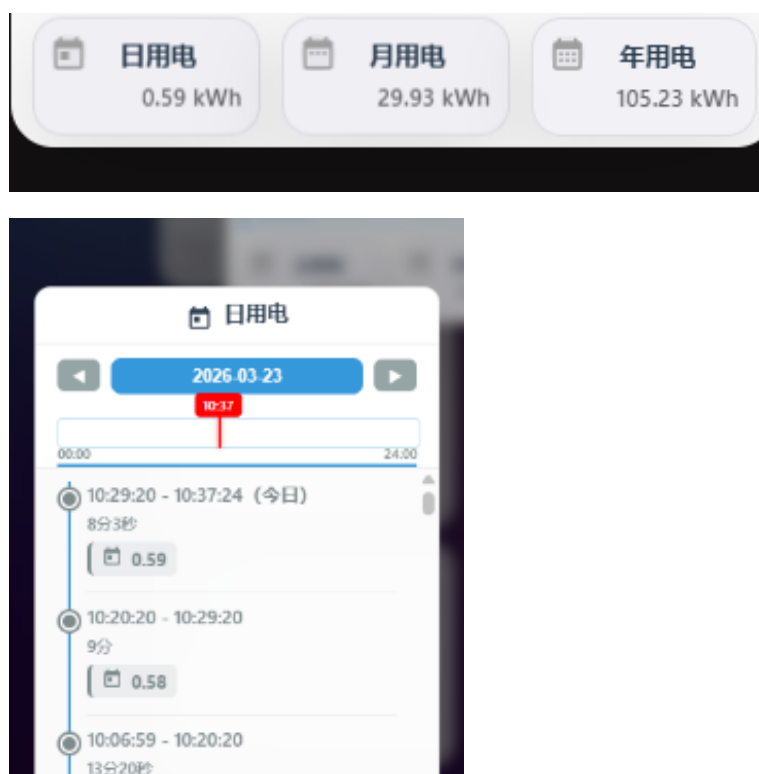
- 统计型柱状图 (type: chart\_bar) - 统计型柱状图  
(按照参数聚合)
- 环形图 (type: chart\_pie) - 统计型柱状图 (使用 ECharts 渲染)
- 网页卡片 (type: html) - 网页型卡片 (支持简单 js、style、运算等)
- 日历热力图 (type: chart\_heatmap) - 日历热力图  
(使用 ECharts 渲染、支持实体状态属性和 api 获取数据)
- 日历图 (type: chart\_calendar) - 日历图 (将数据显示在日历上, 之年年月聚合)
- 混合图 (type: chart\_mixed) - 混合图表 (使用 ECharts 渲染)
- 南丁格尔玫瑰图 (type: chart\_nightingale) - 南丁格尔玫瑰图 (使用 ECharts 渲染)
- 完整饼图 (type: chart\_pie\_full) - 完整饼图 (使用 echarts 渲染)
- 条件卡片 (type: conditional\_tabs) - 根据条件显示对应卡片 (可以适应选项卡或者实体)
- 天气卡片 (type: weather) - 支持多城市天气
- 调试卡片 (type: debug\_card)
- 自动化卡片 (type: automations)

- 最近使用卡片 (type: recently-used)
- 使用量日历卡片 (type: usage\_calendar)
- 打印机用量统计卡片 (type: printer)
- 爱快路由管理卡片 (type: ikuai)
- 设备用电统计卡片 (type: device\_usage)

## (1) 传感器 (type: sensor)

主要用来显示实体值，支持小型按钮 (layout: mini，当不配置时为标准按钮)，支持自定义点击动作，同时该类型按钮默认支持点击图标弹出历史记录 (show\_history 为 true 时)。

另外该类型按钮支持卡片内部的 way 计算引擎 (见..)，可以实现更为复杂的功能。

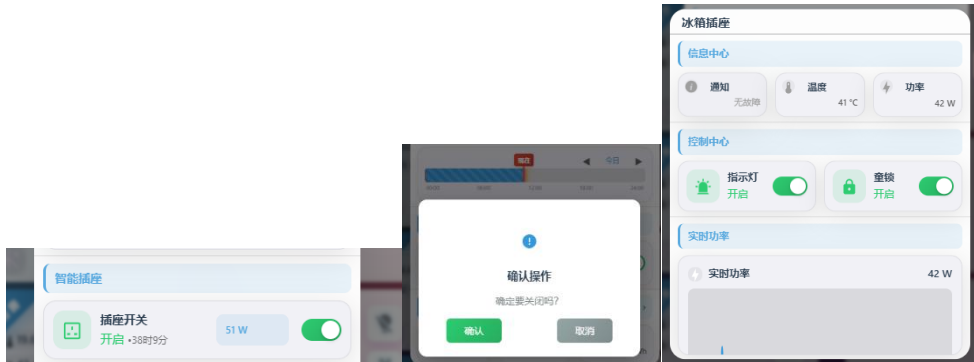


配置代码如下：

```
- type: sensor
entity: sensor.bingxiang_year
show_duration: input_boolean.show_duration #状态持续时长，可以是开关类实体，也可以是字符串 on/off
layout: mini
name: 年用电
icon: mdi:calendar-month
on_icon: mdi:door-open #当配置它们时，表示该传感器为 on/off 类型的传感器，图标动态显示
off_icon: mdi:door-closed
```

## (2) 开关 (type: switch)

主要用来控制设备的开关。



配置代码如下：

```
- entity: switch.canting_bingxiang_chazuo
  type: switch
  show_ios-switch: false      是否显示右侧开关按钮，false 为不显示，此时点击右侧即切换开关
  show_duration: true
  confirm: true
  name: 插座开关
  icon: mdi:power-socket-uk   # 默认显示的图标
  on_icon: mdi:lightbulb-group # 灯开启时的图标
  off_icon: mdi:lightbulb-group-off # 灯关闭时的图标
  on_color: '#f1c40f'         # 灯开启时图标的颜色
  off_color: '#95a5a6'        # 灯关闭时图标的颜色
```

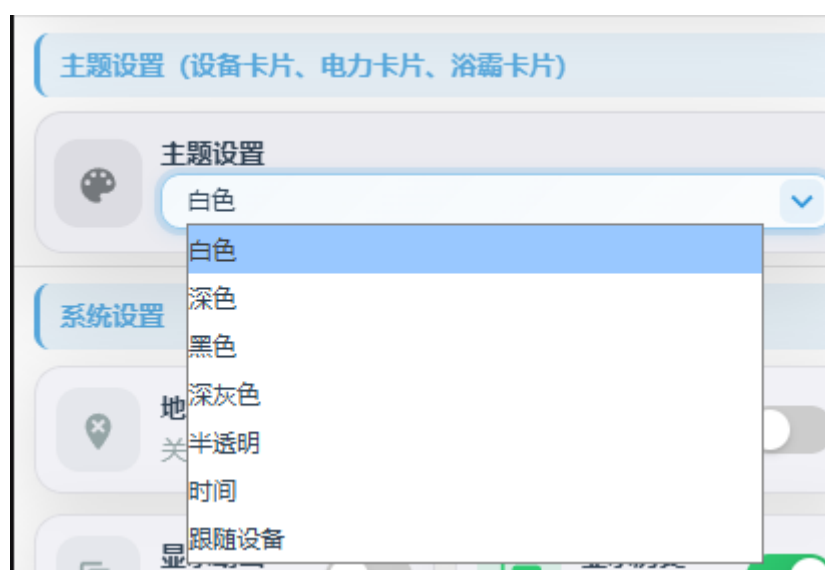
主要参数如下：

- **Confirm:** 表示是否弹出确认对话框，可选参数为 true/false，可以从实体处获取，也可以是字符串
- **show\_duration:** 是否显示最新状态到现在的时间，可选参数为 true/false，可以从实体处获取，也可以是字符串
- **info\_entity:** 信息，可以配置实体（如功率），点击弹出窗口，弹出窗口中，点击图标同样也会弹出该实体的历史记录。

### (3) 选择 (type: select)

提供下拉选项，供用户选择使用。

支持两种方式：①系统自动根据实体值来自动构建下拉选项。②用户定义下拉选择，第 1 种很简单，此处不讲，主要讲第 2 种用户定义下拉选择（使用 map\_table 定义）：

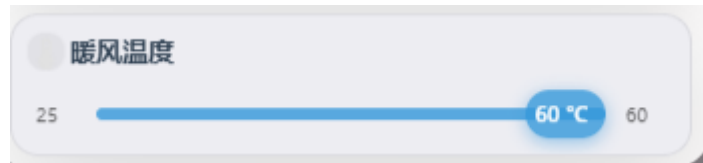


```
- type: select
  name: 主题设置
  entity: input_select.theme
  update_interval: 3
  map_table:
    "on":
      light:
        text: 白色
        icon: mdi:palette
        color: "#74787c"
      dark:
        text: 深色
        icon: mdi:palette
        color: "#323232"
      black:
        text: 黑色
        icon: mdi:palette
        color: "#000000"
      darkgray:
        text: 深灰色
        icon: mdi:palette
        color: "#222222"
      transparent:
        text: 半透明
        icon: mdi:palette
        color: "#0d0015"
```

```
time:
  text: 时间
  icon: mdi:clock
  color: "#824880"
phone:
  text: 跟随设备
  icon: mdi:monitor-cellphone
  color: "#96514d"
"off":
  "off":
    text: 无
    icon: mdi:panorama-fisheye
    color: "#FFFFFF"
```

#### (4) 数值 (type: number)

提供滑块数值输入。



```
- type: number
  entity: number.she_zhi_feng_nuan_wen_du
  name: 暖风温度
  icon: mdi:thermometer
  min: 25
  max: 60
```

## (5) 文本输入 (type: text)

输入后点击右侧按钮为保存按钮，点击左侧图标可以查看历史值，点击右侧按钮提交保存。



```
- type: text
  entity: input_text.guo_wang_yan_zheng_ma
  name: 输入验证码
  icon: mdi:message-text
  icon_color: "#006e54"
  submit_icon: mdi:send
```



(6) 按钮 (type: button)

用于控制 input\_button、button 等按压型实体，或 switch 等开关型实体。点击按钮触发操作，支持点击反馈动画和持续时长显示。

|                                                                                                                                                                                                                                                                                                                                                  |                                                                                                                                                                                                                                |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>- type: button   entity: input_button.ceshi   name: 打开监控   icon: mdi:cctv   icon_color: '#d0273e'   on_color/off_color)   on_icon: mdi:cctv   off_icon: mdi:cctv-off   on_color: '#3498db'   off_color: '#95a5a6'   status_text: 点击   status_text_completion: OK   icon_click_color: '#006e54'   show_duration: true   confirm: false</pre> | <pre># 按钮实体 ID (支持 input_button / button / switch 等) # 显示名称 # 图标 # 图标颜色 (优先级高于 on_color/off_color) # 开启时图标 # 关闭时图标 # 开启时图标颜色 # 关闭时图标颜色 # 自定义状态文本 (替代默认的"开启/ 关闭") # 点击后显示 3 秒的完成文本 # 点击后图标变色 3 秒的颜色 # 是否显示持续时长 # 点击是否需要确认</pre> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

各配置项说明：

| 配置项         | 必填 | 类型     | 说明                                       |
|-------------|----|--------|------------------------------------------|
| entity      | ☑  | string | 按钮实体 ID，支持 input_button、button、switch 等域 |
| name        | ✗  | string | 显示名称，默认 按钮                               |
| icon        | ✗  | string | 默认图标                                     |
| icon_color  | ✗  | string | 图标颜色，优先级高于 on_color / off_color          |
| on_icon     | ✗  | string | 开启时图标，默认 mdi:toggle-switch               |
| off_icon    | ✗  | string | 关闭时图标，默认 mdi:toggle-switch-off           |
| on_color    | ✗  | string | 开启时图标颜色，默认 #3498db                       |
| off_color   | ✗  | string | 关闭时图标颜色，默认 #95a5a6                       |
| status_text | ✗  | string | 自定义状态文本，配置后替代默认的"开启/关闭"                  |

| 配置项                    | 必填 | 类型      | 说明                                            |
|------------------------|----|---------|-----------------------------------------------|
| status_text_completion | ✗  | string  | 点击后按钮状态文本显示的完成文本，3 秒后自动恢复为 status_text（或默认文本） |
| icon_click_color       | ✗  | string  | 点击后图标变色的颜色，3 秒后自动恢复为原来的颜色                     |
| show_duration          | ✗  | boolean | 是否显示距上次状态变化的持续时长，默认 false                     |
| confirm                | ✗  | boolean | 点击时是否弹出确认对话框，默认 false                         |
| tap_action             | ✗  | object  | 自定义点击动作，配置后替代默认的按钮按压行为                        |

### 点击交互流程：

- ① 点击按钮 → 图标变色 (icon\_click\_color) + 状态文本变为完成文本 (status\_text\_completion) + 时长从 0 秒 重新计时
- ② 3 秒后 → 图标恢复原色 + 状态文本恢复为 status\_text（或默认的"开启/关闭"）

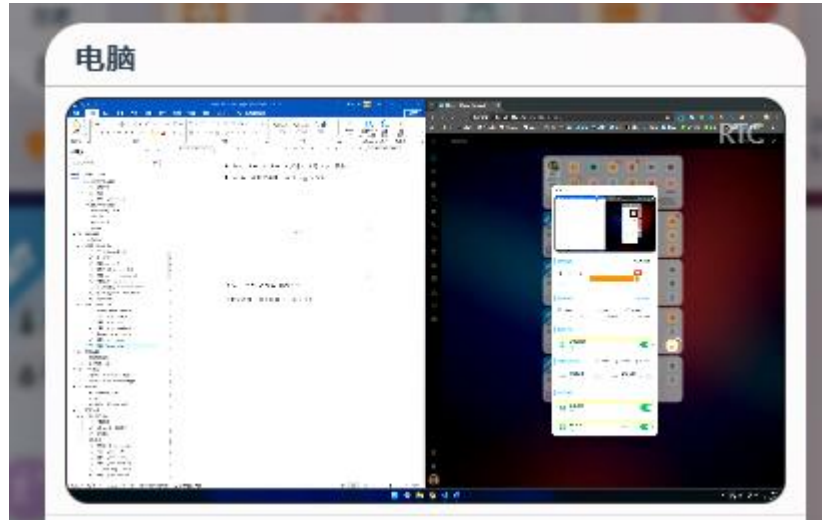
图标可单独点击查看历史记录

### 实体类型与行为：

- input\_button / button 实体：调用 press 服务
- switch 等其他实体：调用 toggle 服务

## (7) 卡片 (type: card)

主要是在弹出窗口等地引入其他卡片。



```
- type: card
  name: 画面
  card_config:
    type: custom:webrtc-camera
    url: rtsp://192.168.1.12:8555/onecloud kvm
```

## (8) 图表 (type: chart\_line)

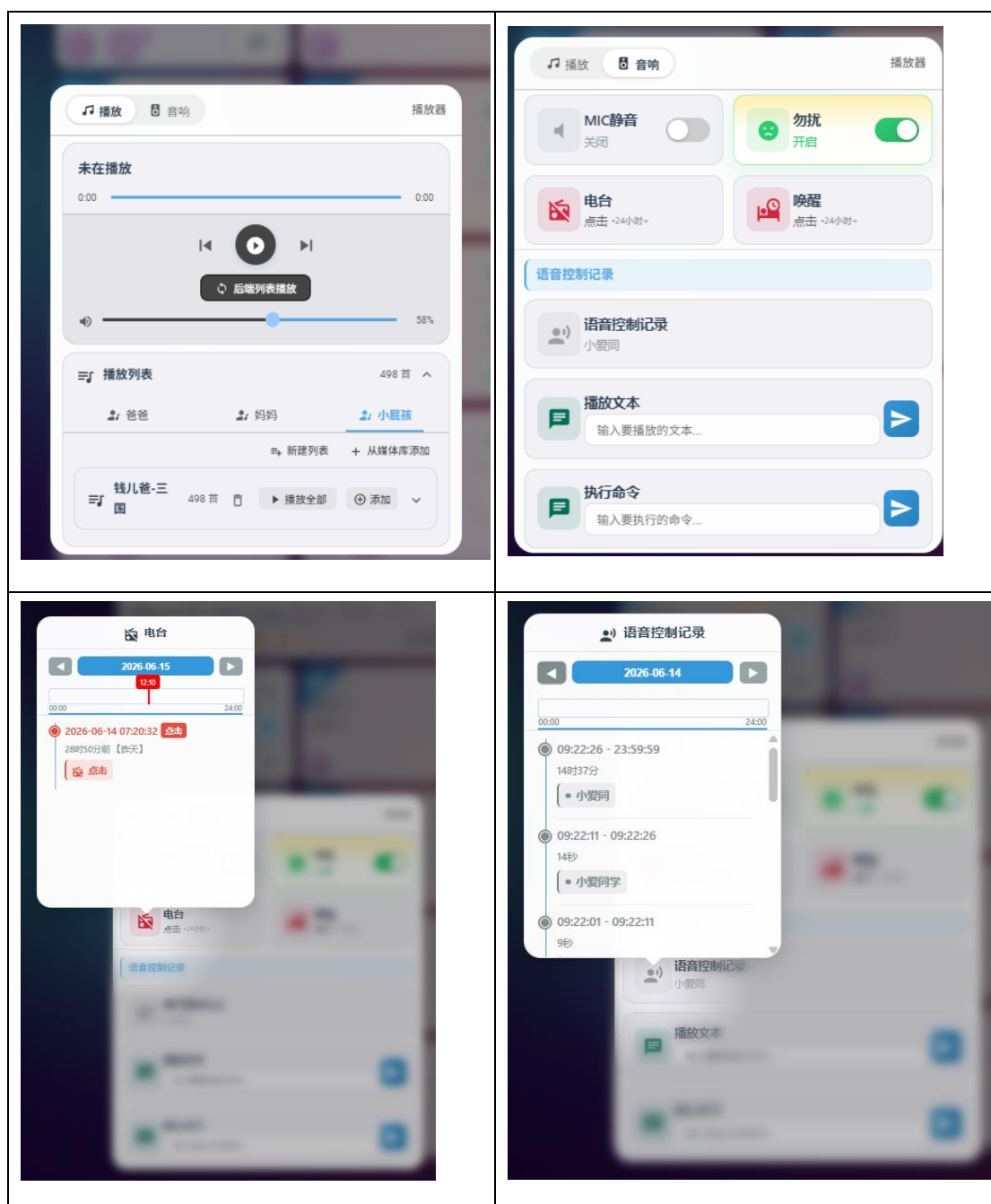
曲线图，使用 ECharts 渲染。



```
- type: chart_line
  entity: sensor.canting_bingxiang_chazuo_gonglv      #数值实体
  height: 120px
  icon: mdi:air-conditioner
  name: 平滑曲线图
  update_interval: 3
  line:
    min: 0
    max: 500
    hours_to_show: 128                                #显示多长时间
```

## (9) 媒体 (type: media) v5.0.0 重新设计

主要是控制本地音乐播放、后后端控制列表播放、音响控制等功能，支持自由布局 (free\_layout) 音响控制界面。播放列表和后端列表播放需要后端集成 (“HA 统一数据存储”) 的支持。



```

- type: media
  on_icon: mdi:music
  off_icon: mdi:music-off
  on_color: "#00a381"
  off_color: "#74787c"
  name: 播放器
  media_entity: media_player.xiaomi_cn_2085832868_oh2p
  popup_position: clone_button_down
  width: 430px
  user: 爸爸,妈妈,小屁孩
  free_layout:
    - row: 1
      per_line: 2
      title: 控制中心
      show_title: false
      items:
        - type: switch
          entity: switch.xiaomi_cn_2085832868_oh2p_mute_p_4_1
          name: MIC 静音
          on_icon: mdi:volume-off
          off_icon: mdi:volume-low
        - type: switch
          entity: switch.xiaomi_cn_2085832868_oh2p_no_disturb_p_9_1
          name: 勿扰
          on_icon: mdi:emoticon-angry
          off_icon: mdi:emoticon-happy
        - type: button
          entity: button.xiaomi_cn_2085832868_oh2p_play_radio_a_7_2
          name: 电台
          icon: mdi:radio
          icon_color: "#d0273e"
          on_icon: mdi:radio
          off_icon: mdi:radio-off
          on_color: "#3498db"
          off_color: "#95a5a6"
          status_text: 点击
          status_text_completion: OK
          icon_click_color: "#006e54"
          show_duration: true
        - type: button
          entity: button.xiaomi_cn_2085832868_oh2p_wake_up_a_7_1
          name: 唤醒
          icon: mdi:sleep
          icon_color: "#d0273e"
          on_icon: mdi:sleep
          off_icon: mdi:bed-clock
          on_color: "#3498db"
          off_color: "#95a5a6"
          status_text: 点击
          status_text_completion: OK
          icon_click_color: "#006e54"
          show_duration: true
    - row: 2
      per_line: 1
      title: 语音控制记录
      items:
        - entity: sensor.xiaomi_oh2p_dac5_conversation
          type: sensor
          name: 语音控制记录
          icon: mdi:account-voice
        - type: text
          entity: text.xiaomi_oh2p_dac5_play_text
          name: 播放文本
          icon: mdi:message-text
          icon_color: "#006e54"

```

```
placeholder: 输入要播放的文本...
submit_icon: mdi:send
- type: text
  entity: text.xiaomi_oh2p_dac5_execute_text_directive
  name: 执行命令
  icon: mdi:message-text
  icon_color: "#006e54"
  placeholder: 输入要执行的命令...
  submit_icon: mdi:send
```

自由布局窗口的布局方式与弹窗系统中的自由布局方式一致（复用自由布局弹窗）。

(10) 图片 (type: picture)

根据实体状态显示对应图片（支持定位、缩放、旋转、优先级等设置）。



```
- type: picture
  show_button_background: false
  bg_image: /local/house/燃气灶油烟机.png
  halo_entity: switch.youyanji #光晕实体，当实体值为 on/true 时，png 非透明区域边缘会显示彩虹光晕效果
  bg_image_position:
    - x: 0
    - "y": 7
    - zoom: 1
    - rotate: 0
  entities:
    - entity: switch.youyanji
      map_table:
        "on":
          image_path: /local/house/火苗.gif
          position:
            - x: -20
            - "y": 28
            - zoom: 0.2
            - rotate: 0
    - entity: switch.youyanji
      map_table:
        "on":
          image_path: /local/house/火苗.gif
          position:
            - x: 20
            - "y": 28
            - zoom: 0.2
            - rotate: 0
```

**priority 参数（优先级）：**数字越小优先级越高，意思是，当都满足条件时，优先显示那个一个实体。

配置项说明：

| 配置项      | 必填 | 类型     | 说明         |
|----------|----|--------|------------|
| bg_image | ✗  | string | 背景图片 URL（支 |

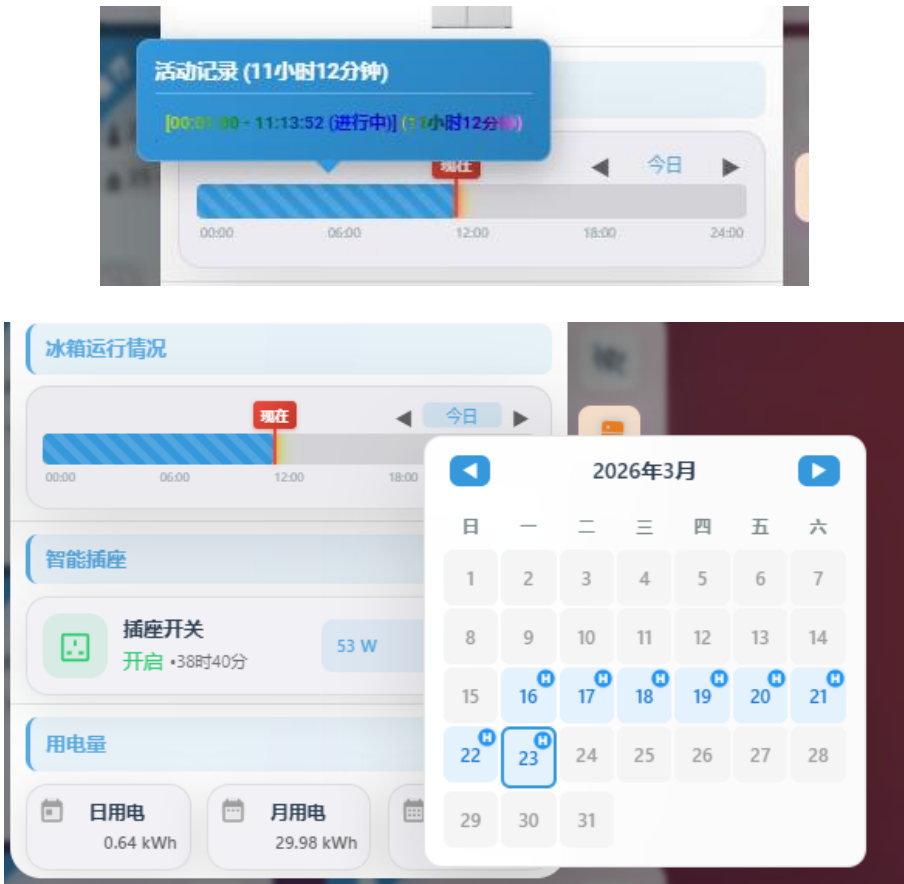


| 配置项                    | 必填 | 类型      | 说明                                    |
|------------------------|----|---------|---------------------------------------|
|                        |    |         | 持 /local/xxx.png 等相对路径)               |
| bg_image_position      | ✗  | array   | 图片位置微调，4 个浮点值，控制图片的 object-position   |
| show_button_background | ✗  | boolean | 是否显示按钮卡片的灰色背景，默认 true，设为 false 时仅显示图片 |
| halo_entity            | ✗  | string  | 控制光晕效果的实体 ID。实体为 on 时图片下方显示暖色光晕       |

halo\_entity 通常配置为对应设备的开关实体，设备开启时图片显示光晕效果，提供直观的视觉反馈

(11) 时间轴 (type: timeline)

在按钮上直接显示设备状态变化的时间轴（例如人体传感器从"无人"到"有人"的变化记录）。点击时间轴段显示详细活动记录。



最简配置:

```
- type: timeline
  entity: binary_sensor.192_168_1_34
```

带 map\_table(字段映射) 配置:

```
- type: timeline
  entity: binary_sensor.living_motion
  map_table:
    on:
      on:
        text: '有人'
        icon: 'mdi:account'
        color: '#2ecc71'
      active:
        text: '活跃中'
```

# 要显示历史数据的实体  
# 状态值→显示文本/颜色的映射  
# on 状态下的子状态  
# 状态值为"on"  
  
# 状态值为"active"

```
        icon: 'mdi:run'
        color: '#3498db'
    off:                                     # off 状态
        off:
            text: '无人'
            icon: 'mdi:account-off'
            color: '#95a5a6'
    api_base_url: http://192.168.1.100:8080 # 如果 HA 没有历史，可从外部 API
    获取
```

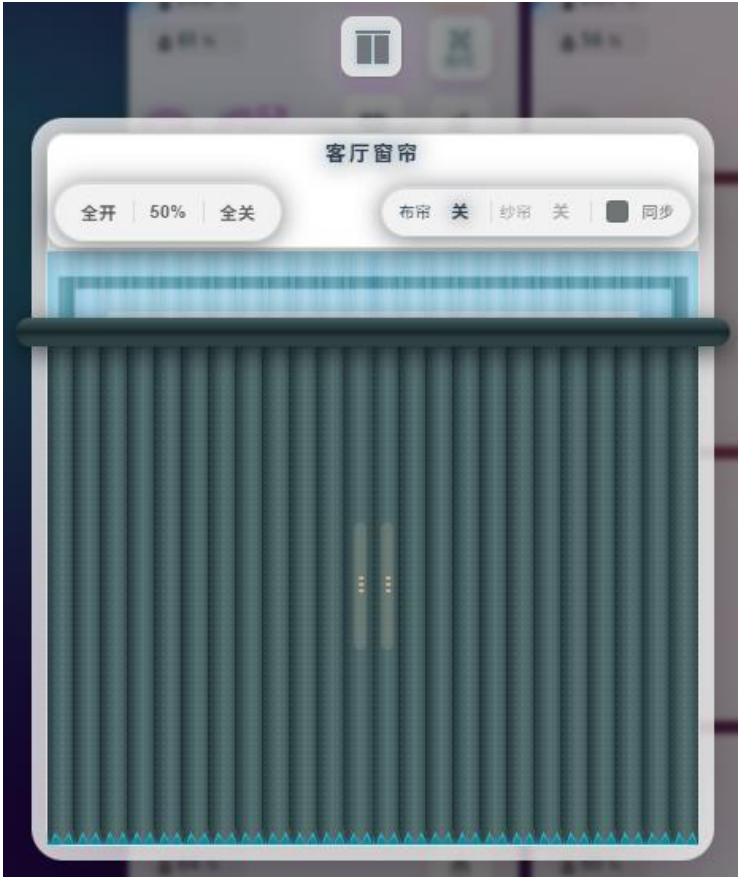
## (12) 单选 (type: radio)



```
- type: radio
  name: 主题设置
  entity: input_select.theme
  update_interval: 3
  map_table:
    "on":
      light:
        text: 白色
        icon: mdi:palette
        color: "#74787c"
      dark:
        text: 深色
        icon: mdi:palette
        color: "#323232"
      black:
        text: 黑色
        icon: mdi:palette
        color: "#000000"
      darkgray:
        text: 深灰色
        icon: mdi:palette
        color: "#222222"
      transparent:
        text: 半透明
        icon: mdi:palette
        color: "#0d0015"
      time:
        text: 时间
        icon: mdi:clock
        color: "#824880"
      phone:
        text: 跟随设备
        icon: mdi:monitor-cellphone
        color: "#96514d"
    "off":
      "off":
        text: 无
        icon: mdi:panorama-fisheye
        color: "#FFFFFF"
```

(13) 窗帘 (type: curtain)

窗帘操作，支持双层窗帘（布帘和纱帘）、单层窗帘的（布帘/纱帘），双层窗帘时点击纱帘激活纱帘操作，操作完成后自动取消纱帘激活，自动将布帘激活。



窗帘支持 3 种配置方式：

① 作为 button 下的按钮

点击自动弹出窗帘操作界面。

```
- type: curtain
  name: "客厅窗帘"
  icon: mdi:curtain
  layer_mode: double          # 层级模式: double | single | single-sheer
  open_mode: double          # 开合方式: double | single-left | single-right
  fabric_entity: cover.living_room_main # 布帘 cover 实体 (必填)
```

```
sheer_entity: cover.living_room_sheer    # 纱帘 cover 实体（可选，单层模式不需要）
width: "400px"                            # 弹窗宽度
popup_position: center                    # 弹窗位置
background_image: "/local/images/curtain-bg.jpg" # 自定义背景图（可选，不配则用默认）
```

## 两个关键配置项（需用户配置）：

- 层级模式 layerMode: double(布+纱) / single(仅布) / single-sheer(仅纱)
- 开合方式 mode: double(中对开) / single-left(靠左单开) / single-right(靠右单开)

## 交互逻辑：

- 拖拽舞台 → 改变窗帘开合位置（映射到 0-100%）。
- 松手时触发褶皱抖动动画（fabricShake / shearShake keyframes）。
- 同步模式下布帘+纱帘联动。
- 纱帘操作 5s 无活动后自动切回布帘层。

## ② 作为服务被调用

```
tap_action:
  action: call-service
  service: room_card.show_curtain_popup
  service_data:
    name: 客厅窗帘
    fabric_entity: cover.living_room_fabric
    sheer_entity: cover.living_room_sheer
    layer_mode: double
    open mode: double
```

width: 430px

### ③ 作为普通按钮使用

```
- type: curtain
  name: 客厅窗帘
  entity: cover.living_curtain
  fabric_entity: cover.living_curtain
  sheer_entity: cover.living_curtain_bc
  layer_mode: double
  open_mode: double
  on_icon: mdi:curtains
  off_icon: mdi:curtains-closed
  on_color: '#3498db'
  off_color: '#95a5a6'
  width: 560px
  popup_position: center
```

# 显示名称  
# 窗帘 cover 实体  
# 布料帘实体（双层帘时指定）  
# 纱帘实体（双层帘时指定）  
# 层级模式: single | double  
# 开合方式: double（对开）| single（同向）  
# 开启时图标  
# 关闭时图标  
# 弹窗宽度  
# 弹窗位置

### 配置字段说明：

| 配置项           | 必填 | 类型     | 默认值      | 说明                                     |
|---------------|----|--------|----------|----------------------------------------|
| entity        | ✓  | string | 无        | 窗帘 cover 实体 ID，用于开合控制                  |
| name          | ✗  | string | 窗帘       | 显示名称                                   |
| fabric_entity | ✗  | string | 同 entity | 布料帘实体 ID（双层模式时指定不同的布料帘实体）              |
| sheer_entity  | ✗  | string | 无        | 纱帘实体 ID（双层模式时需要）                       |
| layer_mode    | ✗  | string | double   | 层级模式: single（单层）或 double（双层）           |
| open_mode     | ✗  | string | double   | 开合方式: double（对开，左右分开）或 single（同向，单侧拉开） |

### 弹窗功能：

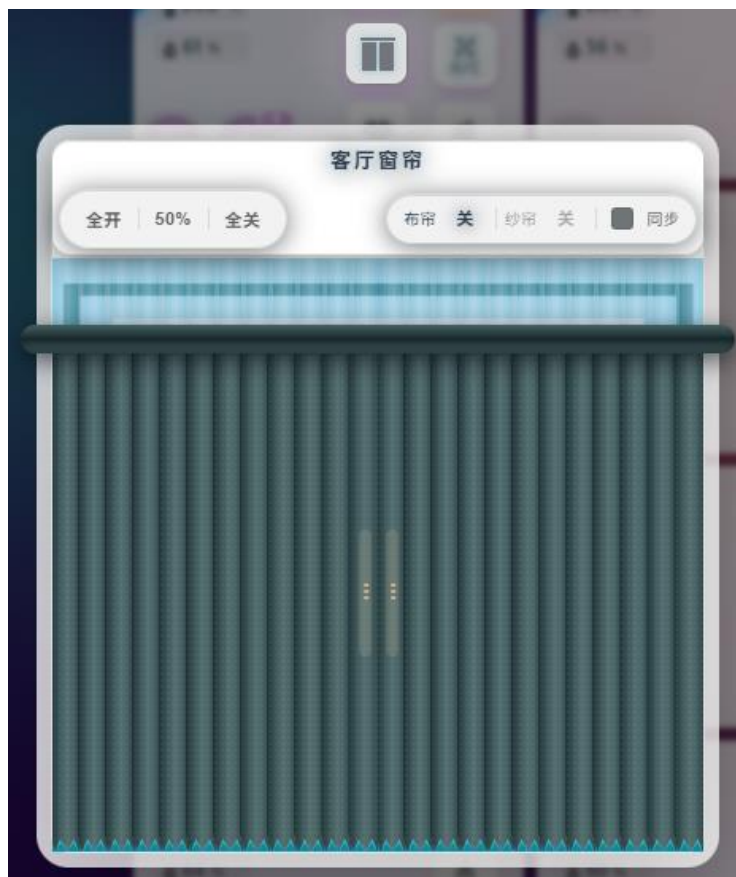
- 百分比滑块：拖动滑块控制窗帘开合程度

- **快速按钮：** 50%快捷位置
- **停止按钮：** 在移动过程中随时停止
- **双层切换** (layer\_mode: double): 切换布料帘/纱帘控制
- **状态显示：** 当前开合百分比和状态文本



#### ④ 单个窗帘

控制电动窗帘的开合、停靠位置和层级切换。点击按钮弹出窗帘控制面板，支持单层/双层窗帘、百分比开合控制、开合模式（同向/对开）等。

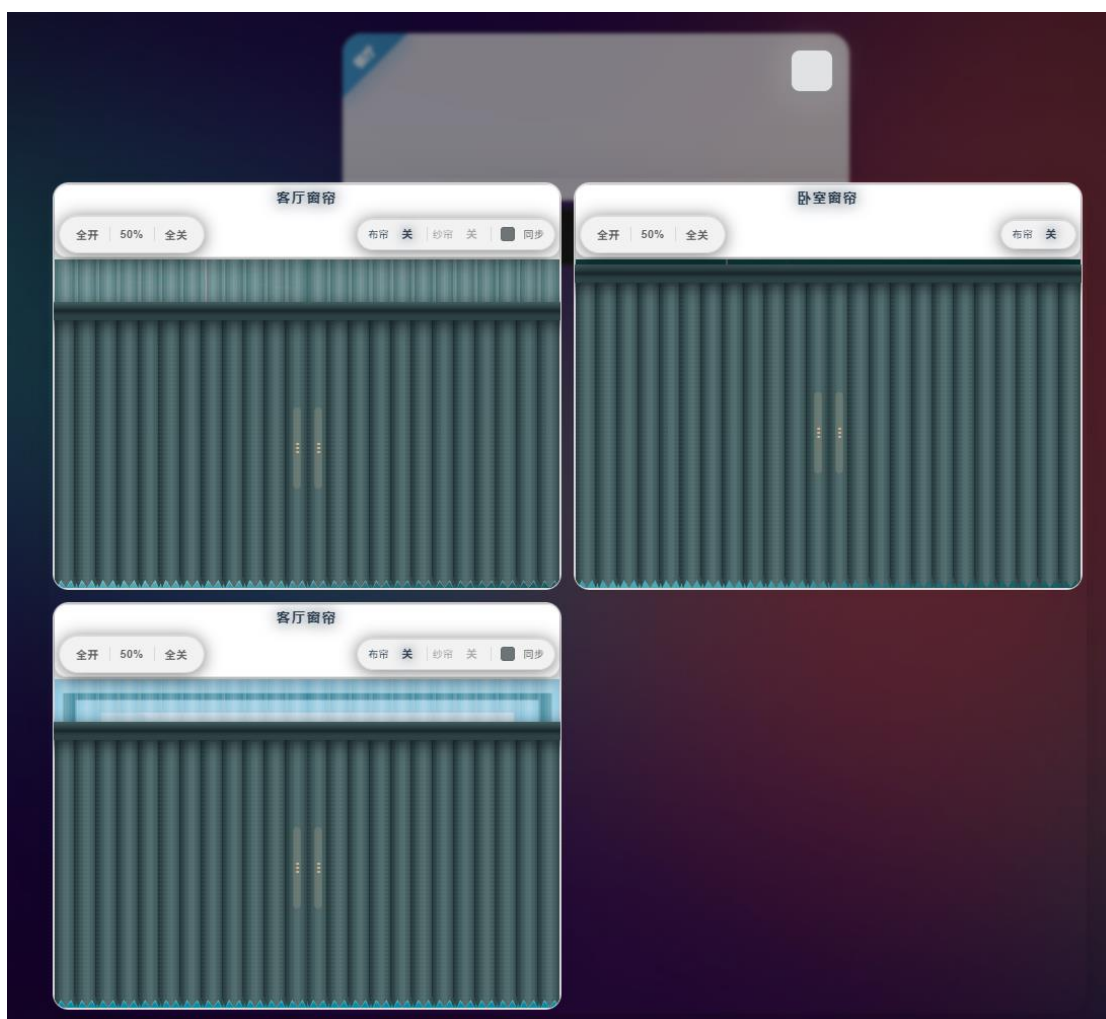


```
- type: curtain
  name: 客厅窗帘
  entity: cover.living_curtain
  fabric_entity: cover.living_curtain
  sheer_entity: cover.living_curtain_bc
  layer_mode: double
  open_mode: double
single (同向)
  on_icon: mdi:curtains
  off_icon: mdi:curtains-closed
  on_color: '#3498db'
  off_color: '#95a5a6'
  width: 560px
  popup_position: center
```

# 显示名称  
# 窗帘 cover 实体  
# 布料帘实体（双层帘时指定）  
# 纱帘实体（双层帘时指定）  
# 层级模式: single | double  
# 开合方式: double (对开) |  
# 开启时图标  
# 关闭时图标  
# 弹窗宽度  
# 弹窗位置

## ⑤ 窗帘聚合（全屋窗帘）

与空调聚合类似，当配置 card 数组时，点击按钮会弹出包含多个窗帘控制舞台的聚合弹窗。PC 端一行 2 个，移动端一行 1 个，方便统一管理全屋窗帘。



```
- type: curtain
  name: 全屋窗帘
  on_icon: mdi:curtains
  off_icon: mdi:curtains-closed
  on_color: '#d4b896'
  off_color: '#7f8c8d'
  show_open: true
  layer_mode: double
  open_mode: double
  background_image: https://xxx/room.jpg
```

# 聚合按钮名称  
# 有窗帘开启时的图标  
# 所有窗帘关闭时的图标  
# 开启颜色  
# 关闭颜色  
# 可选，仅显示已打开的窗帘（默认 false）  
# 全局默认层级模式（子项未配时使用）  
# 全局默认开合方式（子项未配时使用）  
# 全局默认背景图（子项未配时使用）

|                                         |                        |
|-----------------------------------------|------------------------|
| width: 700px                            | # 聚合弹窗宽度 (PC 端自动扩展为双列) |
| card:                                   | # 窗帘列表                 |
| - name: 客厅窗帘                            |                        |
| height: 400px                           | # 可选, 自定义该窗帘舞台高度       |
| fabric_entity: cover.ke_ting_bu_lian    | # 布帘实体                 |
| sheer_entity: cover.ke_ting_sha_lian    | # 纱帘实体                 |
| layer_mode: double                      | # 子项可覆盖全局默认            |
| - name: 卧室窗帘                            |                        |
| fabric_entity: cover.wo_shi_bu_lian     |                        |
| layer_mode: single                      | # 单层窗帘                 |
| open_mode: single-left                  | # 左侧单开                 |
| - name: 书房窗帘                            |                        |
| fabric_entity: cover.shu_fang_bu_lian   |                        |
| background_image: https://xxx/study.jpg | # 子项独立背景图              |

## 聚合模式配置字段说明:

| 配置项              | 必填 | 类型      | 默认值                     | 说明                                        |
|------------------|----|---------|-------------------------|-------------------------------------------|
| type             | ✓  | string  | —                       | 固定为 curtain                               |
| name             | ✗  | string  | 窗帘                      | 聚合按钮显示名称                                  |
| card             | ✓  | array   | []                      | 窗帘列表, 每个子项为一个窗帘配置                         |
| on_icon          | ✗  | string  | mdi:curtain<br>s        | 有窗帘开启时的按钮图标                               |
| off_icon         | ✗  | string  | mdi:curtain<br>s-closed | 所有窗帘关闭时的按钮图标                              |
| on_color         | ✗  | string  | #d4b896                 | 有窗帘开启时的图标颜色                               |
| off_color        | ✗  | string  | #7f8c8d                 | 所有窗帘关闭时的图标颜色                              |
| show_open        | ✗  | boolean | false                   | 设为 true 时, 弹窗中仅显示已打开的窗帘 (布帘位置 > 10% 视为开启) |
| layer_mode       | ✗  | string  | double                  | 全局默认层级模式, 子项未配时使用                         |
| open_mode        | ✗  | string  | double                  | 全局默认开合方式, 子项未配时使用                         |
| background_image | ✗  | string  | 无                       | 全局默认背景图片, 子项未配时使用                         |

| 配置项            | 必填 | 类型     | 默认值    | 说明                 |
|----------------|----|--------|--------|--------------------|
| width          | ✗  | string | 无      | 聚合弹窗宽度，PC 端自动扩展为双列 |
| height         | ✗  | string | 无      | 单个窗帘的高度            |
| popup_position | ✗  | string | center | 弹窗位置               |

### card 子项配置字段：

| 配置项              | 必填 | 类型     | 说明                    |
|------------------|----|--------|-----------------------|
| name             | ✗  | string | 窗帘名称，显示在控制面板顶部        |
| fabric_entity    | ✗  | string | 布帘实体 ID，也可用 entity 字段 |
| sheer_entity     | ✗  | string | 纱帘实体 ID（双层模式时需要）      |
| height           | ✗  | string | 自定义该窗帘舞台高度，如 400px    |
| layer_mode       | ✗  | string | 层级模式，覆盖全局默认           |
| open_mode        | ✗  | string | 开合方式，覆盖全局默认           |
| background_image | ✗  | string | 背景图片，覆盖全局默认           |

### 聚合弹窗响应式布局：

| 设备               | 布局     | 说明            |
|------------------|--------|---------------|
| PC / 平板 (≥768px) | 一行 2 个 | 弹窗自动扩展宽度，双列网格 |
| 移动端 (<768px)     | 一行 1 个 | 单列布局，垂直滚动     |

#### (14) 仪表图 (type: chart\_gauge)

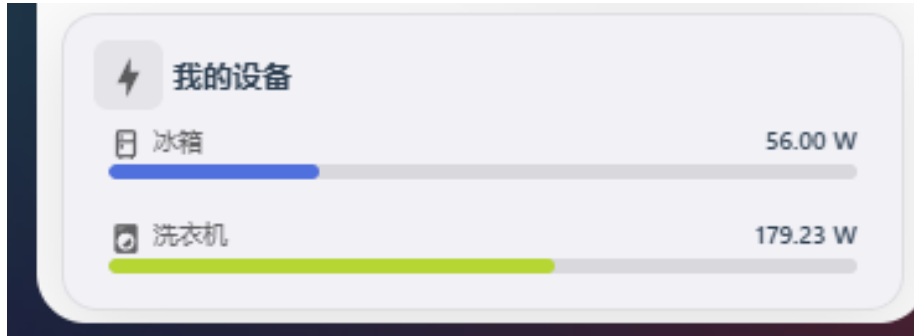
以仪表方式显示实体状态，适用于 cpu 占用率、温度等，该类型的图标使用 ECharts 渲染。



```
- type: chart_gauge
  entity: sensor.canting_bingxiang_chazuo_gonglv
  name: 实时功率
  min: 0
  max: 100
  update_interval: 3 # 每3秒更新一次
```

## (15) 进度条图 (type: chart\_progress)

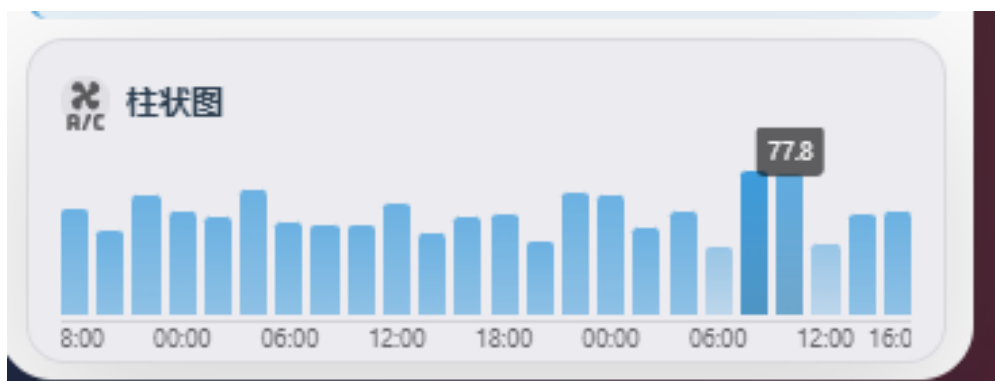
以进度条样式显示实体的状态值，支持多实体。



```
- type: chart_progress
  height: 120px
  name: 我的设备
  icon: mdi:air-conditioner
  update_interval: 3          # 顶层优先级高于子项
  cards:
    - entity: sensor.canting_bingxiang_chazuo_gonglv
      name: 冰箱
      show_name: true
      icon: mdi:fridge-outline
      color: "#5070dd"
      min: 0
      max: 200
      update_interval: 3      # 被顶层覆盖
    - entity: sensor.xiyiji_gonglv
      name: 洗衣机
      show_name: true
      icon: mdi:washing-machine
      color: "#b6d634"
      min: 0
      max: 300
      update_interval: 3
```

## (16) 统计型柱状图 (type: chart\_bar)

以柱状图方式显示指定时长聚合后的实体的历史数据。自适应柱子数量（最多显示 48 根柱子，超出自动裁剪，智能标签，根据柱子密度自动间隔显示时间标签）。



```
- type: chart_bar
  entity: sensor.canting_bingxiang_chazuo_gonglv
  height: 120px
  show_name: false      #是否显示标题 (name 和 icon, 即左上角的图标和标题)
  icon: mdi:air-conditioner
  name: 柱状图
  update_interval: 3
  tap_action:
    action: call-service
    service: switch.toggle
    service_data:
      entity_id: switch.xxx
  bar:
    min: 0
    max: 500
    hours_to_show: 48
    bar_hours: 2
    bar_hours_calc: average # average/max/min/count/max-min/add
```

### 参数说明:

- **bar\_hours\_calc** 支持 6 种计算方式: average(平均值)、max(最大值)、min(最小值)、count(数据点数)、max-min(极差)、add(求和)

- `tap_action` 完整支持: `call-service`、`toggle`、`navigate`、`popup_card`、`more-info`、`none`
- `update_interval`: 按秒数定时刷新图表数据



## (17) 环形图 (type: chart\_pie)

以环形图的方式显示多个实体的数据、占比等信息。该图表使用 ECharts 渲染。



```
- type: chart_pie
  height: 120px
  icon: mdi:lightning-bolt
  name: 我的设备
  show_name: false      #是否显示标题 (name 和 icon, 即左上角的图标和标题)
  update_interval: 3
  cards:
    - entity: sensor.canting_bingxiang_chazuo_gonglv
      name: 冰箱
      icon: mdi:fridge-outline
      color: "#5070dd"
    - entity: sensor.10_ch_monitor_v2_80e2dc_ch_4_power
      name: 洗衣机
      icon: mdi:washing-machine
      color: "#b6d634"
```

(18) 网页卡片 (type: html)

支持 html 语言，支持 js，注意，使用 js 有风险。再使用时请三思，由于使用不当造成的损失与作者无关。

测试html

测试html

21°C / 24.5%

| 日期         | 用量(m³) | 金额(元) |
|------------|--------|-------|
| 2026-05-06 | 0.54   | 0.25  |
| 2026-05-05 | 0.43   | 0.2   |
| 2026-05-04 | 1.46   | 0.68  |
| 2026-05-03 | 3.4    | 1.59  |
| 2026-05-02 | 0.17   | 0.08  |
| 2026-05-01 | 1.56   | 0.73  |
| 2026-04-30 | 2.03   | 0.95  |
| 2026-04-29 | 2.78   | 1.3   |
| 2026-04-28 | 1.41   | 0.66  |
| 2026-04-27 | 1.5    | 0.7   |
| 2026-04-26 | 1.5    | 0.7   |
| 2026-04-25 | 1.24   | 0.58  |
| 2026-04-24 | 1.5    | 0.7   |
| 2026-04-23 | 2.01   | 0.94  |
| 2026-04-22 | 0.64   | 0.3   |
| 2026-04-21 | 2.05   | 0.96  |
| 2026-04-20 | 0.98   | 0.46  |
| 2026-04-19 | 1.31   | 0.61  |
| 2026-04-18 | 0.45   | 0.21  |
| 2026-04-17 | 1.09   | 0.51  |
| 2026-04-16 | 1.8    | 0.84  |
| 2026-04-15 | 1.07   | 0.5   |
| 2026-04-14 | 1.39   | 0.65  |
| 2026-04-13 | 3.6    | 1.68  |
| 2026-04-12 | 1.58   | 0.74  |
| 2026-04-11 | 1.71   | 0.8   |
| 2026-04-10 | 1.99   | 0.93  |
| 2026-04-09 | 2.44   | 1.14  |
| 2026-04-08 | 4.04   | 1.89  |
| 2026-04-07 | 1.9    | 0.89  |

1 / 211

```

- type: html
width: 100%
entities:
  tl: sensor.keting_shidu
  hl: sensor.keting_wendu
  sl: switch.xiaoweishengjian_renzai_led
  dl:
    entity: sensor.tian_ran_qi_2_new_2
    attribute: daylist
  pr:
    entity: sensor.tian_ran_qi_2_new_2
    attribute: price
content: >
  <div style="font-size:16px;margin-bottom:8px">
    <span
style="color: {{IF(tl>30, '#e74c3c', '#3498db')}};font-weight:bold">
      {{ROUND(tl,1)}} ° C
    </span>
    / {{hl}}%
  </div> {{#if sl=='on'}} <div
style="color:#2ecc71;margin-bottom:8px">✅ 电脑已开启
</div> {{/if}}

  <table class="html-table" id="gas-table">
    <thead>
      <tr>
        <th>日期</th>
        <th style="text-align:right">用量(m³)</th>
        <th style="text-align:right">金额(元)</th>
      </tr>
    </thead>
    <tbody>
      {{#each dl as item}}
      <tr>
        <td>{{item.day}}</td>
        <td style="text-align:right">{{ROUND(item.e_gas,2)}}</td>
        <td style="text-align:right">{{ROUND(item.f_gas,2)}}</td>
      </tr>
      {{/each}}
    </tbody>
  </table> <div id="gas-pagination"></div> <script>

(function() {
  var table = document.getElementById('gas-table');
  var pageDiv = document.getElementById('gas-pagination');
  if(!table||!pageDiv) return;
  var tbody = table.querySelector('tbody');
  if(!tbody) return;
  var rows = Array.from(tbody.querySelectorAll('tr'));
  var pageSize = 30;
  var totalPages = Math.ceil(rows.length / pageSize);
  var currentPage = 1;
  function showPage(p) {
    p = Math.max(1, Math.min(totalPages, p));
    currentPage = p;
    rows.forEach(function(r,i) { r.style.display =
(Math.floor(i/pageSize)+1===p)?'':'none'; });
    renderPagination();
  }
  function renderPagination() {
    var html = '<div style="display:flex;align-items:center;gap:6px;font-size:12px;color:rgba(0,0,0,0.6)">';
    html += '<button onclick="window.__gasPrev()"
style="width:26px;height:26px;border:1px solid rgba(0,0,0,0.12);border-radius:6px;cursor:pointer;background:rgba(255,255,255,0.95)"'+(currentPage<=1?' disabled style="opacity:0.4"' : '')+'>&lsaquo;</button>';
    html += '<span style="min-width:50px;text-align:center;font-weight:500">'+currentPage+' / '+totalPages+'</span>';
    html += '<button onclick="window.__gasNext()"

```

```

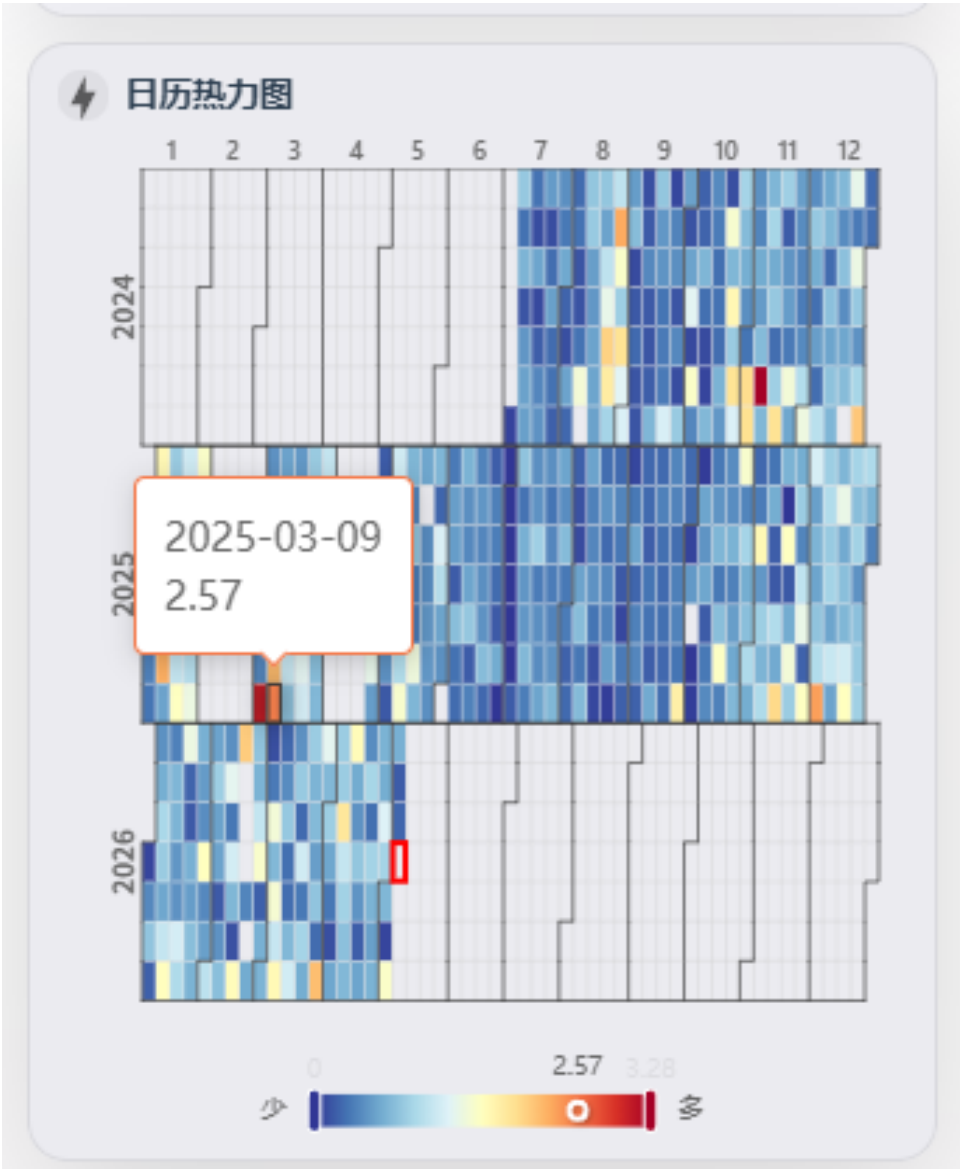
style="width:26px;height:26px;border:1px solid rgba(0,0,0,0.12);border-
radius:6px;cursor:pointer;background:rgba(255,255,255,0.95)"+(currentPage>=t
otalPages?' disabled style="opacity:0.4"':'')+ '&rsquo;</button>';
    html += '<input type="number" id="gas-page-input"
value="'+currentPage+' " min="1" max="'+totalPages+' "
style="width:42px;height:26px;border:1px solid rgba(0,0,0,0.12);border-
radius:6px;text-align:center;font-size:12px;outline:none;-moz-
appearance:textfield">';
    html += '<button onclick="window.__gasGoto()'
style="height:26px;border:1px solid rgba(0,0,0,0.12);border-
radius:6px;cursor:pointer;background:rgba(255,255,255,0.95);font-
size:11px;padding:0 8px">跳转</button>';
    html += '</div>';
    pageDiv.innerHTML = html;
}
window.__gasPrev = function() { showPage(currentPage-
1); };
window.__gasNext =
function() { showPage(currentPage+1); };
window.__gasGoto = function() {
    var input = document.getElementById('gas-page-input');
    if(input) showPage(parseInt(input.value,10)||1);
};
showPage(1);
})(); </script>

```

再次提示，使用 js 有风险，请使用前三思，由于使用不当造成的损失与作者无关。

(19) 日历热力图 (type: chart\_heatmap)

日历热力图 (type: chart\_heatmap), 使用 ECharts 渲染。支持从实体状态属性和 api 两种数据获取方式, 同时支持自定义节点和字段。



① 从实体状态属性获取数据

```
type: chart_heatmap
entity: sensor.xxx
height: 120px
icon: mdi:lightning-bolt
name: 日历热力图
```

```
show_name: false      #是否显示标题 (name 和 icon, 即左上角的图标和标题)
update_interval: 360  #更新时间 (秒)
entity_array:
  attr: daylist      # 属性名, 数据在顶层, 则不填写
  day: day           # 日期字段 (自动处理 yyyy-mm-dd hh:mm:ss → yyyy-mm-dd)
  value: f_gas       # 值字段 (自动处理带/不带双引号的字段名)
```

## ② 从外部 API 接口获取数据

```
type: chart_heatmap
height: 120px
icon: mdi:lightning-bolt
name: 日历热力图
update_interval: 360  #更新时间 (秒)
api_base_url: /api/gateway /clock_device/api
api_array:
  attr: daily_data    # API 返回 JSON 中的数组字段名, 数据在顶层, 则不填写
  day: date           # 日期字段
  value: day_yong_dian_kwh # 值字段
```

(20) 日历图 (type: chart\_calendar)

日历图 (type: chart\_calendar) 使用 ECharts 渲染，支持从实体状态属性和 api 两种数据获取方式，同时支持自定义节点和字段。



① 从实体状态属性获取数据

```
- type: chart_calendar
entity: sensor.tian_ran_qi_2_new_2
height: 120px
icon: mdi:lightning-bolt
name: 日历图
show_name: false #是否显示标题 (name 和 icon, 即左上角的图标和标题)
update_interval: 360
entity_array:
```

```
attr: daylist
day: day
value_1: f_gas
value_1_name: 使用量
value_1_unit: m3
value_1_background: "#00a381"
value_2: e_gas
value_2_name: 金额
value_2_unit: ¥
```

## ② 从外部 API 接口获取数据

```
- type: chart_calendar
api_base_url: /api/gateway/take1943/clock_device/api
height: 120px
icon: mdi:lightning-bolt
name: 日历图
show_name: false      #是否显示标题（name 和 icon，即左上角的图标和标题）
update_interval: 360
api_array:
  attr: daylist
  day: day
  value_1: f_gas
  value_1_name: 使用量
  value_1_unit: m3
  value_1_background: "#00a381"
  value_2: e_gas
  value_2_name: 金额
  value_2_unit: ¥
  value_2_background: "#c85179"
```



(21) 混合图 (type: chart\_mixed)

混合图表卡片 (type: chart\_mixed) 使用 ECharts 渲染，支持柱状图、折线图、面积图的组合，可以是多个也可以是单个，当配置多个时，day 字段会自动对齐，同时支持下钻。

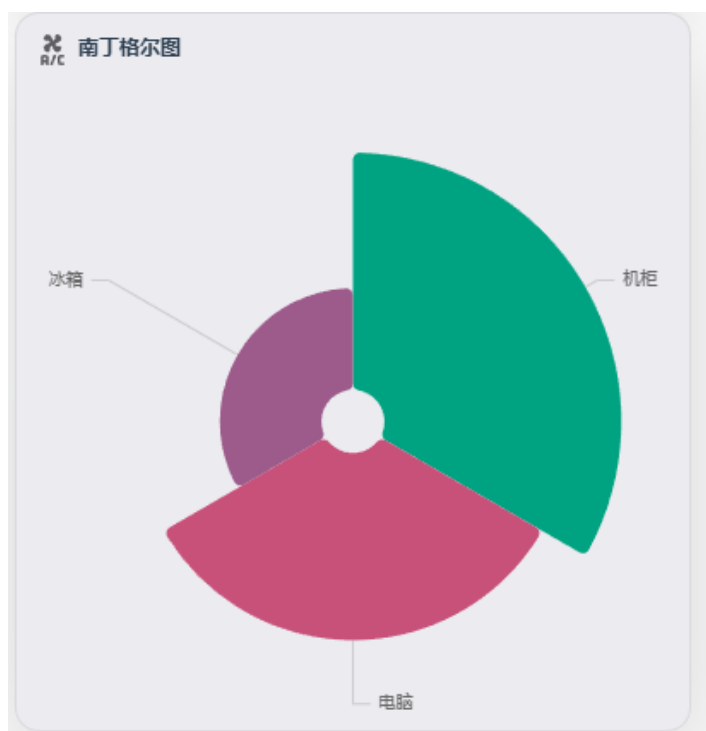


```
type: chart_mixed
height: 320px
icon: mdi:air-conditioner
name: 复合图
show_name: false
update_interval: 360
chart:
  - type: bar
    entity: sensor.xxx
    attr: daylist
    day: day
    value: f_gas
    value_name: 使用量
    value_unit: m³
    bar_color: "#00a381"
  - type: line
    # 可选，不配置时数据在顶层
    # 日期字段
    # 值字段
    # 系列名称
    # 单位
    # 柱状图颜色
```

```
entity: sensor.xxx
attr: daylist
day: day
value: e_gas
value_name: 金额
value_unit: ¥
line_color: "#c85179" # 折线图颜色
- type: area
entity: sensor.xxx
attr: daylist
day: day
value: e_gas
value_name: 趋势
value_unit: ¥
area_color: "#c85179" # 面积图颜色
value_2_unit: ¥
```

## (22) 南丁格尔玫瑰图 (type: chart\_nightingale)

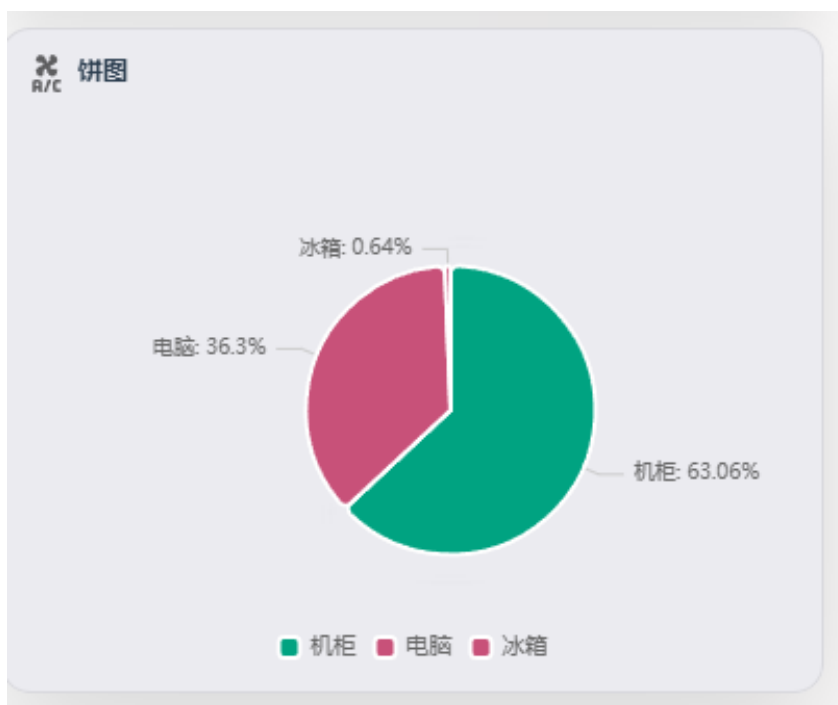
南丁格尔图 (type: chart\_nightingale), 使用 ECharts 渲染, 支持多个实体, 使用实体的状态值来渲染。



```
- type: chart_nightingale
  height: 120px
  icon: mdi:air-conditioner
  name: 南丁格尔图
  show_name: false      #是否显示标题 (name 和 icon, 即左上角的图标和标题)
  update_interval: 10
  chart:
    - entity: sensor.cuco_cn_944196247_v3_electric_power_p_11_2
      name: 机柜
      color: "#00a381"
    - entity: sensor.zimi_cn_1137225298_zncz01_electric_power_p_10_1
      name: 电脑
      color: "#c85179"
    - entity: switch.canting_bingxiang_chazuo
      name: 冰箱
      color: "#c85179"
```

### (23) 完整饼图 (type: chart\_pie\_full)

完整饼图 (type: chart\_pie\_full), 使用 ECharts 渲染, 支持使用多实体状态值来绘制饼图。



```
- type: chart_pie_full
  height: 320px
  icon: mdi:air-conditioner
  name: 饼图
  show_name: false      #是否显示标题 (name 和 icon, 即左上角的图标和标题)
  update_interval: 10
  chart:
    - entity: sensor.cuco_cn_944196247_v3_electric_power_p_11_2
      name: 机柜
      color: "#00a381"
    - entity: sensor.zimi_cn_1137225298_zncz01_electric_power_p_10_1
      name: 电脑
      color: "#c85179"
    - entity: switch.canting_bingxiang_chazuo
      name: 冰箱
      color: "#c85179"
```

## (24) 条件卡片 (type: conditional\_tabs)

条件卡片 (type: conditional\_tabs)，根据条件显示对应卡片，可以适应选项卡或者实体。支持三种用法：自带导航、纯显示容器、纯手动切换。



```
type: conditional_tabs      # 【必填】驱动实体，值变化时自动切换面板
entity: input_select.mode   # 【可选】卡片名称，显示在导航栏左侧
name: 模式切换
show_tabs: true            # 【可选】是否显示导航栏，默认 true，false 时纯
                             靠实体值驱动，不渲染 tab-nav-bar
tab_style: nav             # 【可选】导航栏样式，默认 nav、nav: 下划线风格（复用 tab-
                             nav-bar）、radio: 圆角按钮风格（复用 radio 选项样式）、
items:                     # 【必填】条件面板列表
  # —— 面板 1 ——
  - value: home             # 匹配实体值
    name: 在家模式         # tab 显示名（可选，默认取 value）
    icon: mdi:home         # tab 图标（可选）
    badge: 3               # tab 角标，数字或模板（可选）
    # badge_entity: sensor.home_count # 角标从实体取值（可选，优先于
    badge)
    disabled: false        # 是否禁用此 tab（可选）
    cards:                 # 此面板内的子卡片列表
      - type: switch
        entity: light.living_room
        name: 客厅灯
      - type: sensor
        entity: sensor.temperature
```

```

# —— 面板 2 ——
- value: away
  name: 离家模式
  icon: mdi:shield-check
  color: '#e74c3c'          # tab 高亮颜色（可选，默认主题色）
  cards:
    - type: switch
      entity: alarm_control_panel.home
      name: 安防

# —— 面板 3（最简写法） ——
- value: sleep              # 只有 value + cards
  cards:
    - type: switch
      entity: light.bedroom_night

```

三种典型用法：

### ① 完整选项卡（自带导航）

```

type: conditional_tabs
entity: input_select.mode
show_tabs: true
items:
  - value: home
    name: 在家
    icon: mdi:home
    cards:
      - type: switch
        entity: light.living_room
  - value: away
    name: 离家
    icon: mdi:shield-check
    cards:
      - type: switch
        entity: alarm_control_panel.home

```

### ② 纯显示容器（外部选择器）

```

- type: radio
  entity: input_select.mode
  map_table:
    home: { text: 在家, icon: mdi:home, color: '#2ecc71' }
    away: { text: 离家, icon: mdi:shield-check, color: '#e74c3c' }

# 纯显示，无导航栏
- type: conditional_tabs
  entity: input_select.mode
  show_tabs: false
  items:
    - value: home
      cards:
        - type: switch
          entity: light.living_room
    - value: away
      cards:
        - type: switch
          entity: alarm_control_panel.home

```

### ③ 纯手动切换（无实体）

```
type: conditional_tabs
# 不配置 entity
show_tabs: true
items:
  - value: tab1
    name: 详情
    cards:
      - type: sensor
        entity: sensor.temperature
  - value: tab2
    name: 控制
    cards:
      - type: switch
        entity: light.living_room
```

### 配置项汇总

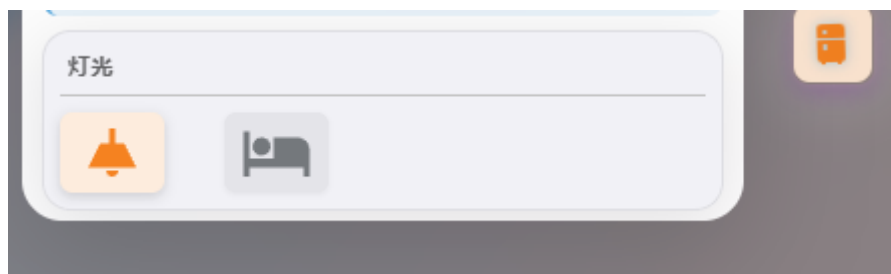
| 字段                   | 必填 | 默认值     | 说明                   |
|----------------------|----|---------|----------------------|
| entity               | 否  | -       | 驱动实体，不配则为纯手动模式       |
| name                 | 否  | -       | 卡片标题                 |
| show_tabs            | 否  | true    | 是否显示导航栏              |
| tab_style            | 否  | nav     | nav 下划线 / radio 圆角按钮 |
| items                | 是  | -       | 面板列表                 |
| items[].value        | 是  | -       | 匹配实体值（手动模式下为标识符）     |
| items[].name         | 否  | 取 value | tab 显示名              |
| items[].icon         | 否  | -       | tab 图标               |
| items[].color        | 否  | 主题色     | tab 高亮颜色             |
| items[].badge        | 否  | -       | tab 角标（固定值）          |
| items[].badge_entity | 否  | -       | tab 角标（从实体取值）        |

| 字段               | 必填 | 默认值   | 说明        |
|------------------|----|-------|-----------|
| items[].disabled | 否  | false | 是否禁用此 tab |
| items[].cards    | 是  | -     | 子卡片列表     |



## (25) 按钮组 (type: button\_group)

将多个小按钮组合在一起，以 Windows 选项卡样式排列在同一个容器中，适合将同一区域的设备（如同一个房间的灯、同一类设备）集中管理。



```
- type: button_group
  direction: horizontal          # (可选) 排列方向: horizontal /
  vertical, 默认 horizontal
  name: 灯光控制                # (可选) 组标签
  on_color: '#3498db'           # (可选) 子按钮开启状态默认颜色
  off_color: '#95a5a6'          # (可选) 子按钮关闭状态默认颜色
  icon_size: 16                 # (可选) 图标大小 px, 默认 16
  show_name: true               # (可选) 是否显示按钮名称, 默认 true
  buttons:                      # 子按钮列表 (必填)
    - entity: light.living_room
      name: 客厅灯
      icon: mdi:ceiling-light
      on_icon: mdi:lightbulb-on  # 可选, 开启时图标
      off_icon: mdi:lightbulb-off # 可选, 关闭时图标
      on_color: '#f1c40f'        # 可选, 覆盖默认 on_color
      off_color: '#95a5a6'       # 可选, 覆盖默认 off_color
      icon_size: 18              # 可选, 覆盖组的 icon_size
      show_name: true            # 可选, 覆盖组的 show_name
      confirm: true              # 可选, 点击前弹出确认对话框
      tap_action:
        action: toggle           # 支持 toggle / set_value / more-info
        / navigate / call-service / popup_card / none
      badge:                     # 可选, 条件角标数组
        - entity: binary_sensor.motion_living
          condition: 'on'
    - entity: light.bedroom
      name: 卧室灯
      icon: mdi:bed
    - entity: light.kitchen
      name: 厨房灯
      icon: mdi:lightbulb
```

## 顶层配置项

| 配置项       | 必填 | 类型      | 默认值        | 说明                                       |
|-----------|----|---------|------------|------------------------------------------|
| type      | ✓  | string  | —          | 固定为 button_group                         |
| direction | ✗  | string  | horizontal | 排列方向，horizontal（水平）<br>/ vertical（垂直）    |
| name      | ✗  | string  | 无          | 组标签，显示在按钮组上方（小字）                         |
| on_color  | ✗  | string  | #3498db    | 子按钮开启状态的默认颜色                             |
| off_color | ✗  | string  | #95a5a6    | 子按钮关闭状态的默认颜色                             |
| icon_size | ✗  | number  | 16         | 子按钮图标大小（px），子按钮可单独覆盖                     |
| show_name | ✗  | boolean | true       | 是否显示子按钮文字，子按钮可单独覆盖                       |
| gap       | ✗  | string  | 4px        | 按钮间距，支持 CSS 间距值<br>（如 30px、8px、12px 8px） |
| buttons   | ✓  | array   | []         | 子按钮列表                                    |

### 子按钮（buttons 中的每项）配置

| 配置项     | 必填 | 类型     | 默认值  | 说明                  |
|---------|----|--------|------|---------------------|
| entity  | ✓  | string | —    | 实体 ID，用于读取状态和自动推断服务 |
| name    | ✗  | string | 实体后缀 | 按钮显示名称              |
| icon    | ✗  | string | 无    | 按钮图标（启用时也使用此图标）     |
| on_icon | ✗  | string | icon | 开启时图标               |

| 配置项        | 必填 | 类型      | 默认值                  | 说明                      |
|------------|----|---------|----------------------|-------------------------|
| off_icon   | ✗  | string  | icon                 | 关闭时图标                   |
| on_color   | ✗  | string  | 组 on_color           | 开启时颜色，覆盖组默认             |
| off_color  | ✗  | string  | 组 off_color          | 关闭时颜色，覆盖组默认             |
| icon_size  | ✗  | number  | 组 icon_size          | 图标大小 (px)，覆盖组默认         |
| show_name  | ✗  | boolean | 组 show_name          | 是否显示按钮名称，覆盖组默认          |
| confirm    | ✗  | boolean | false                | 点击前是否弹出确认对话框            |
| tap_action | ✗  | object  | { action: 'toggle' } | 点击动作，支持所有标准动作类型         |
| badge      | ✗  | array   | 无                    | 条件角标数组，满足条件的规则数量显示为红色角标 |

## badge 子项配置

每个 badge 规则格式与主系统 badge 一致：

| 配置项       | 必填 | 类型                  | 说明                                  |
|-----------|----|---------------------|-------------------------------------|
| entity    | ✓  | string              | 要判断状态的实体 ID                         |
| condition | ✗  | string/array/object | 条件判断，缺省时实体非 unknown/unavailable 即为真 |

condition 支持的格式：

| 格式  | 说明       | 示例            |
|-----|----------|---------------|
| 字符串 | 实体状态等于该值 | condition: on |

| 格式     | 说明                          | 示例                                                                |
|--------|-----------------------------|-------------------------------------------------------------------|
| 状态     |                             |                                                                   |
| 字符串表达式 | 数值比较                        | condition: '<10'、>=50、!=0                                         |
| 数组     | OR 语义，任一满足即可                | condition: ['<10', '>100']                                        |
| 对象     | 透传给条件引擎                     | condition: { operator: '>', value: 50, attribute: 'temperature' } |
| 缺省     | 实体非 unknown/unavailable 即为真 | 不写 condition                                                      |

## 动作类型（tap\_action）

按钮组内的子按钮支持以下 tap\_action 类型：

| action       | 说明                       |
|--------------|--------------------------|
| toggle       | 切换实体状态（默认）               |
| set_value    | 设置实体目标值，需配置 value        |
| more-info    | 弹出 HA 实体详情弹窗             |
| navigate     | 页面导航，需配置 navigation_path |
| call-service | 调用 HA 服务，需配置 service     |
| popup_card   | 弹出自定义卡片，需配置 card         |
| none         | 不执行操作                    |

## 垂直布局（vertical）

设置 direction: vertical 可使按钮垂直排列，每个按钮占满

整行宽度：

```
- type: button_group
  direction: vertical
  name: 房间设备
  buttons:
    - entity: light.living_room
      name: 客厅灯
      icon: mdi:ceiling-light
    - entity: fan.living
      name: 客厅风扇
      icon: mdi:fan
    - entity: switch.living_tv
      name: 电视
      icon: mdi:television
```

## 仅图标模式

设置 `show_name: false` 可隐藏按钮名称，只显示图标，  
适合按钮较多或空间有限的场景：

```
- type: button_group
  show_name: false
  icon_size: 18
  buttons:
    - entity: light.living_room
      icon: mdi:ceiling-light
    - entity: light.bedroom
      icon: mdi:bed
```

## 样式说明

按钮组以浅色圆角容器包裹，每个子按钮为紧凑小按钮，  
默认显示图标+名称：

- **水平布局**：按钮从左到右排列，自动换行。
- **垂直布局**：按钮从上到下排列，每行占满。
- **开启状态**：使用配置的 `on_color` 背景 + 白色文字。
- **关闭状态**：使用浅灰背景 + 灰色文字。

- **鼠标悬停：**轻微上浮，点击时微缩。
- **角标：**配置了 `badge` 的子按钮在右上角显示红色计数角标。
- **确认：**配置了 `confirm: true` 的子按钮点击后弹出确认对话框。

(26) 快捷操作 (type: action)

快捷操作卡片将情景模式的功能集成到统一卡片系统中，可在自由布局弹窗、title\_entities 等任意支持 card\_config 的位置使用。点击卡片弹出与 scene\_mode 相同的情景模式气泡，支持多步骤快捷操作、执行进度展示和状态验证。

与 scene\_mode 按钮的区别： scene\_mode 是头部模式下的专属按钮类型，action 是统一卡片系统的一种卡片类型，可在弹窗、title\_entities 等任意位置使用，功能完全相同。



```
- type: action
  name: 情景模式
  icon: mdi:palette
  icon_color: "#3498db"
  description: 3 个模式
  scene_mode: bubble
direct (直接执行)
scenes:
  全一致)
  - name: 回家模式
    confirm: true
    actions:
      - entity: light.living_room
        value: "on"
        delay: 2
      - entity: climate.ac
        service_data:
          temperature: 26
  - name: 离家模式
    actions:
      - entities:
          客厅灯: light.keting_dadeng
          卧室灯: light.bedroom
        value: "off"
```

# 显示名称  
# 图标  
# 图标颜色  
# 描述文本（不配则自动显示"N 个模式"）  
# 执行模式：bubble（气泡选择）|  
# 情景列表（配置与 scene\_mode 完

## 配置项说明：

| 配置项            | 必填 | 类型      | 默认值         | 说明                                       |
|----------------|----|---------|-------------|------------------------------------------|
| type           | ✓  | string  | —           | 固定为 action                               |
| name           | ✗  | string  | 快捷操作        | 显示名称                                     |
| icon           | ✗  | string  | mdi:palette | 图标                                       |
| icon_color     | ✗  | string  | #3498db     | 图标颜色                                     |
| description    | ✗  | string  | 自动          | 描述文本，不配置时自动显示"N 个模式"                     |
| scenes         | ✓  | array   | []          | 情景列表，配置格式与 scene_mode 完全一致               |
| scene_mode     | ✗  | string  | 自动          | 执行模式：bubble（气泡选择）/ direct（直接执行），不配置时自动判断 |
| fold           | ✗  | boolean | false       | 动作列表是否默认展开                               |
| verify_timeout | ✗  | number  | 10          | 执行后验证状态的超时秒数                             |

scenes 的完整配置格式（actions 的 entity/entities/value/service\_data/delay/confirm/select 等）与情景模式完全一致，此处不再重复。

## 在自由布局弹窗中使用：

```
cards:
  - row: 1
    per_line: 1
    items:
```



```

- type: action
  name: 快捷操作
  icon: mdi:palette
  scenes:
    - name: 回家模式
      actions:
        - entity: light.living_room
          value: "on"

```

在 `title_entities` 中使用：

```

title_entities:
- type: action
  name: 情景
  icon: mdi:palette
  scenes:
    - name: 回家
      actions:
        - entity: light.living_room
          value: "on"

```

无论哪种按钮类型，都可以配置以下通用字段：

| 配置项          | 说明          | 示例                |
|--------------|-------------|-------------------|
| entity       | 绑定的实体 ID    | light.living      |
| name         | 显示名称        | 客厅灯               |
| icon         | 默认图标        | mdi:lightbulb     |
| on_icon      | 开启时图标       | mdi:lightbulb-on  |
| off_icon     | 关闭时图标       | mdi:lightbulb-off |
| on_color     | 开启时颜色       | #2ecc71           |
| off_color    | 关闭时颜色       | #e74c3c           |
| show_badge   | 显示角标数字      | true              |
| badge_entity | 指定实体值作为角标   | sensor.count      |
| badge        | 条件角标数组（独立按钮 | 见 5.16 节          |

| 配置项     | 说明                   | 示例                              |
|---------|----------------------|---------------------------------|
|         | 专用)                  |                                 |
| confirm | 点击是否需要确认             | true                            |
| primary | 按钮下方的文字              | 已开启 3 个                         |
|         | tap_action           | 点击动作 (见下文)                      |
|         | width                | 弹窗宽度 (像素)                       |
|         | per_line             | 弹窗每行按钮数                         |
|         | row_column           | 网格位置 (head 按钮/<br>弹窗 item 跨行跨列) |
|         | close_time           | 弹窗自动关闭倒计时<br>(秒), 0=不自动关闭       |
|         | show_duration        | 显示距上次变化的持续<br>时长                |
|         | popup_position       | 弹窗位置                            |
|         | tabs_by              | 聚合弹窗按字段分组<br>(如 room / group)   |
|         | group_lights_by_room | 灯光弹窗中按 room 字<br>段分组显示设备        |

(27) 健康 (type: health) v4.0.0 新增

健康数据管理面板，支持记录和查询家庭成员的健康指标  
(血压、身高体重、体温)。点击按钮弹出弹窗，包含查询和新增两个功能标签页。



```
- type: health
  name: 健康档案
  icon: mdi:heart-pulse
  user: CM,CHJ,SPP # 家庭成员姓名列表（逗号分隔）
  popup_position: clone_button_down
  width: 480px
```

该功能需要后端配置，需要“HA 数据统一存储系统”支持的支持。

配置项

| 配置项            | 必填 | 类型             | 默认值             | 说明                                  |
|----------------|----|----------------|-----------------|-------------------------------------|
| type           | ✓  | string         | —               | 固定为 health                          |
| name           | ✗  | string         | 健康数据            | 弹窗标题                                |
| icon           | ✗  | string         | mdi:heart-pulse | 按钮图标                                |
| user           | ✗  | string 或 array | —               | 家庭成员姓名列表，可用逗号分隔字符串或数组 ["CM", "CHJ"] |
| api_base_url   | ✓  | string         | —               | 可不填写，顶部配置后此处省略                      |
| key            | ✗  | string         | —               | 可不填写，顶部配置后此处省略                      |
| popup_position | ✗  | string         | —               | 弹窗弹出位置                              |
| width          | ✗  | string         | 480px           | 弹窗宽度                                |

功能说明

点击健康按钮弹出弹窗，包含两个功能标签页：

### 查询标签页：

- **成员选择：**通过姓名按钮切换要查询的家庭成员，支持"全部"查看所有人的数据
- **最新数据概述：**查询后显示该成员的最新一次各项健康指标快照（身高/体重/BMI、血压、脉搏、体温），并标注数据更新时间
- **图表视图（默认）：**基于 ECharts 的趋势折线图，按子类型（身高体重/血压/脉搏/体温）切换显示；支持 dataZoom 缩放和平滑曲线
- **表格视图：**列表展示所有记录，支持按类型筛选（全部/血压/身高体重/体温）

### 新增标签页：

- **姓名：**选择要记录的家庭成员
- **日期时间：**记录的时间点（默认为当前时间）
- **数据类型：**支持三种类型
  - **血压 (BP)：**收缩压/高压 (sp, mmHg)、舒张压/低压 (dp, mmHg)、脉搏 (pr, 次/分)
  - **身高体重 (WT)：**身高 (height, cm)、体重 (weight, kg)、BMI（填身高体重后自动计算）

。 体温 (temp)：体温 (temp, °C)

- 提交后数据自动保存，页面不刷新即可继续录入

## API 接口

后端需提供以下 RESTful 接口：

查询接口：

GET {api\_base\_url}query?type=health\_history&key={key}&name={name}

响应格式：{ "success": true, "data": { "rows": [...] } }

当 name 为空时返回所有成员的数据

新增接口：

- POST {api\_base\_url}health/add?key={key}
- 请求体 (JSON)：

```
{
  "name": "CM",
  "date_time": "2026-06-03 17:30:45",
  "type": "BP",
  "sp": 120,
  "dp": 80,
  "pr": 72
}
```

- 响应格式：{ "success": true }

数据字段说明：

| 字段        | 类型     | 说明                         | 所属类型 |
|-----------|--------|----------------------------|------|
| name      | string | 家庭成员姓名                     | 通用   |
| date_time | string | 时间点，格式 YYYY-MM-DD HH:mm:ss | 通用   |
| type      | string | 数据类型：BP / WT / temp        | 通用   |
| sp        | number | 收缩压/高压 (mmHg)              | BP   |

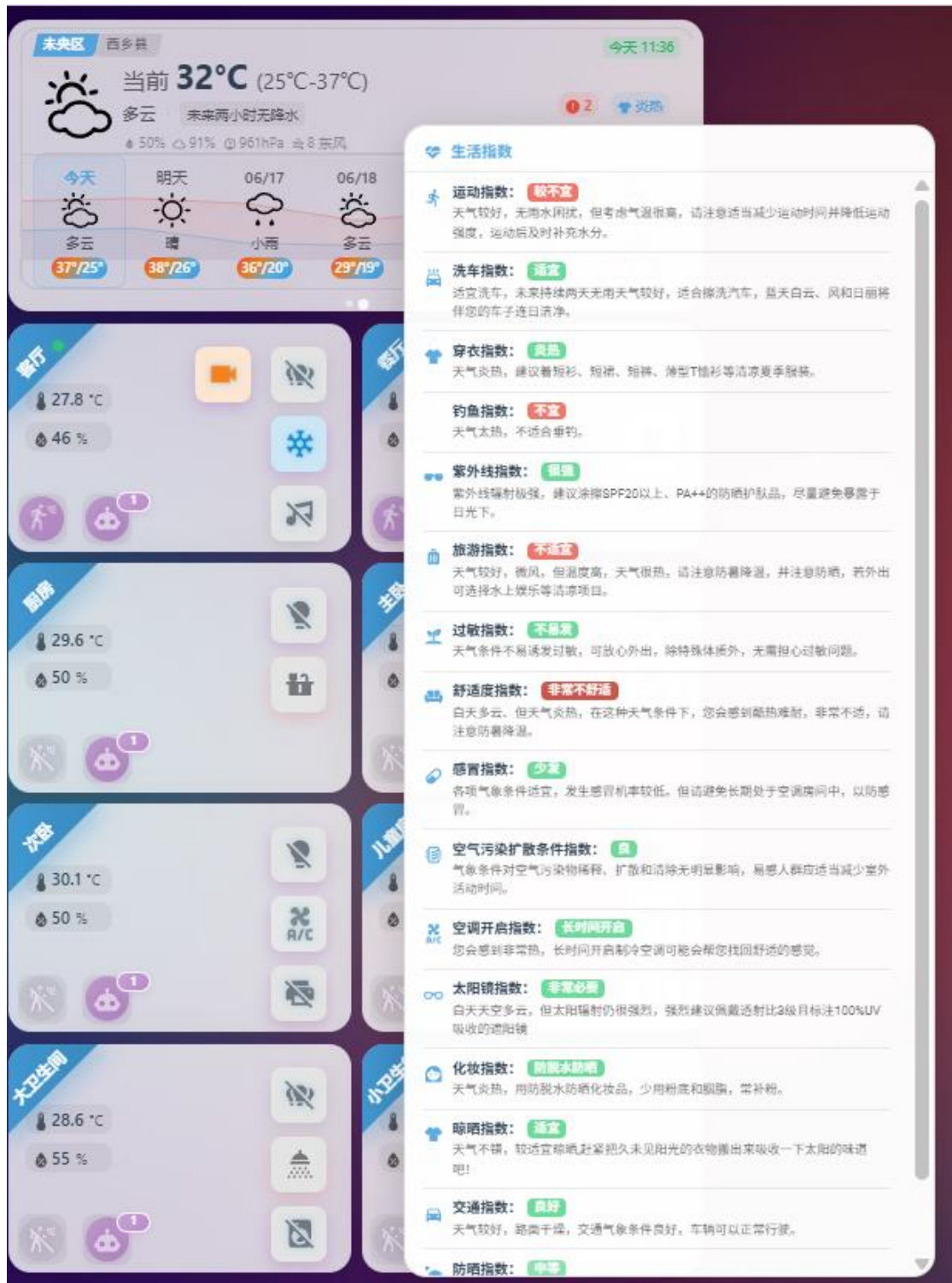
| 字段     | 类型     | 说明              | 所属类型 |
|--------|--------|-----------------|------|
| dp     | number | 舒张压/低压 (mmHg)   | BP   |
| pr     | number | 脉搏 (次/分)        | BP   |
| height | number | 身高 (cm)         | WT   |
| weight | number | 体重 (kg)         | WT   |
| bmi    | number | BMI 值 (不传则自动计算) | WT   |
| temp   | number | 体温 (℃)          | temp |

(28) 天气卡片 (type: weather) v5.0.0 新增

新增天气卡片，可以随处调用，建议使用和风天气的集成，该集成有小时预报，点击今日弹出今日的按小数预报天气







```
type: weather
weather:
  - name: 未央区
    entity: weather.qweather_pro_wei_yang_weather
  - name: 西乡县
    entity: weather.qweather_pro_xi_xiang_weather
```

## (29) 调试卡片 (debug\_card) v5.6.4 新增

卡片内置了一套统一的调试信息管理系统

(DebugManager)，实现了“一个公共方法，三处同步”的调试信息查看机制，方便排查卡片/模块问题。

debug\_card 头部选项卡：弹窗头部含「调试 / 性能 / 版本 / 数据库」四个选项卡。

- 调试：模块开关 + 调试文本直显（本节前述功能）
- 性能：运行态性能指标（更新耗时 / 帧率 / 索引 / 定时器 / 后端请求等 13 项 + 控制台命令）
- 版本：卡片/后端版本信息 + HA 服务器信息概览 + 系统资源 24h 趋势图 + 已注册调试模块
- 数据库：v5.6.4 起开始内置数据库浏览器（表选择 + 字段选择 + 多条件筛选 + 浏览/修改模式原地编辑 + 前端 SQL 执行 + 分页/表头排序）。

同时支持浏览器控制台命令查看，命令如下：

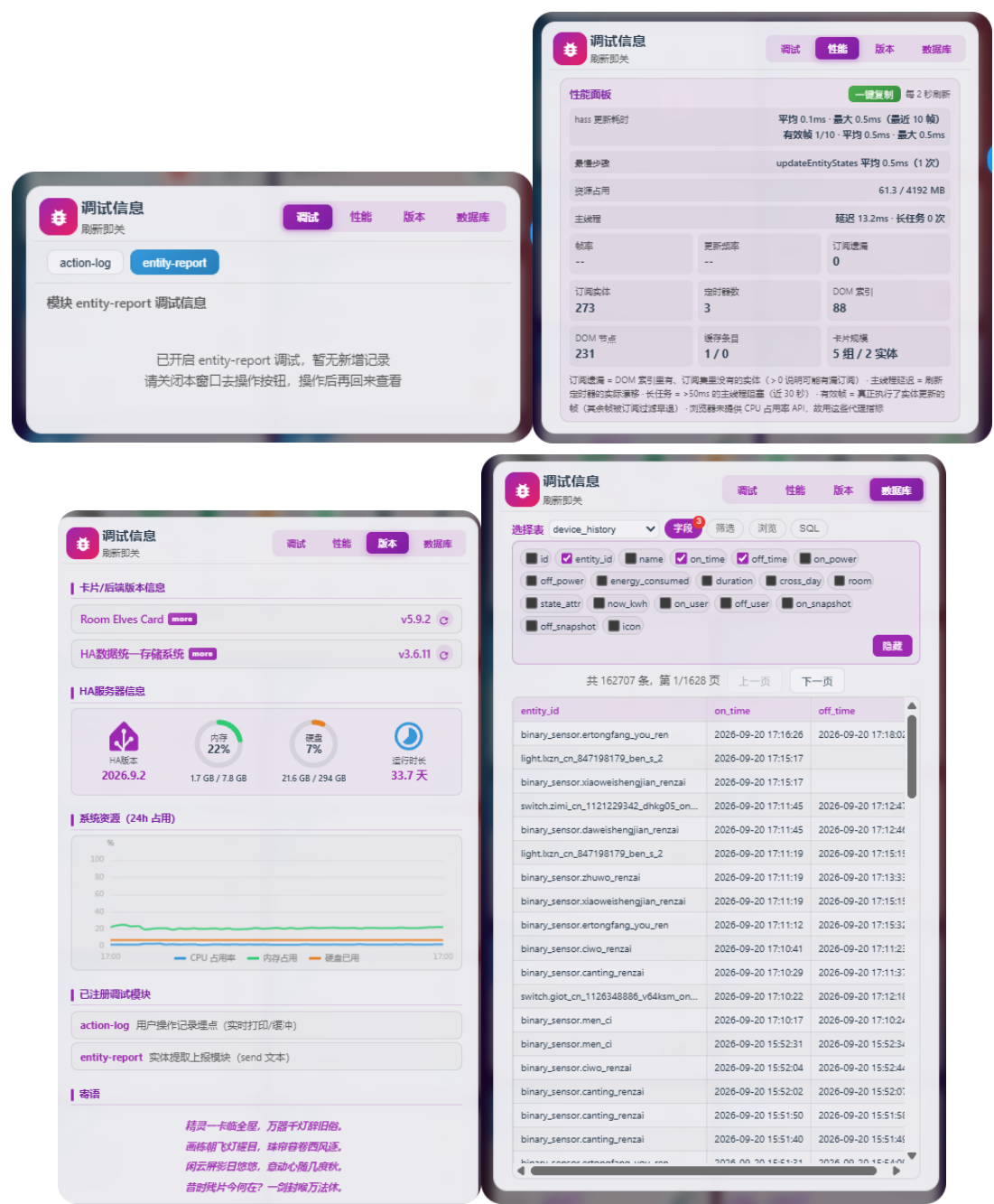
```
roomElvesDebug() // 帮助 + 所有模块开关状态
roomElvesDebug('list') // 列出已注册模块
roomElvesDebug('<模块名>') // 切换开关（如 entity-report）
roomElvesDebug('<模块名>', true/false) // 显式开/关
roomElvesDebug('all') // 全部开启
roomElvesDebug('none') // 全部关闭
roomElvesDebug('view', '<模块名>') // 查看该模块已发送的调试信息
roomElvesDebug('dump', '<模块名>') // 输出该模块 dump() 缓冲数据
roomElvesDebug('clear', '<模块名>') // 清空该模块缓冲
```

示例：

```
roomElvesDebug('entity-report') // 开启上报模块调试
roomElvesDebug('entity-report', false) // 关闭上报模块调试
roomElvesDebug('view', 'entity-report') // 查看上报模块已发送的调试信息
```

已接入的调试模块

| 模块            | 说明       | 接入方式                      |
|---------------|----------|---------------------------|
| action-log    | 用户操作记录埋点 | register + dump（完整操作记录字段） |
| entity-report | 实体提取上报   | register + send（完整实体字段）   |



```
buttons:  
  - name: 调试  
    tap_action:  
      action: popup_card  
    card:  
      type: debug_card  
      name: 调试信息  
      width: 560px
```

### (30) 自动化卡片 (type: automations) v5.5.0 新增

在概览栏最左侧固定入口列 (.ov-entry-col, 约 70px, 与"最近使用设备"横向并排两格、独立成列、带右分隔线) 增加"自动化管理"入口 (ov-seg ov-automations), 点击弹出自动化管理弹窗卡片, 含两个选项卡:

**信息:** 自动化整体统计 + 明细列表。

- 数据源 sensor.ha\_data\_store\_automation 实体: state = 启用自动化个数
- attributes: total/enabled/disabled = 总数/启用/停用; success/failed/skipped/never = 最近一次执行结果统计 (成功/失败/条件跳过/从未执行); total\_runs/updated\_at = 总执行次数/数据更新时间; automations[] = 明细项 { id, name, enabled, trigger\_type, trigger\_desc, stop\_on\_error, next\_run, last\_run, last\_result, last\_duration\_ms, run\_count, success\_count, failed\_count, skipped\_count }

**编辑:** 自动化完整管理 (后端 REST API, api\_base\_url + key 鉴权)

- GET /api/ha\_data\_store/automations 列表
- POST /api/ha\_data\_store/automations 新建

● PUT

/api/ha\_data\_store/automations/{id} 更新（启停/编辑）

● DELETE

/api/ha\_data\_store/automations/{id} 删除

● POST

/api/ha\_data\_store/automations/{id}/run 手动运行  
（?force=1 跳过条件）

| 字段          | 必填 | 类型            | 说明                                                            |
|-------------|----|---------------|---------------------------------------------------------------|
| automations | ✓  | string/object | 数据源实体 ID 字符串，或 { entity } 对象（写法 1）                            |
| type        | ✓  | string        | 固定为 automations（写法 2）                                         |
| entity      | ✗  | string        | 数据源传感器，覆盖默认实体 sensor.ha_data_store_automation；兼容点号写法自动归一化为下划线 |
| name        | ✗  | string        | 分段/弹窗标题，默认「自动化管理」                                             |
| width       | ✗  | string        | 弹窗宽度，默认 640px                                                 |

注意：需要“HA 统一存储系统 v3.0.0 版本”的支持



"自动化管理"卡片，点击入口弹出通用弹窗，含两个选项卡，另外 sensor.ha\_data\_store\_automation 实体是“HA 统一存储系统”自动生成的：

- 信息选项卡：自动化整体统计 + 明细列表
- 编辑选项卡：自动化完整管理（后端 REST API，

api\_base\_url + key 鉴权）

配置方法（2 种写法）

```
# 写法 1: overview 键名 automations (值为实体 ID 字符串, 或 { entity } 对象)
overview:
  - automations: "sensor.ha_data_store_automation"
    width: "400px"
    name: "自动化管理"

# 写法 2: 独立弹出窗口 (buttons 按钮 + tap_action popup_card, card.type:
automation)
buttons:
  - name: 自动化管理
    icon: mdi:robot
    tap_action:
      action: popup_card
      card:
        type: automation
        entity: sensor.ha.data.store.automation
        name: 自动化管理
        width: 400px
```



(31) 最近使用卡片 (recently-used) v5.4.0 新增

在概览栏 (Overview Bar) 最左侧新增"常用"入口, 点击弹出最近使用设备气泡卡片, 展示最近使用设备列表 (按使用时间倒序、同实体去重)。点击设备项可按其对应弹窗打开; 可开关设备 (插座/灯) 右侧显示切换开关, 点击直接切换。

**依赖:** 需配合 HA 统一数据存储系统 2.13.0 及以上版本 (提供"近期使用设备"实体及 config\_id / device\_type 字段)。

配置说明

在 overview 数组中配置 recently-used (值 = "近期使用设备"实体的实体 ID, 该实体 ID 是 HA 统一数据存储系统自动生成的, 无需修改):

```
overview:
  - recently-used:
    sensor.hashu_ju_tong_yi_cun_chu_xi_tong_jin_qi_shi_yong_she_bei
    width: 430px # 可选, 气泡宽度, 默认 320px (空数据时 300px)
    maxItems: 20 # 可选, 气泡中最多显示的设备数, 默认 50
```



## 功能：

- 入口：概览栏最左侧固定入口列（.ov-entry-col，约 70px）中与"自动化管理"横向并排一格，显示"常用"文本 + 去重后的设备个数；固定入口列自带右分隔线，与中间动态指标列（.ov-left-col）隔开，点击入口列间隙/边距不会误开"全量概览"弹窗。
- 气泡宽度：可通过 width 参数自定义（如 430px），未配置时默认 440px。
- 数据集选项卡「卡片 / 全部」（recently-used-header 右端，胶囊样式 .recently-used-view-tabs；默认「卡片」，无 devices 数据时自动落到「全部」）：
- 卡片（默认）：数据源为实体 attributes.devices[]（用户操作过的设备卡片，含 user\_name / config\_id），即原有内容；标题「卡片内最近使用 (N)」。
- 全部：数据源为实体 attributes.all[]（全部受监控设备，无用户维度：room → 房间名，无 user\_name / config\_id，点击按实体反查配置树）；列表行布局、图标/状态标签、房间标签、右侧控件、点击弹窗、实时状态同步与「卡片」完全一致，只是数据不同；标题「全部最近使用 (N)」。
- 切换选项卡时重建下方子选项卡行（用户筛选 + 时间线 + 时空），两个选项卡各自的「时空 / 时间线」惰性渲染状态

互相独立；标题括号内数量为该数据集去重后的设备总数，不随用户筛选变化。

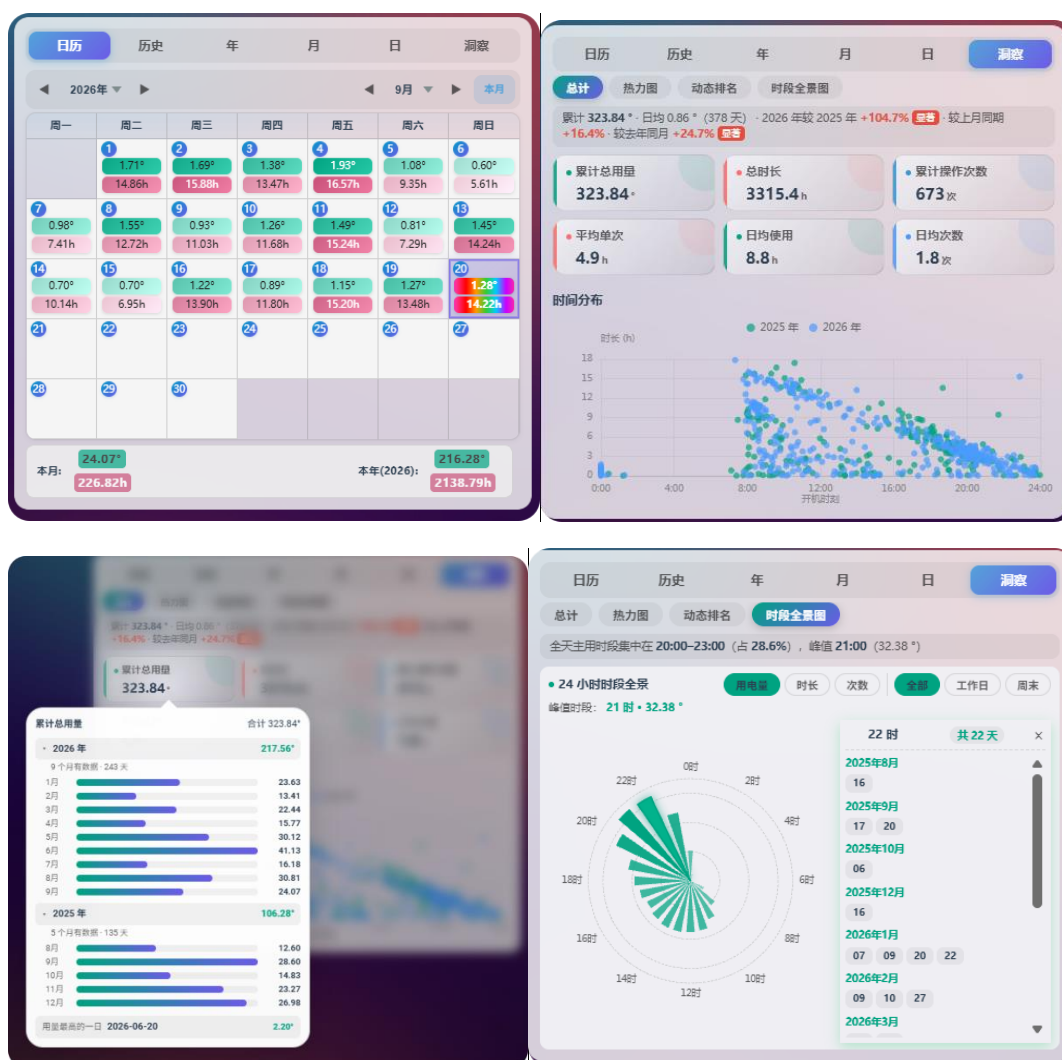
- 气泡列表：按 `entity_id` 去重、按最近使用时间倒序；设备项之间显示分割线；header 固定不滚动、列表独立滚动；图标按实体/名称推断（实体域 `climate` → `mdi:air-conditioner`，名称含"灯"或"照片" → `mdi:lightbulb`，名称含"插座/插排" → `mdi:power-socket-au`）。
- 用户筛选：子选项卡行按 `user_name` 筛选（"全部" + 各用户选项卡，复用 `eh-tabs` 样式），默认选中当前登录用户；「全部」数据集无用户维度 → 不显示用户选项卡。
- "时间线"选项卡（子选项卡行内固定项）：按时间倒序展示当日事件（日期导航 ◀ 日期 ▶ + 可滚动列表），条目 = 左侧节点（有用户 → 用户头像；无用户 → 事件来源图标，底色按 `source` 区分）+ 设备名 + 状态徽章 + 房间/状态/用户 + 右下角操作控件。数据源按数据集区分：

## (32) 使用量日历卡片 (usage\_calendar) 新增

使用量日历卡片用于展示任意实体的用量/能耗/时长数据，支持日历视图和年/月/日 ECharts 图表视图。数据来源既可以是 HA 数据统一存储系统 (ha\_data\_store) API，也可以是实体属性。

**注意：**需要 HA 统一存储系统集成的数据支持。

六个选项卡视图：





| 选项卡 | 说明                                                                                                |
|-----|---------------------------------------------------------------------------------------------------|
| 日历  | 月历网格，每个单元格显示当日用量和时长，点击有数据的日期弹出详情                                                                  |
| 历史  | 两张对比图：「历史本月」与「历史今日」                                                                               |
| 年   | 年度柱状图（ECharts），显示所有可用年份的用量/能耗对比                                                                   |
| 月   | 月度柱状图（ECharts），显示选定年份 12 个月的用量/能耗对比                                                               |
| 日   | 日度柱状图（ECharts），显示选定月份每日的用量/能耗对比                                                                   |
| 洞察  | 跨年 / 全局视角（v5.7.3 新增），顶部筛选按钮切换四个子视图：总计 / 热力图 / 动态排名 / 时段全景图；其中时段全景图（v5.7.1 新增）支持点击柱体下钻到「该小时有使用的日期」 |

## 数据源配置：

支持三种数据源：API 单实体、API 多实体聚合、实体属性。优先使用 API，未配置 API 时使用实体属性。

## 方式一：API 数据源（推荐）

通过 api 配置块对接 ha\_data\_store,  
api\_base\_url 和 key 可省略（自动从顶层配置继承）:

```
- type: usage_calendar
  api:
    entity: input_boolean.bing_xiang # 设备实体 ID (必需)
    # api_base_url: /api/ha_data_store/ # 可选, 省略时从顶层继承
    # key: your_api_key # 可选, 省略时从顶层继承
```

方式二：实体属性数据源，通过 entities 配置块从实体属性读取数据:

```
- type: usage_calendar
  entities:
    data_entity: sensor.tianranqi # 数据实体 ID
    consumption_attr: gas_consumption # 用量属性名, 默认
gas_consumption
    duration_attr: gas_duration_seconds # 时长属性名 (秒), 默认
gas_duration_seconds
    date attr: date # 日期属性名, 默认 date
```

方式三：多实体聚合（多个实体合计），配置 api.entities（实体数组）即进入多实体聚合模式：由后端把这一组实体合并成合计返回，卡片按合计展示全部选项卡。

```
- type: usage_calendar
  title: 全屋空调用电量日历
  api:
    entities:
      - climate.keting_ac_keting_ac # 简写: 只给实体 ID
      - entity: climate.zhuwo_xxx
        name: 主卧空调 # 可选: 图表/图例/日历角标显示的名字
        color: "#4C9AFF" # 可选: 柱状图堆叠颜色 (不写自动分配)
      - { entity: climate.ertongfang_xxx, name: 儿童房空调 }
```

多实体模式下的差异:

| 能力 | 说明                                                                                       |
|----|------------------------------------------------------------------------------------------|
| 数据 | 走 device_usage_multi / device_usage_detail / entities_hours_agg 等多实体接口，各选项卡展示的都是这一组实体的合计 |
| 标题 | 自动追加「(N 个设备合计)」后缀                                                                        |

| 能力      | 说明                                                                                                      |
|---------|---------------------------------------------------------------------------------------------------------|
| 柱状图     | <b>按设备堆叠</b> ：每台设备一段，颜色取配置的 color（未配置时按 8 色调色板自动分配）                                                     |
| 日历角标    | 单元格左上角标注 <b>当天有几台设备有数据</b> （形如 2 台）。它与用量数值 <b>互相独立</b> —— 某设备当天只瞬间开关过、用量四舍五入为 0 时数值不显示，但「当天有几台设备动过」仍然显示 |
| 时段全景图下钻 | 点击柱子所在的 <b>那一段</b> 即下钻到 <b>该设备</b> ：右侧列出「该设备在这一个小时有使用的日期」（标题带设备名），点日期看明细                                |
| 每日明细    | 单日明细为 <b>分设备甘特图</b> ，逐台展开每段开关记录                                                                         |
| 图表图例    | 洞察/年/月/日等图表自动带 <b>设备图例</b> ，可点选隐藏某台设备                                                                   |

## 完整配置：

|                                                                                                                                                                                                                                                                                                                                         |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> type: usage_calendar title: 客厅空调用电量日历 show_title: false width: 400px show_popup: true api:   entity: climate.keting_ac_keting_ac   calc_value_choose: value   value_font_size: 10px   date_font_size: 10px   data_value_color: "#00a381"   calc_value_color: "#c85179"   units: "° ,h"   decimals: "2,2"   series: "用量,时长" </pre> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 在 popup\_card 中使用（vertical-stack 嵌套）：

|                                                                                                                                                                                                                                                                                             |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> - name: 客厅空调   entity: climate.keting_ac_keting_ac   tap_action:     action: popup_card     popup_position: clone_button_down   card:     type: vertical-stack     cards:       - type: ac         entity: climate.keting_ac_keting_ac         name: 客厅空调         width: 400px </pre> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

```
- type: usage_calendar
  title: 客厅空调用电量日历
  show_title: false
  width: 400px
  show_popup: true
  api:
    entity: climate.keting_ac_keting_ac
  calc_value_choose: value
  units: "°C,h"
  series: "用量,时长"
```

### 选项卡导航说明：

- **日历选项卡：**底部显示 ◀ 年份 ▶ ◀ 月份 ▶ 本月 导航条，支持快速切换年月；底部显示本月/本年用量和时长汇总
- **年选项卡**（v5.7.0 起为跨年对比分析）：顶部筛选按钮切换子视图，无年份导航栏
  - **总计：**全部年份合计指标卡（累计总用量 / 总时长 / 累计操作次数，三卡单行）+ 逐年对比表（年份 | 用电量 | 总时长，用电量带占比进度条，最新年份带「最新」徽标）
  - **年度：**年度柱状图，显示所有可用年份的用量对比（默认子视图）
  - **热力图：**日用电热力图，横向 12 个月、纵向日期，颜色深浅表示单日用量；标题右侧可切**用量 / 时长**两种口径（切时长后标题变「日时长热力图」，tooltip 单位变 h，零额外请求）
  - **动态排名：**各年累计**用量 / 时长**（按钮可切）bar-racing 动画，支持按日/按月累积、播放/



暂停与调速、时间轴拖拽回放（切换主选项卡时自动停止播放）

。 **时段全景图**（v5.7.1 新增）：24 小时用电时段极坐标柱状图（ECharts），偶数小时显示刻度、柱高表示数值、透明度随数值渐变

- 顶部可切换 **用电量 / 时长 / 次数** 指标，并实时显示峰值时段；hover 柱体查看该小时用电量/时长/次数

- **点击柱体**：图表缩窄左移，右侧展开该小时「使用日期」面板（按月分组胶囊，点击日期胶囊可查看明细）

- **点击日期胶囊**：弹出该日使用明细气泡（径向模糊遮罩、三角箭头指向点击处）——顶部显示该日总时长/总用电/时段数，逐条列出开始→结束、时长与用电量；正在运行段（on\_time 非空且 off\_time 为空）按「当前时间 - on\_time」实时计算时长、按「now\_kwh - on\_power」计算用电，并以「● 运行中」高亮

- 再次点击同一日期胶囊收起气泡；切换指标/小时/收起详情时气泡自动关闭

- **月选项卡：**显示年度按钮 + 月度柱状图，无导航栏
- **日选项卡：**显示 ◀ 年份 ▶ ◀ 月份 ▶ 本月 导航条，支持快速切换年月
- **详情弹窗：**点击日历中有数据的日期，弹出该日的详细事件记录（设备开关机时段、时长、用量），弹窗宽度自动使用卡片配置的 width

#### 配置项汇总：

| 参数                        | 类型      | 默认值                  | 说明                   |
|---------------------------|---------|----------------------|----------------------|
| api.entity                | string  | —                    | API 数据源的设备实体 ID      |
| api.api_base_url          | string  | 顶层继承                 | ha_data_store API 地址 |
| api.key                   | string  | 顶层继承                 | API 密钥               |
| entities.data_entity      | string  | —                    | 实体属性数据源的实体 ID        |
| entities.consumption_attr | string  | gas_consumption      | 用量属性名                |
| entities.duration_attr    | string  | gas_duration_seconds | 时长属性名（秒）             |
| entities.date_attr        | string  | date                 | 日期属性名                |
| title                     | string  | 天然气使用量               | 卡片标题                 |
| show_title                | boolean | true                 | 是否显示标题               |
| width                     | string  | 100%                 | 卡片宽度                 |
| show_popup                | boolean | true                 | 点击日期是否弹出详情           |
| data_value_color          | color   | #F9D505              | 用量数据颜色               |
| calc_value_color          | color   | #804AFF              | 时长/费用颜色              |

| 参数                | 类型     | 默认值                | 说明                                                        |
|-------------------|--------|--------------------|-----------------------------------------------------------|
| chart_line_color  | color  | #FF6B6B            | 图表曲线颜色                                                    |
| date_font_size    | size   | 11px               | 日期字号                                                      |
| value_font_size   | size   | 10px               | 数值字号                                                      |
| units             | string | m <sup>3</sup> , 元 | 用量单位, 时长/费用单位                                             |
| decimals          | string | 2, 2               | 用量小数位, 时长/费用小数位                                           |
| series            | string | 用量, 费用             | 柱状图系列名, 曲线图系列名                                            |
| calc_value_choose | string | 智能推断               | calculate (费用) 或 value (时长)                               |
| calculate         | number | 1.0                | 换算系数<br>(calc_value_choose: calculate 时生效; 也可填实体 ID 实时取价) |

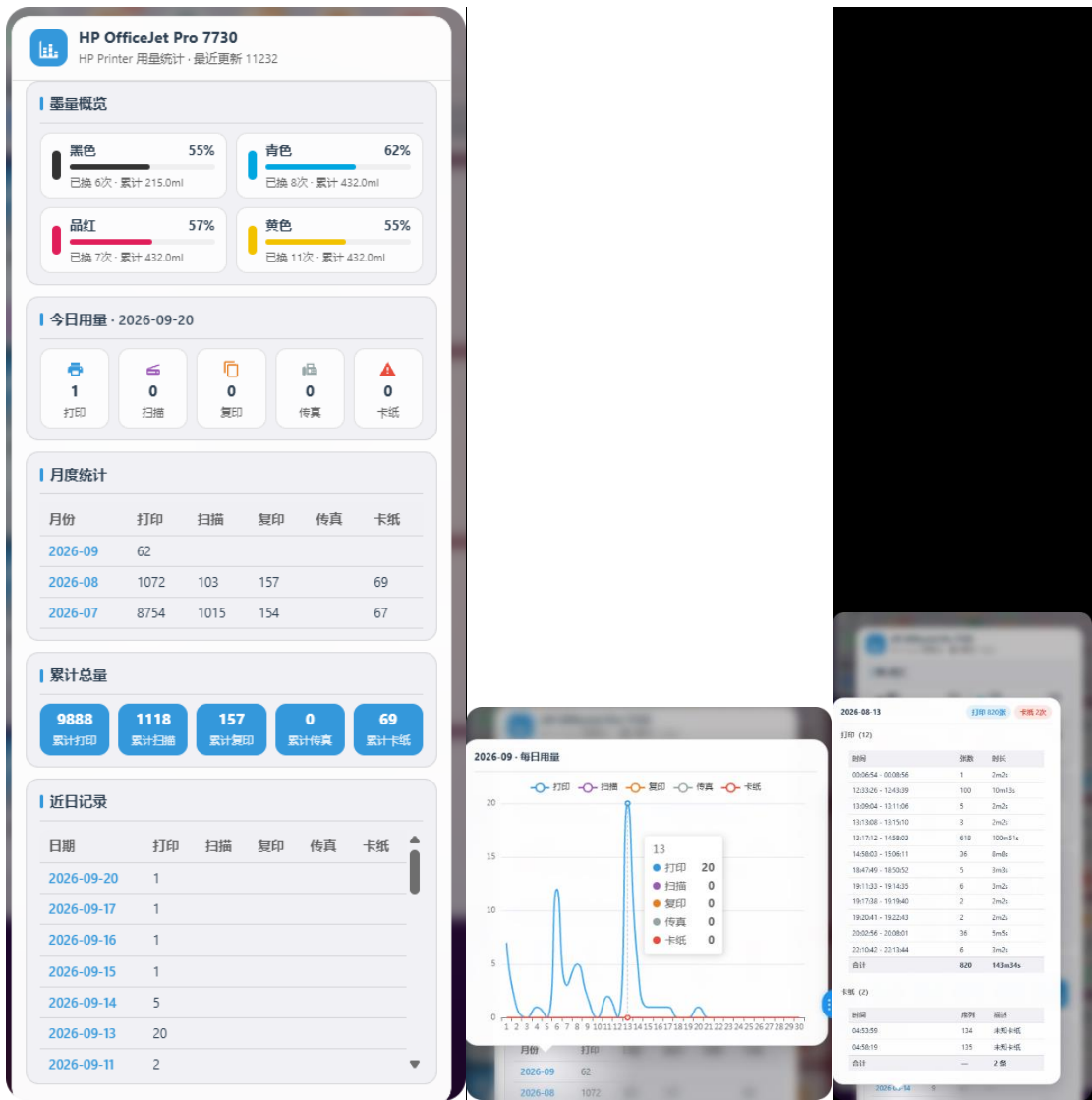
### 多实体与其它可选配置项:

| 参数                    | 类型      | 默认值                                                      | 说明                                                                |
|-----------------------|---------|----------------------------------------------------------|-------------------------------------------------------------------|
| api.entities          | array   | —                                                        | <b>多实体聚合:</b> 实体 ID 列表, 元素可为字符串或 { entity, name, color } (见「方式三」) |
| card_width            | string  | 100%                                                     | 卡片宽度 (与 width 同义, 两者取先生效的一个)                                      |
| card_background_color | color   | 主题卡片底色                                                   | 卡片背景色                                                             |
| calendarbox           | css     | 1px solid var(--uc-grid-line, rgba(128, 128, 128, 0.32)) | 日历格子边框 (默认跟随 --uc-grid-line 变量, 覆盖它即可统一改线色)                       |
| daybackground         | color   | rgba(104, 202, 150, 0.05)                                | 日历格子底色                                                            |
| running_animation     | boolean | true                                                     | 「运行中」格子的流光动画; false 关闭以降低 CPU 占用                                  |

| 参数             | 类型     | 默认值 | 说明                                  |
|----------------|--------|-----|-------------------------------------|
| monthly_budget | number | —   | 月度预算（单位同 units 第一项），配置后在日历下方显示预算进度条 |

(33) 打印机用量统计卡片 (type: printer)新增

打印机用量统计卡片用于展示打印机（如 HP）的墨量、日/月/年用量、累计统计。数据来源为打印机用量统计实体（sensor.hp\_printer\_yong\_liang\_tong\_ji）的 attributes。点击按钮弹出打印机用量统计面板，支持通过 popup\_card 或 card 动作打开。



```
- entity: input_boolean.da_yin_ji
  type: sensor
  name: 打印机
  icon_text: preset_state
  on_icon: mdi:printer
  off_icon: mdi:printer-off
  on_color: "#f58220"
```

```
off_color: "#74787c"
tap_action:
  action: popup_card # 或 action: card (走独立弹窗)
  popup_position: clone_button_top
  width: 400px # 外层弹窗宽度 (可选)
card:
  type: printer
  name: HP Printer 用量统计 # 可选, 弹窗标题, 默认同上
  entity: sensor.hp_printer_yong_liang_tong_ji # 可选, 默认此实体
```

- **依赖说明：**打印机卡片数据来源于打印机用量统计实体（`sensor.hp_printer_yong_liang_tong_ji`），需安装对应打印机状态统计集（集成在我的 GitHub 中，已经 HP 7720、7730、7740。其他机型未实测）或生成该实体。

(34) 爱快路由管理卡片 (type: ikuai)

爱快路由管理卡片用于展示爱快路由器运行状态，并支持 Docker 容器与虚拟机管理（支持虚拟机的画面传输）。数据来源于 ikuai\_router 集成（我的 GitHub 中下载对应集成） 生成的 sensor / switch / camera 实体。

卡片采用三选项卡结构（默认在"路由系统"），点击按钮弹出管理面板，支持通过 popup\_card 或 card 动作打开。

选项卡：

| 选项卡    | 内容                                            |
|--------|-----------------------------------------------|
| 路由系统   | 系统信息、硬件信息（折叠）、在线客户端、网络信息、应用流量（环形图）、路由器管理      |
| Docker | 容器列表，显示名称/状态/IP/CPU 占用，支持启停 + 日志查看            |
| 虚拟机    | 每台 VM 的画面（开机显示摄像头画面，关机显示占位）、核心/内存/CPU 占用、启停开关 |

```
- entity: input_boolean.ikuai
  type: sensor
  name: 爱快
  icon: mdi:router-network
  tap_action:
    action: popup_card
    width: 400px
    card:
      type: ikuai
      name: 爱快
      router:
        - sys_info: sensor.ikuai_overview
          name: 系统信息
        - hardware_info: sensor.ikuai_system_info
          name: 硬件信息
        - online_client: sensor.ikuai_online_client
          name: 在线客户端
        - network_info: sensor.ikuai_iface_check
          name: 网络信息
        - app_flow_1d: sensor.ikuai_app_flow_1d
          name: 1 天应用流量
        - app_flow_1h: sensor.ikuai_app_flow_1h
          name: 1 小时应用流量
        - app_flow_30min: sensor.ikuai_app_flow_30min
          name: 30 分钟应用流量
      network_control:
        - mac_address: text.ikuai_mac_input
```

```

- allow: button.ikuai_yun_xu_lian_wang
- reject: button.ikuai_jin_zhi_lian_wang
network_traffic:
- speed: 网速
  upload: sensor.ikuai_upload
  download: sensor.ikuai_download
- total: 合计流量
  upload: sensor.ikuai_totalup
  download: sensor.ikuai_totaldown
log:
- log: sensor.ikuai_audit_url_log
docker:
- name: NextTerminal
  entity: switch.ikuai_docker_nextterminal
- name: AdGuardHome
  entity: switch.ikuai_docker_adguardhome
vm:
- name: win7
  entity: switch.ikuai_vm_win7
  camera: camera.ikuai_vm_win7

```

路由系统

Docker

虚拟机

路由器管理

重启

重连WAN

系统信息

运行: 33天2小时22分钟

CPU: 0.2% · 62°C

内存: 9% · 14.5 GB

硬件信息

CPU: Intel(R) Xeon(R) CPU E3-1265L V2 @ 2.50GHz

内存: 14.9 GB

硬盘: ATA SSD 1TB 2A0 (953.87GB)

主板: /DQ77KB

在线客户端

46 个在线

| 名称                                  | 速度                       | 流量                     | 联网                                  |
|-------------------------------------|--------------------------|------------------------|-------------------------------------|
| HP_OfficeJet_Pro_773<br>192.168.1.7 | ↑ 47.0 B/s<br>↓ 0.00 B/s | ↑ 31.9 MB<br>↓ 70.2 MB | <input checked="" type="checkbox"/> |
| NUC<br>192.168.1.12                 | ↑ 930 B/s<br>↓ 965 B/s   | ↑ 2.39 GB<br>↓ 2.50 GB | <input checked="" type="checkbox"/> |
| 小米电压力锅<br>192.168.1.13              | ↑ 0.00 B/s<br>↓ 0.00 B/s | ↑ 9.24 MB<br>↓ 9.79 MB | <input checked="" type="checkbox"/> |
| HP_GEN8_iLO<br>192.168.1.16         | ↑ 0.00 B/s<br>↓ 0.00 B/s | ↑ 1008 B<br>↓ 1.96 KB  | <input checked="" type="checkbox"/> |
| Annie TV                            | ↑ 0.00 B/s<br>↑ 50.3 KB  |                        | <input checked="" type="checkbox"/> |

网络信息

wan2 正常

| 接口         | IP          | 连接 | 下载                  | 上传                   |
|------------|-------------|----|---------------------|----------------------|
| lan1       | 192.168.1.1 | —  | 2.78 KB/s<br>361 GB | 6.13 KB/s<br>67.7 GB |
| lan2       | —           | —  | 0.00 B/s<br>0.00 B  | 0.00 B/s<br>0.00 B   |
| doc_AdG... | 192.168.1.1 | —  | 0.00 B/s<br>333 MB  | 0.00 B/s<br>13.8 MB  |
| doc doc... | 192.168.1.1 | —  | 236 B/s             | 195 B/s              |

路由系统

Docker

虚拟机

Docker 容器

13 个容器

|              |     |                                        |                                     |
|--------------|-----|----------------------------------------|-------------------------------------|
| adguardhome  | 运行中 | <input checked="" type="checkbox"/> 日志 | <input checked="" type="checkbox"/> |
| aria2        | 已停止 |                                        | <input type="checkbox"/>            |
| clash        | 已停止 |                                        | <input type="checkbox"/>            |
| kms          | 已停止 |                                        | <input type="checkbox"/>            |
| kodexplorer  | 已停止 |                                        | <input type="checkbox"/>            |
| nextcloud    | 已停止 |                                        | <input type="checkbox"/>            |
| nps_47       | 运行中 | <input checked="" type="checkbox"/> 日志 | <input checked="" type="checkbox"/> |
| npoc_101     | 已停止 |                                        | <input type="checkbox"/>            |
| phpmyadmin   | 已停止 |                                        | <input type="checkbox"/>            |
| v2raya       | 已停止 |                                        | <input type="checkbox"/>            |
| alist        | 已停止 |                                        | <input type="checkbox"/>            |
| guacd        | 运行中 | <input checked="" type="checkbox"/> 日志 | <input checked="" type="checkbox"/> |
| nextterminal | 运行中 | <input checked="" type="checkbox"/> 日志 | <input checked="" type="checkbox"/> |

路由系统

Docker

虚拟机

win7

关机

已关机

4 核 · 8192 MB

CPU 占用 100%

centos\_7

关机

已关机

2 核 · 2048 MB

CPU 占用 100%

centos\_7\_gui

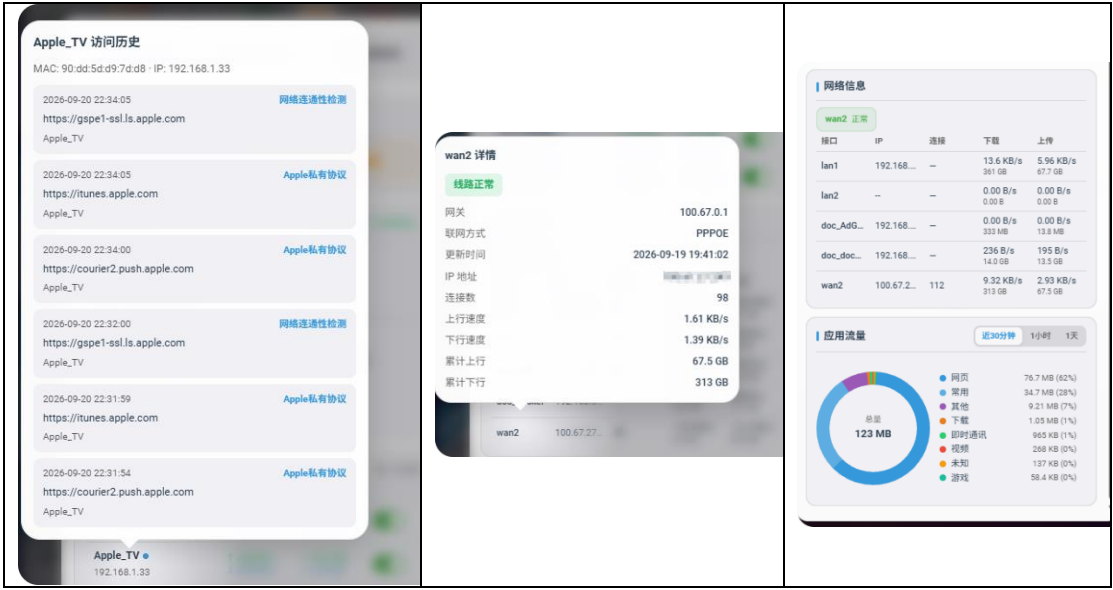
关机

已关机

2 核 · 4096 MB

CPU 占用 100%





配置项说明

| 配置项                    | 必填                                  | 类型     | 说明                                         |
|------------------------|-------------------------------------|--------|--------------------------------------------|
| type                   | <input checked="" type="checkbox"/> | string | ikuai                                      |
| name                   | <input type="checkbox"/>            | string | 卡片标题（默认 爱快）                                |
| width                  | <input type="checkbox"/>            | string | 弹窗宽度（默认 400px）                             |
| router                 | <input type="checkbox"/>            | array  | 路由系统区域数组，按数组顺序渲染                           |
| router[].sys_info      | <input type="checkbox"/>            | string | 系统信息实体<br>(sensor.ikuai_overview)          |
| router[].hardware_info | <input type="checkbox"/>            | string | 硬件信息实体<br>(sensor.ikuai_system_info, 折叠显示) |
| router[].online_client | <input type="checkbox"/>            | string | 在线客户端实体<br>(sensor.ikuai_online_client)    |
| router[].network_info  | <input type="checkbox"/>            | string | 网络信息实体<br>(sensor.ikuai_iface_check)       |

| 配置项                           | 必填 | 类型     | 说明                                           |
|-------------------------------|----|--------|----------------------------------------------|
| router[].app_flow_1d          | 否  | string | 1 天应用流量实体<br>(sensor.ikuai_app_flow_1d)      |
| router[].app_flow_1h          | 否  | string | 1 小时应用流量实体<br>(sensor.ikuai_app_flow_1h)     |
| router[].app_flow_30min       | 否  | string | 30 分钟应用流量实体<br>(sensor.ikuai_app_flow_30min) |
| network_control               | 否  | array  | 在线客户端联网控制配置                                  |
| network_control[].mac_address | 否  | string | MAC 地址写入实体<br>(text.ikuai_mac_input)         |
| network_control[].allow       | 否  | string | 允许联网按钮<br>(button.ikuai_yun_xu_lian_wan_g)   |
| network_control[].reject      | 否  | string | 禁止联网按钮<br>(button.ikuai_jin_zhi_lian_wan_g)  |
| network_traffic               | 否  | array  | 流量图表配置（点击系统信息标题栏速度区域弹出 ECharts 流量图表）         |
| network_traffic[].speed       | 否  | string | 网速分组（值可为标题，如 网速）                             |
| network_traffic[].upload      | 否  | string | 上行速度实体                                       |
| network_traffic[].download    | 否  | string | 下行速度实体                                       |
| network_traffic[].total       | 否  | string | 合计流量分组（值可为标题，如 合计流量）                         |
| log                           | 否  | array  | 客户端访问历史配置                                    |
| log[].log                     | 否  | string | 访问历史实体<br>(sensor.ikuai_audit_url_log)       |
| docker                        | 否  | array  | Docker 容器数组                                  |
| docker[].name                 | 否  | string | 容器显示名                                        |

| 配置项             | 必填 | 类型     | 说明                                  |
|-----------------|----|--------|-------------------------------------|
| docker[].entity | ☑  | string | 容器电源开关实体<br>(switch.ikuai_docker_*) |
| vm              | 否  | array  | 虚拟机数组                               |
| vm[].name       | 否  | string | 虚拟机显示名                              |
| vm[].entity     | ☑  | string | 虚拟机电源开关实体<br>(switch.ikuai_vm_*)    |
| vm[].camera     | 否  | string | VM 画面摄像头实体<br>(camera.ikuai_vm_*)   |

## 交互说明

- **系统信息：**运行时长 / CPU（占用+温度） / 内存（已用+总量）每行一条；标题栏右侧显示实时上/下行速度（配置 network\_traffic 后点击速度区域弹出流量图表）
- **硬件信息：**整合在系统信息内，默认折叠，点击展开显示 CPU / 内存 / 硬盘 / 主板
- **在线客户端：**名称+IP 小字、上行/下行速度与流量两行显示、按 IP 升序排列、刷新保持滚动位置；客户端名称中的 %20/% 自动替换为 \_
  - **联网开关：**根据 client.reject (0 允许 / 1 禁止) 显示开关状态（允许绿色、禁止红色）；点击开关先把 MAC 写入 mac\_address 实体，再根据当前状态按 allow/reject 按钮（用开关模拟点击按钮）

- **访问历史**：配置 log 后点击客户端名称/单元格弹出气泡，显示该客户端（按 MAC 过滤）的访问历史（时间/应用名/访问地址/备注）
- **网络信息**：线路检测状态 chips + 各接口 IP/连接数/上下行速度/累计流量；点击接口名弹出详情气泡（线路检测信息 + 流量信息）
- **流量图表**：配置 network\_traffic 后点击系统信息标题栏速度区域弹出气泡，标题"流量"，右侧"1 小时/12 小时/1 天"周期切换按钮，上方实时速度曲线 + 下方累计流量曲线（ECharts 渲染，气泡宽度为卡片宽度的 90%）
- **应用流量**：近 30 分钟/1 小时/1 天合并为一个区域，标题栏切换按钮 + SVG 环形图（中心总量 + 协议占比图例），单位智能换算 B→MB/GB/TB
- **路由器管理**：重启、重连 WAN，均弹出确认对话框
- **Docker / 虚拟机**：每行显示名称 + 状态徽标 + 启停开关；Docker 支持查看日志气泡
- **虚拟机画面**：开机显示摄像头实体画面（HA 系统自带 picture 卡片），关机显示"已关机"占位；信息栏含全屏按钮，点击弹出全屏 VM 画面弹窗

刷新机制

卡片只构建一次 DOM，周期刷新仅更新各数据容器内部内容（不重建结构），保留选项卡状态、VM 画面、折叠状态、滚动位置。默认每 10 秒局部刷新一次。

(35) 设备用电统计卡片 (type: device\_usage)

设备用电统计卡片用于展示指定房间（或全屋）当日各设备的使用时长、用电量和事件分布。通过 api\_base\_url 对接 HA 数据统一存储系统（ha\_data\_store）的设备事件 API。

两种显示模式：

| 模式       | 说明                                      |
|----------|-----------------------------------------|
| mini（默认） | 紧凑卡片，显示摘要 + 设备列表，适合嵌入头部模式网格             |
| full     | 完整弹窗，含 24h 甘特图 + 完整设备表 + 多种可视化视图 + 日期切换 |

```
buttons_2_columns: 1
buttons_2:
  - type: card
    row_column: 1,1-1
    card_config:
      type: device_usage
      show_device_replay: huifang
      layout: mini
      top_count: 4
      energy_center_hide_room: 儿童房外,次卧外,楼道,客厅外
      energy_center_hide: null
```







## 4. 通用动作系统 (tap\_action)

所有支持 tap\_action 的位置（按钮、title\_entities、entities 区域、传感器卡片等）共享同一套动作执行引擎。动作类型统一，配置方式一致，set\_value 和 toggle 还支持 entities 多实体批量操作。统一动作清单：

| action       | 说明                                  | 多实体 | 必需参数                      |
|--------------|-------------------------------------|-----|---------------------------|
| toggle       | 切换开关状态 (on↔off)，自动推断服务              | ✓   | entity 或 entities         |
| set_value    | 设置实体目标值，自动推断服务（最灵活）                 | ✓   | entity 或 entities + value |
| quick-action | 快捷操作，弹出情景模式气泡执行多步骤操作                | —   | scenes 或 actions          |
| more-info    | 弹出 HA 详细信息弹窗                        | ✗   | entity                    |
| navigate     | 页面导航                                | ✗   | navigation_path           |
| call-service | 调用 HA 服务                            | ✗   | service                   |
| popup_card   | 弹出自定义卡片弹窗                           | ✗   | card                      |
| card         | 弹出设备内置控制弹窗                          | ✗   | card（含 type 字段）           |
| toggle-all   | 切换分组中所有设备（仅聚合按钮）                    | —   | —                         |
| loop_assign  | 循环赋值：每次点击把实体推进到 values 序列下一个值，到末尾回绕 | ✓   | entity/loop + values      |
| none         | 不执行任何操作                             |     |                           |

## (1) toggle — 切换开关

自动根据实体域推断调用 turn\_on / turn\_off / press 等服务，无需手动指定。

```
# 单实体 toggle
tap_action:
  action: toggle

# 多实体批量 toggle
tap_action:
  action: toggle
  entities:
    - light.keting_dadeng
    - light.keting_xuanguan
    - light.keting_xuanguan2
```

自动推断逻辑：

| 实体域                    | on → 调用              | off → 调用              |
|------------------------|----------------------|-----------------------|
| light                  | light.turn_on        | light.turn_off        |
| switch / input_boolean | turn_on              | turn_off              |
| button / input_button  | press                | press                 |
| cover                  | cover.open_cover     | cover.close_cover     |
| fan                    | fan.turn_on          | fan.turn_off          |
| media_player           | media_player.turn_on | media_player.turn_off |
| vacuum                 | vacuum.start         | vacuum.return_to_base |
| 其他域                    | turn_on              | turn_off              |

## (2) set\_value — 设置目标值（最灵活）

根据实体域和 value 值自动推断应调用的 HA 服务，与快捷操作（情景模式）共享同一套服务推断引擎。无需手动指定服务名，只需提供目标值即可控制灯光、空调、遮盖、风扇等任意设备。

```
# ---- 灯光 ----
# 开灯
tap_action:
  action: set_value
  value: "on"

# 关灯
tap_action:
  action: set_value
  value: "off"

# 开灯 + 亮度/色温
tap_action:
  action: set_value
  value: "on"
  service_data:
    brightness: 200
    kelvin: 4000

# 多实体一键关灯
tap_action:
  action: set_value
  entities:
    - light.keting_dadeng
    - light.keting_xuanguan
  value: "off"

# ---- 空调 ----
# 设置温度（数值 → climate.set_temperature）
tap_action:
  action: set_value
  entity: climate.xiaomi_air_conditioner
  value: "26"

# 设置模式（模式字符串 → climate.set_hvac_mode）
tap_action:
  action: set_value
  entity: climate.xiaomi_air_conditioner
  value: cool

# 制冷 + 温度 + 风速（自动拆分为 3 次服务调用）
tap_action:
  action: set_value
  entity: climate.xiaomi_air_conditioner
  value: cool
  service_data:
    temperature: "26"      # → climate.set_temperature
    fan_mode: level6       # → climate.set_fan_mode（自动拆分）

# 关闭空调
tap_action:
  action: set_value
  entity: climate.xiaomi_air_conditioner
  value: "off"
```

```

# ---- 遮盖 ----
# 打开
tap_action:
  action: set_value
  value: "open"

# 关闭
tap_action:
  action: set_value
  value: "close"

# 设置位置百分比
tap_action:
  action: set_value
  value: "50"

# ---- 风扇 ----
# 开关
tap_action:
  action: set_value
  value: "on"          # 或 "off"

# 设置风速百分比
tap_action:
  action: set_value
  value: "60"

# ---- 选择器 ----
# select / input_select
tap_action:
  action: set_value
  value: "制冷"

# ---- 数值输入 ----
# input_number
tap_action:
  action: set_value
  value: "25"

# ---- 按钮 ----
# input_button / button (任意 value 均调用 press)
tap_action:
  action: set_value
  value: "press"

# ---- 自动化 ----
# 开关
tap_action:
  action: set_value
  value: "on"          # → automation.turn_on

# 触发
tap_action:
  action: set_value
  value: "trigger"      # → automation.trigger

# ---- 脚本 ----
tap_action:
  action: set_value
  entity: script.morning_routine
  value: "run"          # → script.morning_routine

# ---- 门锁 ----
tap_action:
  action: set_value
  value: "locked"       # → lock.lock

tap_action:

```

```

    action: set_value
    value: "unlocked"      # → lock.unlock

# ---- 扫地机 ----
# 开始清扫
tap_action:
    action: set_value
    entity: vacuum.roborock
    value: "on"           # → vacuum.start ("start" / "clean" 同理)

# 返回充电
tap_action:
    action: set_value
    entity: vacuum.roborock
    value: "off"          # → vacuum.return_to_base ("return" / "home" /
"dock" 同理)

# 停止清扫
tap_action:
    action: set_value
    entity: vacuum.roborock
    value: "stop"         # → vacuum.stop

# 暂停清扫
tap_action:
    action: set_value
    entity: vacuum.roborock
    value: "pause"        # → vacuum.pause

# ---- 透传模式（强制指定服务） ----
tap_action:
    action: set_value
    entity: climate.xiaomi_air_conditioner
    value: cool
    service_data:
        service: climate.set_hvac_mode    # 强制使用此服务，跳过自动推断
        # 其他字段原样传递

```

## set\_value 支持的实体类型与服务自动推断对照表：

| 实体域                       | value 值         | 调用服务               | 说明                                                       |
|---------------------------|-----------------|--------------------|----------------------------------------------------------|
| light                     | "on" / "true"   | light.turn_on      | 开灯，service_data 可传 brightness、kelvin、color_temp、effect 等 |
| light                     | "off" / "false" | light.turn_off     | 关灯，自动清理 kelvin/brightness 等参数                            |
| light                     | 其他值             | light.turn_on      | 作为亮度等参数                                                  |
| switch /<br>input_boolean | "on" / "off"    | turn_on / turn_off | 开关切换                                                     |

| 实体域          | value 值                                  | 调用服务                            | 说明                                                 |
|--------------|------------------------------------------|---------------------------------|----------------------------------------------------|
| climate      | "off"                                    | climate.set_hvac_mode           | 关闭空调 (hvac_mode: "off"), 自动清理 temperature/fan_mode |
| climate      | "on" / "true"                            | climate.turn_on                 | 开启空调                                               |
| climate      | 数值<br>(如 "26")                           | climate.set_temperature         | 设置温度, temperature 取数值                              |
| climate      | 模式字符串<br>+ service_data<br>含 temperature | climate.set_temperature         | 同时设置 hvac_mode + temperature                       |
| climate      | 模式字符串<br>(无 temperature)                 | climate.set_hvac_mode           | 仅设置模式                                              |
| climate      | service_data<br>含 fan_mode               | 额外调用 climate.set_fan_mode       | fan_mode 自动拆分为独立服务调用; 关闭模式 (value: "off") 时跳过风速设置  |
| cover        | "open"                                   | cover.open_cover                | 打开遮盖                                               |
| cover        | "closed" / "close"                       | cover.close_cover               | 关闭遮盖                                               |
| cover        | 数值                                       | cover.set_cover_position        | 设置位置百分比                                            |
| fan          | "on" / "off"                             | fan.turn_on / fan.turn_off      | 风扇开关                                               |
| fan          | 数值                                       | fan.set_percentage              | 设置风速百分比                                            |
| media_player | "on" / "off"                             | media_player.turn_on / turn_off | 媒体设备开关; 验证时 standby 也视为关闭成功                        |

| 实体域                          | value 值                                    | 调用服务                              | 说明             |
|------------------------------|--------------------------------------------|-----------------------------------|----------------|
| media_p<br>layer             | "play" / "pause"<br>"                      | media_play / medi<br>a_pause      | 播放/暂停          |
| button /<br>input_b<br>utton | 任意                                         | press                             | 按压按钮           |
| input_s<br>elect             | 任意                                         | input_select.sel<br>ect_option    | option = value |
| select                       | 任意                                         | select.select_op<br>tion          | option = value |
| input_n<br>umber             | 数值                                         | input_number.set<br>_value        | 设置数值           |
| automat<br>ion               | "on" / "off"                               | automation.turn_<br>on / turn_off | 自动化开关          |
| automat<br>ion               | 其他                                         | automation.trigg<br>er            | 触发自动化          |
| vacuum                       | "on" / "start"<br>/ "clean"                | vacuum.start                      | 开始清扫           |
| vacuum                       | "off" / "return<br>" / "home" / "do<br>ck" | vacuum.return_to<br>_base         | 返回充电           |
| vacuum                       | "stop"                                     | vacuum.stop                       | 停止清扫           |
| vacuum                       | "pause"                                    | vacuum.pause                      | 暂停清扫           |
| script                       | 任意                                         | script.{实体名}                      | 执行脚本           |
| lock                         | "locked" / "loc<br>k"                      | lock.lock                         | 上锁             |

| 实体域  | value 值      | 调用服务                  | 说明   |
|------|--------------|-----------------------|------|
| lock | 其他           | lock.unlock           | 解锁   |
| 其他域  | "on" / "off" | turn_on / turn_off    | 通用开关 |
| 其他域  | 其他           | homeassistant.turn_on | 通用回退 |

此对照表与快捷操作（情景模式）的推断逻辑完全一致，  
因为底层共享同一个\_inferSceneService 引擎。



### (3) quick-action — 快捷操作

在任意支持 tap\_action 的位置触发情景模式快捷操作，弹出气泡选择器并执行多步骤操作。桥接情景模式执行引擎，支持执行进度展示、状态验证、延时执行等全部功能。

**完整模式 (scenes 数组):** 配置多个情景，点击弹出气泡选择器

```
tap_action:
  action: quick-action
  name: 情景模式                                # 气泡标题
  scenes:                                       # 情景列表（格式与 scene_mode 一致）
    - name: 回家模式
      confirm: true
      actions:
        - entity: light.living_room
          value: "on"
        - entity: climate.ac
          service_data:
            temperature: 26
    - name: 离家模式
      actions:
        - entities:
            客厅灯: light.keting_dadeng
            卧室灯: light.bedroom
          value: "off"
  scene_mode: bubble                            # 可选: bubble | direct
```

**简写模式 (actions 数组):** 直接配置操作列表，自动包装为单情景直接执行

```
tap_action:
  action: quick-action
  name: 一键回家
  actions:                                       # 简写模式，自动包装为 [{ name: "一键回家", actions }]
    - entity: light.living_room
      value: "on"
    - entity: climate.ac
      service_data:
        temperature: 26
  confirm: true                                # 可选: 执行前确认
```

## 配置项说明：

| 配置项            | 必填 | 类型      | 说明                                  |
|----------------|----|---------|-------------------------------------|
| action         | ✓  | string  | 固定为 quick-action                    |
| scenes         | ✗* | array   | 情景列表，格式与 scene_mode（5.18 节）完全一致     |
| actions        | ✗* | array   | 操作列表（简写模式，与 scenes 二选一）             |
| name           | ✗  | string  | 操作名称，用于气泡标题，默认 快捷操作                 |
| scene_mode     | ✗  | string  | bubble（气泡选择）/ direct（直接执行），不配置时自动判断 |
| confirm        | ✗  | boolean | 简写模式下执行前是否确认，默认 false               |
| icon           | ✗  | string  | 图标，默认 mdi:palette                   |
| fold           | ✗  | boolean | 动作列表是否默认展开，默认 false                 |
| verify_timeout | ✗  | number  | 验证超时秒数，默认 10                        |

\*scenes 和 actions 至少配置一个。scenes 优先级更高，同时配置时使用 scenes。

#### (4) more-info — 详细信息弹窗

弹出 HA 标准的实体详细信息弹窗。

```
tap_action:  
  action: more-info
```

#### (5) navigate — 页面导航

跳转到 HA 的另一个页面。

```
tap_action:  
  action: navigate  
  navigation_path: /lovelace/bedroom    # 目标页面路径
```

## (6) call-service — 调用 HA 服务

调用任意 HA 服务，支持卡片本地服务路由。

```
# 标准 HA 服务
tap_action:
  action: call-service
  service: light.turn_on
  service_data:
    entity_id: light.living
    brightness: 255

# 卡片本地服务（自由布局弹窗等）
tap_action:
  action: call-service
  service: modern_room_card.show_free_layout_popup
  service_data:
    title: 控制面板
    cards: [...]
```

## (7) popup\_card — 弹出自定义卡片

弹出一个包含自定义内容的弹窗。

```
tap_action:
  action: popup_card
  card:
    - type: sensor
      entity: sensor.temperature
      name: 温度
    - type: switch
      entity: switch.plug
  popup_position: top      # 可选: top / bottom / left / right / center
  width: 350              # 可选: 弹窗宽度 (像素)
```

## (8) card — 弹出设备内置控制弹窗

弹出卡片内置的设备控制弹窗（如窗帘面板、灯光面板、空调面板等），配置方式与 buttons 数组中的按钮配置一致。与 popup\_card 的区别：popup\_card 弹出的是 HA 原生卡片（如 weather-forecast、entities 等），card 弹出的是卡片自身内置的设备专属控制面板（带拖拽滑块、动画等交互）。

支持的设备类型：

| type          | 说明       | 弹窗效果               |
|---------------|----------|--------------------|
| lights        | 灯光组控制面板  | 亮度/色温调节、按房间分组、批量开关 |
| light         | 单灯控制面板   | 精简灯光弹窗             |
| socket        | 插座控制面板   | 批量开关、功率桑基图         |
| ac            | 空调控制面板   | 温度/模式/风速调节         |
| curtain       | 窗帘控制面板   | 百分比滑块、单双层切换、拖拽控制   |
| clothes_dryer | 晾衣架控制面板  | 机械仿真拖拽、收藏位置        |
| consumables   | 耗材管理面板   | 电池/滤芯剩余量           |
| device        | 通用设备状态面板 | 设备开关状态             |
| media         | 媒体控制面板   | 播放/暂停/音量           |
| phone         | 话费信息面板   | 余额/流量/通话           |
| scene_mode    | 情景模式气泡   | 气泡选择/执行进度          |

配置示例：

```

# 弹出全屋窗帘聚合弹窗
tap_action:
  action: card
  card:
    type: curtain
    name: 全屋窗帘
    show_open: false
    on_icon: mdi:curtain-open
    off_icon: mdi:curtain
    card:
      - name: 客厅窗帘
        height: 400px
        fabric_entity: cover.ke_ting_bu_lian
        sheer_entity: cover.ke_ting_sha_lian
        layer_mode: double
      - name: 卧室窗帘
        height: 400px
        fabric_entity: cover.wo_shi_bu_lian
        layer_mode: single

# 弹出灯光控制面板
tap_action:
  action: card
  card:
    type: lights
    name: 客厅灯光
    per_line: 2
    card:
      - entity: light.living_ceiling
        name: 天花灯
      - entity: light.living_floor
        name: 落地灯

# 弹出空调控制面板
tap_action:
  action: card
  card:
    type: ac
    card:
      - entity: climate.living
        name: 客厅空调
      - entity: climate.bedroom
        name: 卧室空调

# 弹出情景模式气泡
tap_action:
  action: card
  card:
    type: scene_mode
    scenes:
      - name: 回家
        actions:
          - entity: light.living
            value: "on"
      - name: 离家
        actions:
          - entity: light.living
            value: "off"

```

## 弹窗参数：

popup\_position 和 width 可配置在 tap\_action 层级，对所

有设备类型生效（card 层级未配置时降级使用）：

```
tap_action:
  action: card
  popup_position: center      # 可选：弹窗位置
  width: 600px               # 可选：弹窗宽度
  card:
    type: lights
    card:
      - entity: light.living_room
        name: 客厅灯
```

card 与 popup\_card 的区别：

| 对比        | popup_card                       | card                           |
|-----------|----------------------------------|--------------------------------|
| 弹窗内容      | HA 原生卡片（第三方/内置卡片）                | 卡片内置的设备专属控制面板                  |
| card 字段含义 | HA 卡片配置（{ type: entities, ... }） | 设备类型配置（{ type: curtain, ... }） |
| 交互能力      | 取决于嵌入的卡片                         | 带拖拽滑块、动画等丰富交互                  |
| 适用场景      | 嵌入任意 HA 卡片（天气、地图等）               | 弹出灯光/空调/窗帘等设备控制面板              |

## (9) entities 多实体批量操作

toggle 和 set\_value 支持 entities 数组，一次性对多个实体执行相同操作：

```
# 多实体 toggle: 一键切换所有灯
tap_action:
  action: toggle
  entities:
    - light.living_ceiling
    - light.living_led
    - light.living_spot

# 多实体 set_value: 一键关闭所有灯
tap_action:
  action: set_value
  entities:
    - light.living_ceiling
    - light.living_led
    - light.living_spot
  value: "off"
```

entity（单实体）和 entities（多实体）二选一，同时配置时 entities 优先。

通用动作系统在以下位置均可用：

| 位置             | 配置字段                        | 说明                   |
|----------------|-----------------------------|----------------------|
| 按钮的 tap_action | buttons[].tap_action        | 概览按钮点击               |
| 独立按钮           | buttons[].tap_action        | 无 type 的独立按钮         |
| 弹窗内设备卡片        | card[].tap_action           | 灯光/插座/耗材弹窗内的设备       |
| title_entities | title_entities[].tap_action | 弹窗标题右侧的实体/按钮         |
| entities 区域    | entities[].tap_action       | 卡片左侧传感器区域            |
| 传感器卡片          | tap_action                  | 自由布局弹窗内的 sensor 类型卡片 |
| 设备状态卡片         | tap_action                  | 设备状态弹窗内的卡片           |



(10) 自动弹窗 (auto\_open\_entity)

当 tap\_action 配置了 call-service 且服务为 show\_free\_layout\_popup 时，可以通过 auto\_open\_entity 参数实现自动弹出弹窗——当指定实体变为 on 时，自动打开该按钮的弹窗，无需用户手动点击。

```
buttons:
- name: 晾衣架
  entity: switch.giot_cn_l126348886_v64ksm_on_p_4_1
  on_icon: mdi:hanger
  off_icon: mdi:hanger
  on_color: "#f58220"
  off_color: "#74787c"
  tap_action:
    action: call-service
    service: modern_room_card.show_free_layout_popup
    popup_position: clone_button_top
    auto_open_entity: input_boolean.ce_shi    # 当此实体为 on 时自动弹出
    service_data:
      title: 晾衣架
      width: 360px
      cards:
        - row: 1
          per_line: 1
          items:
            - type: clothes_dryer
              entity: cover.clothes_dryer
              name: 晾衣架
```

行为规则：

| 场景                            | 行为                                    |
|-------------------------------|---------------------------------------|
| 页面加载时 auto_open_entity 已为 on  | ✅ 自动弹出弹窗                              |
| 运行中 auto_open_entity 从 off→on | ✅ 自动弹出弹窗                              |
| 运行中 auto_open_entity 从 on→off | ❌ 不关闭弹窗（默认），配置 auto_close: true 可自动关闭 |
| 用户手动关闭弹窗后，                    | ❌ 本轮 on 周期内不再自动弹出                     |

| 场景                               | 行为                  |
|----------------------------------|---------------------|
| auto_open_entity 仍为 on           |                     |
| auto_open_entity 变为 off 后再次变为 on | ✅ 新一轮 on 周期，可再次自动弹出 |

### 可选参数：

| 参数               | 类型      | 默认值   | 说明                                                                        |
|------------------|---------|-------|---------------------------------------------------------------------------|
| auto_open_entity | string  | 无     | （必填）触发自动弹窗的实体 ID，状态为 on 时弹出                                               |
| auto_open_delay  | number  | 0     | 延迟弹出时间（毫秒）。entity 变为 on 后等待指定时间再弹出，避免短暂触发时立即弹出。延迟期间如果 entity 变回 off 则取消弹出 |
| auto_close       | boolean | false | entity 变为 off 时是否自动关闭弹窗。默认 false（不关闭），设为 true 则 entity 变 off 时自动关闭        |

### auto\_open\_delay 延迟弹出示例：

```
tap_action:
  action: call-service
  service: modern_room_card.show_free_layout_popup
  auto_open_entity: binary_sensor.door_open
  auto_open_delay: 2000 # 门打开后等 2 秒再弹出（避免短暂开门误触发）
  popup_position: clone_button_top
  service_data:
    title: 门口监控
    cards: [...]
    name: 晾衣架
```

### auto\_close 自动关闭示例：

```
tap_action:
```

```
action: call-service
service: modern_room_card.show_free_layout_popup
auto_open_entity: input_boolean.show_panel
auto_close: true # 开关关闭时弹窗自动关闭
popup_position: clone_button_top
service_data:
  title: 控制面板
  cards: [...]
```

### 使用场景举例：

- 用 input\_boolean 作为"展示模式"开关，开启后自动弹出设备详情
- 用某个传感器的自动化联动开关，当设备异常时自动弹出告警弹窗
- 用定时器实体，在特定时间段自动弹出控制面板
- 用 auto\_open\_delay 避免传感器短暂触发时误弹窗（如人体传感器经过而非停留）
- 用 auto\_close 配合临时开关，实现"展示后自动收回"的效果

### 注意事项：

- auto\_open\_entity 仅在 action: call-service + show\_free\_layout\_popup 服务下生效
- auto\_open\_entity、auto\_open\_delay、auto\_close 均放在 tap\_action 层级（与 action、service\_data 同级），而非 service\_data 内部
- 弹窗位置由 popup\_position 控制，与手动点击时一致（如 clone\_button\_top 会定位在按钮上方）

- 多个按钮可以配置不同的 `auto_open_entity`，各自独立触发
- `auto_open_entity` 实体类型建议使用 `input_boolean`、`switch`、`binary_sensor` 等二态实体

## (11) 循环赋值 (loop\_assign)

每次点击把实体推进到 values 序列中的下一个值，到达末尾后回绕到第一个。典型场景是下拉型实体 (input\_select / select) 逐级循环切换，一个按钮即可按预设顺序轮换选项。

**定位方式：**读取实体当前状态，在 values 中找索引并赋下一个值；若当前值不在序列中，则从序列第一个开始。赋值底层复用情景模式的服务推断引擎（支持 select\_option / set\_value 等，详见上方 set\_value 对照表）。

**防抖：**卡片会过滤连续点击，两次点击间隔小于 interval（默认 500 ms）的点击被忽略，确保每次点击都能读到 HA 更新后的最新状态，避免快速连点跳值/漏值。

```
# 单实体：每次点击依次切换 input_select. 模式 为 自动 → 睡眠 → 强风 → 自动...
tap_action:
  action: loop_assign
  entity: input_select. 模式
  values: ['自动', '睡眠', '强风']
  interval: 800          # 可选，连续点击最小间隔(ms)，默认 500

# 多实体：各自独立轮换 (loop 数组)
tap_action:
  action: loop_assign
  loop:
    - entity: input_select. 模式
      values: ['自动', '睡眠', '强风']
    - entity: input_select. 风速
      values: ['低', '中', '高']
  interval: 1000
```

| 参数     | 必填   | 类型     | 说明        |
|--------|------|--------|-----------|
| entity | 单实体时 | string | 要轮换的实体 ID |

| 参数           | 必填                                                                                | 类型       | 说明                                                                           |
|--------------|-----------------------------------------------------------------------------------|----------|------------------------------------------------------------------------------|
|              | 必填                                                                                |          | (与 loop 二选一)                                                                 |
| values       |  | string[] | 轮换的目标值序列，按点击顺序依次赋值，末尾回绕                                                      |
| loop         | 多实体时<br>必填                                                                        | array    | 多实体数组，每项含 entity / values / 可选 interval / service_data (与 entity+values 二选一) |
| interval     |  | number   | 连续点击最小间隔 (ms)，默认 500                                                         |
| service_data |  | object   | 透传给服务调用的附加参数 (如 _service 强制指定服务)                                             |

loop\_assign 的 entity / values 可直接改用  
service\_data 透传 \_service 强制指定赋值服务；单实体简写  
entity+values 与多实体 loop 数组均可。

## 5. 弹窗系统

Room Elves Card 的弹窗系统是所有交互控制的核心载体——点击按钮弹出控制面板、空调调节面板、灯光面板等，都是通过统一的弹窗引擎实现的。本节详细说明弹窗系统的架构、配置和行为。

### (4) 弹窗架构概览

所有弹窗通过统一的 `showPopup(options)` 方法创建，该方法返回一个 `closePopup()` 函数供外部调用关闭弹窗。

#### 弹窗打开流程：

- ① 解析 options 参数
- ② 通过 `_zIndexManager` 分配动态 Z-Index（支持多层弹窗嵌套）
- ③ 非嵌套弹窗时，先移除同类型的已有弹窗和遮罩
- ④ 创建背景遮罩（`popup-overlay`）
- ⑤ 创建弹窗主体 DOM，调用 `content()` 函数生成内容
- ⑥ 收集弹窗内所有定时器，关闭时统一清理
- ⑦ 根据触发按钮位置计算弹窗位置，选择动画方向
- ⑧ 添加 `.visible` 类触发动画

#### 弹窗关闭流程：

① 清理所有局部定时器、ECharts 实例、

IntersectionObserver

② 释放 Z-Index

③ 执行关闭动画后移除 DOM

④ 移除遮罩、克隆按钮、光晕层

⑤ 调用 onClose 回调

**点击遮罩关闭：** 默认行为，点击弹窗外的半透明遮罩区域即可关闭弹窗。





(5) 弹窗类型一览

每种按钮类型都有对应的专属弹窗，点击按钮后自动触发：

| 按钮类型            | 弹窗方法                      | CSS 类名                | 功能说明                     |
|-----------------|---------------------------|-----------------------|--------------------------|
| lights          | showLightControlPopup()   | light-control-popup   | 灯光控制面板（亮度/色温/颜色/分组/批量操作） |
| light           | showLightCardPopup()      | light-card-popup      | 单灯控制面板                   |
| ac              | showAcControlPopup()      | ac-control-popup      | 空调控制面板（温度/模式/风速）         |
| socket          | showSocketControlPopup()  | socket-control-popup  | 插座控制面板（批量开关）             |
| consumables     | showConsumablesPopup()    | consumables-popup     | 耗材/电池面板                  |
| media           | showMediaControlPopup()   | media-control-popup   | 媒体控制面板                   |
| curtain         | showCurtainControlPopup() | curtain-control-popup | 窗帘控制面板（百分比/双层/开合模式）      |
| clothes_dryer   | showClothesDryerPopup()   | —                     | 晾衣架控制面板（拖拽/收藏位置）         |
| fan             | —                         | —                     | 风扇控制面板（风速/摇头）            |
| phone           | showPhonePopup()          | phone-control-popup   | 话费面板                     |
| device / heater | showDevicePopup()         | device-popup          | 通用设备/暖气控制面板              |

| 按钮类型                       | 弹窗方法                            | CSS 类名                 | 功能说明            |
|----------------------------|---------------------------------|------------------------|-----------------|
| scene_mode                 | showSceneModeBubble()<br>)      | —                      | 情景模式气泡选择面板      |
| popup_card<br>(tap_action) | handlePopupCardAction()<br>)    | custom-card-popup      | 自定义 HA 卡片弹窗     |
| call-service<br>(特定服务)     | showFreeLayoutPopup()<br>)      | free-layout-popup      | 自由布局控制面板        |
| 人在传感器                      | showPersonPopup()<br>)          | person-popup           | 人员活动弹窗（时间轴/户型图） |
| 自动化                        | showAutomationPopup()<br>)      | automation-popup       | 自动化管理面板         |
| 概览栏                        | showOverviewControlPopup()<br>) | overview-control-popup | 概览控制弹窗          |

showDevicePopup() 是路由分发器，根据按钮的 type 字段分派到各专用弹窗方法。

### 弹窗位置系统 (popup\_position)

通过 popup\_position 配置项控制弹窗相对于触发按钮的弹出位置。

### 支持的位置值

| 位置值               | 别名          | 效果     |
|-------------------|-------------|--------|
| center            | —           | 视口居中   |
| clone_button_top  | up、top      | 触发按钮上方 |
| clone_button_down | down、bottom | 触发按钮下方 |

| 位置值                      | 别名    | 效果       |
|--------------------------|-------|----------|
| clone_button_left        | left  | 触发按钮左侧   |
| clone_button_right       | right | 触发按钮右侧   |
| clone_button_upper_left  | —     | 触发按钮左上方  |
| clone_button_upper_right | —     | 触发按钮右上方  |
| clone_button_lower_left  | —     | 触发按钮左下方  |
| clone_button_lower_right | —     | 触发按钮右下方  |
| 不配置（默认）                  | —     | 自动选择最佳位置 |

## 偏移值语法

支持在位置值后附加自定义偏移量，格式为 "位置，偏移量"：

popup\_position: "clone\_button\_down, 30px"      # 按钮下方，偏移 30px

popup\_position: "center, -10px"      # 居中，向上偏移 10px

默认偏移量为 25px。

## 自动定位算法（不配置 popup\_position 时）

当未指定位置时，系统自动选择最佳弹出方向，优先级为：右 → 左 → 空间更大的一侧 → 上下。算法会计算按钮四周的可用空间，选择能完整容纳弹窗的方向。

## 边界保护

- **非居中位置：** 水平+垂直方向均 clamp 到安全区域，确保弹窗不超出视口
- **居中位置：** 仅垂直边界保护，水平方向严格居中
- **人在传感器弹窗：** 自适应高度，超出时限制 maxHeight 并启用滚动

(6) 弹窗动画

弹窗支持多种出现/消失动画，系统会根据弹窗位置自动选择最合适的动画方向，不支持参数配置。

| 动画类                     | 效果                    | 时长    | 触发条件              |
|-------------------------|-----------------------|-------|-------------------|
| popup-fade-in           | 淡入 + 缩放<br>(0.95→1.0) | 0.2s  | 默认动画              |
| popup-center-fade-in    | 居中淡入 + 缩放             | 0.2s  | 居中弹出时             |
| popup-slide-from-right  | 从右侧滑入                 | 0.25s | 按钮在弹窗左侧           |
| popup-slide-from-left   | 从左侧滑入                 | 0.25s | 按钮在弹窗右侧           |
| popup-slide-from-bottom | 从下方滑入                 | 0.25s | 按钮在弹窗上方           |
| popup-slide-from-top    | 从上方滑入                 | 0.25s | 按钮在弹窗下方           |
| heater-popup-anim       | 弹性缩放动画                | 0.4s  | heater 类型弹窗<br>专用 |

**动画方向智能选择：** 系统根据克隆按钮与弹窗的相对位置自动选择滑入方向——按钮在左则从右滑入，按钮在上则从下滑入，以此类推。

(7) 弹窗宽度 (width)

多种按钮类型支持通过 width 配置项控制弹窗宽度：

```
buttons:
  - type: curtain
    entity: cover.living_curtain
    width: 560px # 弹窗宽度
```

宽度处理逻辑：

| 配置值                | 效果           |
|--------------------|--------------|
| 不配置                | 使用各弹窗类型的默认宽度 |
| 具体值（如 400px、560px） | 固定为该宽度       |
| auto               | 不限制宽度，由内容自适应 |
| 百分比（如 80%）         | 基于视口宽度计算     |

移动端会自动限制弹窗高度不超过视口的 60%，确保操作便利性。

(8) 自定义卡片弹窗 (popup\_card)

通过 tap\_action: popup\_card 可以在弹窗中嵌入任意 HA 卡片，实现完全自定义的控制面板。

```
buttons:
- entity: switch.some_switch
  name: 自定义面板
  tap_action:
    action: popup_card
    card: # 嵌入的 HA 卡片配置
      type: entities
      entities:
        - switch.some_switch
        - sensor.some_sensor
    popup_position: center # 弹窗位置
    width: 500px # 弹窗宽度
```

配置项

| 配置项            | 必填 | 类型     | 默认值  | 说明                                     |
|----------------|----|--------|------|----------------------------------------|
| card           | ✔  | object | 无    | 要嵌入的 HA 卡片配置对象（标准 HA 卡片格式）             |
| popup_position | ✘  | string | null | 弹窗位置                                   |
| width          | ✘  | string | null | 弹窗宽度，支持具体值（500px）、百分比（80%）、auto（内容自适应） |

width 详细处理：

| 配置值    | 效果                               |
|--------|----------------------------------|
| 不配置    | min-width: 380px, max-width: 95% |
| "auto" | 不限制宽度，由内容决定                      |



| 配置值     | 效果         |
|---------|------------|
| "500px" | 固定宽度 500px |
| "80%"   | 视口宽度的 80%  |

**vertical-stack 类型特殊处理：** 当嵌入的卡片为 vertical-stack 类型时，自动创建折叠/展开功能，可收起/展开内含的子卡片。

## (9) 自由布局弹窗 (show\_free\_layout\_popup)

自由布局弹窗是 Room Elves Card 中最灵活、最常用的弹窗形式。它允许你在弹窗中自由排列各种控制卡片（开关、传感器、选择器、图表等），支持选项卡分组、行分组、跨行跨列、条件过滤等高级功能，可以快速搭建出复杂的设备控制面板。

### ① 触发方式

自由布局弹窗通过 tap\_action: call-service 触发，服务名 room\_card.show\_free\_layout\_popup 或 modern\_room\_card.show\_free\_layout\_popup（两者等价）。

### 顶层配置项

|                                           |                                |
|-------------------------------------------|--------------------------------|
| tap_action:                               |                                |
| action: call-service                      |                                |
| service: room_card.show_free_layout_popup |                                |
| popup_position: center                    | # 弹窗位置（可放在 tap_action 层，优先级更高） |
| auto_open_entity: binary_sensor.xxx       | # 自动弹窗实体（放在 tap_action 层）      |
| auto_open_delay: 2000                     | # 延迟弹出毫秒（放在 tap_action 层）      |
| auto_close: true                          | # 自动关闭（放在 tap_action 层）        |
| service_data:                             |                                |
| title: 控制面板                               | # 弹窗标题                         |
| show_title: true                          | # 是否显示标题栏                      |
| width: 360px                              | # 弹窗宽度                         |
| cards: [...]                              | # 卡片列表（与 tabs 互斥）              |
| # tabs: [...]                             | # 选项卡列表（与 cards 互斥）            |
| layout: grid                              | # 布局方式（仅 cards 传统模式）           |
| grid_config:                              | # 网格配置                         |
| columns: 2                                |                                |
| gap: 10                                   |                                |
| title_entities: [...]                     | # 标题右侧实体                       |
| auto_redirect: false                      | # 自动跳转到设备开启的选项卡                |
| display_only:                             | # 仅显示实体值匹配的选项卡                 |
| - 'on'                                    |                                |
| table_per_line: null                      | # 每行选项卡数                       |
| table_one_width: null                     | # 选项卡固定宽度                      |
| update_interval: 0                        | # 更新间隔（秒）                      |
| style: ''                                 | # 自定义 CSS                      |
| popup_position: center                    | # 弹窗位置（也可放 service_data 内）     |

完整配置项说明：

| 配置项             | 必填 | 类型      | 默认值                 | 说明                                                  |
|-----------------|----|---------|---------------------|-----------------------------------------------------|
| title           | ✗  | string  | 控制面板                | 弹窗标题文本                                              |
| show_title      | ✗  | boolean | true                | 是否显示标题栏，设为 false 时隐藏整个标题区域                          |
| width           | ✗  | string  | auto                | 弹窗宽度，支持 auto（自适应，最小 300px）、具体值（360px）、百分比（80%）      |
| cards           | ✗  | array   | []                  | 卡片列表（非选项卡模式），与 tabs 互斥                              |
| tabs            | ✗  | array   | null                | 选项卡列表（选项卡模式），与 cards 互斥                             |
| layout          | ✗  | string  | grid                | 布局方式：grid（网格）/ flex（弹性）/ custom（自定义，仅 cards 传统模式生效） |
| grid_config     | ✗  | object  | {columns:2, gap:10} | 网格配置，含 columns（列数）和 gap（间距 px）                      |
| title_entities  | ✗  | array   | []                  | 标题右侧实体配置（见 26.6 节）                                  |
| auto_redirect   | ✗  | boolean | false               | 自动跳转到设备开启的选项卡（见 26.7 节）                             |
| display_only    | ✗  | array   | null                | 仅显示实体值匹配的选项卡（见 26.8 节）                              |
| table_per_line  | ✗  | number  | null                | 每行显示的选项卡数量                                          |
| table_one_width | ✗  | string  | null                | 每个选项卡的固定最小宽度（如 80px）                                |
| update_interval | ✗  | number  | 0                   | 定时刷新间隔（秒），0 = 跟随 HA 实时推送                            |
| style           | ✗  | string  | ''                  | 追加到弹窗的自定义 CSS 字符串                                   |
| popup_position  | ✗  | string  | null                | 弹窗位置 tap_action 层优先级高于 service_data 内               |

注意：cards 和 tabs 不能同时配置，否则报错。

## 两种内容模式

### Cards 模式（非选项卡）

Cards 模式有两种结构，系统自动检测。

#### ② 结构一：Row 分组模式（推荐，支持行标题、行内跨列）

当 cards 数组中第一项包含 row 和 items 字段时，自动进入 Row 分组模式：

```
cards:
- row: 1                                # 行号，决定排列顺序
  per_line: 2                           # 每行显示几个卡片
  title: 灯光控制                       # 行标题（可选）
  show_title: true                     # 是否显示标题（默认 true）
  title_entities: [...]                # 行级标题右侧实体（可选）
  items:                               # 该行的卡片列表
    - type: switch
      entity: switch.light1
      name: 客厅灯
    - type: switch
      entity: switch.light2
      name: 卧室灯
- row: 2
  per_line: 3
  title: 传感器
  items:
    - type: sensor
      entity: sensor.temperature
      name: 温度
    - type: sensor
      entity: sensor.humidity
      name: 湿度
    - type: sensor
      entity: sensor.pm25
      name: PM2.5
```

#### ③ 结构二：传统布局模式（简单网格排列）

直接列出卡片，使用 layout 和 grid\_config 控制布局：

```
layout: grid
grid_config:
```

```

columns: 2
gap: 10
cards:
  - type: sensor
    entity: sensor.temperature
    name: 温度
  - type: switch
    entity: switch.plug
    name: 插座
  - type: select
    entity: select.mode
    name: 模式

```

#### ④ Tabs 模式（选项卡分组）

使用选项卡将内容分为多个面板，每个选项卡有独立的行内容：

```

tabs:
  - name: 客厅
    icon: mdi:sofa
    entity: switch.living_light      # 绑定实体（用于高亮/跳转）
    entity_value: 'on'              # 实体匹配值
    rows:
      - row: 1
        per_line: 2
        items:
          - type: switch
            entity: switch.living_light
            name: 主灯
          - type: switch
            entity: switch.living_floor_lamp
            name: 落地灯
  - name: 卧室
    icon: mdi:bed
    entity: switch.bedroom_light
    rows:
      - row: 1
        per_line: 2
        items:
          - type: switch
            entity: switch.bedroom_light
            name: 卧室灯

```

#### ⑤ 弹窗内支持的卡片类型

自由布局弹窗内置了 30+ 种卡片类型，可直接在 items 中使用：

| type 值             | 说明      | 主要配置项                                              |
|--------------------|---------|----------------------------------------------------|
| sensor             | 传感器数值   | entity, name, unit, icon, layout (mini/full)       |
| switch             | 开关控制    | entity, name, icon                                 |
| select             | 下拉选择器   | entity, name                                       |
| number             | 数值输入    | entity, name, min, max, step                       |
| text               | 文本输入    | entity, name                                       |
| button             | 按压按钮    | entity, name, icon, show_duration, confirm         |
| light              | 灯光控制    | entity, name (含亮度/色温调节)                            |
| cover              | 遮盖控制    | entity, name                                       |
| curtain            | 窗帘控制    | entity, fabric_entity, sheer_entity                |
| ac                 | 空调控制    | entity                                             |
| media              | 媒体控制    | entity                                             |
| clothes_dryer      | 晾衣架控制   | entity, light_entity                               |
| radio              | 单选按钮组   | entity, options                                    |
| button_group       | 按钮组     | buttons, direction, on_color, off_color, show_name |
| action             | 快捷操作/情景 | name, icon, scenes, scene_mode                     |
| chart / chart_line | 折线图     | entity, name                                       |
| chart_gau          | 仪表盘图    | entity, name                                       |
| chart_progress     | 进度条     | entity, name                                       |

| type 值            | 说明       | 主要配置项                                    |
|-------------------|----------|------------------------------------------|
| chart_bar         | 柱状图      | entity, name                             |
| chart_pie         | 环形图      | entity, name                             |
| chart_pie_full    | 完整饼图     | entity, name                             |
| chart_mixed       | 混合图表     | entity, name                             |
| chart_nightingale | 南丁格尔玫瑰图  | entity, name                             |
| chart_heatmap     | 日历热力图    | entity, name                             |
| chart_calendar    | 日历图      | entity, name                             |
| timeline          | 时间轴      | entity, map_table, api_base_url          |
| picture           | 图片展示     | bg_image, halo_entity, bg_image_position |
| html              | 自定义 HTML | content                                  |
| user              | 用户信息     | persons                                  |
| usage             | 使用量统计    | entity, name                             |
| card              | 自定义卡片    | 标准 HA 卡片配置                               |
| conditional_tabs  | 条件选项卡    | 条件控制 Tab 显示                              |

### 每个卡片通用字段：

| 字段              | 类型     | 默认值 | 说明                                 |
|-----------------|--------|-----|------------------------------------|
| update_interval | number | 0   | 单独的刷新间隔（秒），覆盖弹窗级别的 update_interval |

| 字段         | 类型      | 默认值    | 说明                                  |
|------------|---------|--------|-------------------------------------|
| layout     | string  | normal | 卡片布局: normal (标准) / mini / min (紧凑) |
| show_name  | boolean | true   | 是否显示名称                              |
| row_column | string  | 无      | 跨行跨列配置 (见 26.5 节)                   |
| position   | object  | 无      | 传统模式网格位置 {row, column}              |
| size       | object  | 无      | 传统模式跨列跨行 {width, height}            |

## ⑥ 跨行跨列布局 (row\_column)

在 Row 分组模式的 items 中, 可以通过 row\_column 让卡片占据多个网格单元, 格式为 '序号, 行数-列数':

```
items:
- type: sensor
  entity: sensor.temperature
  name: 温度
  row_column: '1,1-2'           # 序号 1, 占 1 行 2 列 (跨 2 列宽)
- type: switch
  entity: switch.light1
  name: 灯 1
  row_column: '2,1-1'           # 序号 2, 占 1 行 1 列
- type: switch
  entity: switch.light2
  name: 灯 2
  row_column: '3,1-1'           # 序号 3, 占 1 行 1 列
- type: chart_line
  entity: sensor.power
  name: 功率曲线
  row_column: '4,2-2'           # 序号 4, 占 2 行 2 列 (大图表)
```

规则:

- 只要 items 中有任意一个配置了 row\_column, 该行组就切换为 CSS Grid 跨行跨列模式



- 没有 row\_column 的 item 按 1×1 自动排列
- per\_line 作为列数参考
- row\_column 只影响同一个 row 组内的布局

常见布局示例：

| row_column | 效果             | 适用场景     |
|------------|----------------|----------|
| '1,1-2'    | 1 行 2 列（跨 2 列） | 宽传感器、滑块  |
| '1,2-2'    | 2 行 2 列（方形）    | 大图表      |
| '1,1-1'    | 1×1 标准卡片       | 普通       |
| '1,2-3'    | 2 行 3 列        | 大型图表/控制区 |

## ⑦ 标题右侧实体（title\_entities）

在弹窗标题栏右侧或行标题右侧显示实体状态/快捷操作，支持三种模式：

### 模式一：传感器模式

显示实体当前值或 Way 计算结果：

```
title_entities:
- entity: switch.dian_fan_guo
  icon: mdi:stove
  name: 今日使用次数
  show_name: false
  way: today_usage_count      # Way 计算方式
  format: '今天用了{_default}次' # Way 格式模板
```

支持 way、format、unit 等 Way 计算引擎字段。

### 模式二：控制按钮模式

显示可点击的控制按钮，支持多实体批量操作：

```
title_entities:
- entities:
  - light.keting_dadeng
  - light.keting_xuanguan
name: 灯光
icon: mdi:lightbulb-group
tap_action:
  action: toggle # 点击切换所有灯的开关
```

### 模式三：快捷操作模式

显示情景模式快捷按钮：

```
title_entities:
- type: action
  name: 情景
  icon: mdi:palette
  scenes:
  - name: 全关
    actions:
    - entities:
      客厅灯: light.keting_dadeng
      value: "off"
  scene_mode: bubble # bubble | direct
```

title\_entities 每项字段：

| 字段        | 必填 | 类型           | 说明               |
|-----------|----|--------------|------------------|
| entity    | ✖* | string       | 单实体 ID           |
| entities  | ✖* | array        | 多实体 ID 数组（批量操作）  |
| name      | ✖  | string       | 显示名称             |
| icon      | ✖  | string       | 图标               |
| show_name | ✖  | boolean      | 是否显示名称（默认 true）  |
| way       | ✖  | string/array | Way 计算方式（详见第十六节） |
| format    | ✖  | string       | Way 显示格式模板       |

| 字段         | 必填 | 类型     | 说明                      |
|------------|----|--------|-------------------------|
| unit       | ✗  | string | 单位                      |
| tap_action | ✗  | object | 点击动作（详见第六-A 节）          |
| type       | ✗  | string | 设为 'action' 启用快捷操作模式    |
| scenes     | ✗  | array  | 快捷操作情景列表（type=action 时） |

entity 和 entities 至少配置一个（type=action 除外）。

### 自动跳转（auto\_redirect）

当选项卡中绑定了 entity 和 entity\_value 时，开启 auto\_redirect 可自动跳转到最近开启的设备选项卡。

|                                                                                                                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> auto_redirect: true tabs:   - name: 客厅     entity: switch.living_light     entity_value: 'on'     rows: [...]   - name: 卧室     entity: switch.bedroom_light     entity_value: 'on'     rows: [...] </pre> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### 跳转规则：

| 场景                           | 行为                                                        |
|------------------------------|-----------------------------------------------------------|
| auto_redirect: false<br>(默认) | 始终激活第一个非禁用且可见的 Tab                                        |
| auto_redirect: true          | 在所有 entity+entity_value 匹配的 Tab 中，选择 last_changed 时间最新的跳转 |
| 无匹配                          | 降级为第一个非禁用且可见的 Tab                                         |

| 场景                    | 行为      |
|-----------------------|---------|
| 禁用的 Tab               | 不参与自动跳转 |
| 被 displayOnly 隐藏的 Tab | 不参与自动跳转 |

## 条件过滤 (display\_only)

仅显示实体状态匹配指定值的选项卡，未匹配的选项卡自动隐藏。

```
display_only:
  - 'on'
  - Delay
  - Keep Warm
tabs:
  - name: 设备 A
    entity: switch.device_a      # 绑定实体
    rows: [...]
  - name: 设备 B
    entity: select.device_b      # 如果 state 为 "Delay" 则显示
    rows: [...]
```

## 过滤规则：

| 条件                            | 结果        |
|-------------------------------|-----------|
| display_only 未配置或为空           | 所有 Tab 可见 |
| Tab 未配置 entity                | 始终可见      |
| 实体不存在或状态为 unavailable/unknown | 始终可见      |
| 实体状态匹配 display_only 中任一项      | 可见        |
| 实体状态不匹配                       | 隐藏        |

**实时更新：** 当实体状态变化时，系统自动更新 Tab 可见性。如果当前激活的 Tab 被隐藏，自动切换到第一个可见

Tab。

空状态提示： 当所有 Tab 都不可见时，弹窗显示提示信息： "N 个设备中没有现在处于开启的设备"。

选项卡高级配置

选项卡完整字段

| 字段           | 必填 | 类型            | 默认值   | 说明                              |
|--------------|----|---------------|-------|---------------------------------|
| name         | ✗  | string        | Tab N | 选项卡显示名称                         |
| icon         | ✗  | string        | 无     | 选项卡图标                           |
| belong       | ✗  | string        | 无     | 所属分组（用于按楼层/区域分组显示）              |
| entity       | ✗  | string        | 无     | 绑定实体 ID（用于高亮/自动跳转/display_only） |
| entity_value | ✗  | string/array  | 无     | 实体匹配值，支持数组（任一匹配即算）              |
| rows         | ✗  | array         | []    | 选项卡内容行（与 items 格式相同）            |
| disabled     | ✗  | boolean       | false | 是否禁用（灰色不可点击）                    |
| badge        | ✗  | string/number | 无     | 角标固定值                           |
| badge_entity | ✗  | string        | 无     | 角标动态取值实体（实体值作为角标数字）             |
| config_id    | ✗  | string        | 无     | 注册 ID（供 tabs_config 引用）         |

## 选项卡分组 (belong)

当任何 Tab 配置了 belong 时，选项卡按 belong 值分组显示，每组有标签 + 按钮容器。无 belong 的 Tab 平铺到导航栏最前面。

```
tabs:
- name: 客厅灯
  belong: 1楼 # 属于"1楼"分组
  rows: [...]
- name: 客厅空调
  belong: 1楼
  rows: [...]
- name: 主卧灯
  belong: 2楼 # 属于"2楼"分组
  rows: [...]
```

## 选项卡高亮

当 Tab 配置了 entity+entity\_value 且实体状态匹配时，选项卡按钮自动高亮显示。

## 选项卡宽度控制

```
table_per_line: 3 # 每行显示 3 个选项卡按钮
table_one_width: 80px # 每个选项卡最小宽度 80px
```

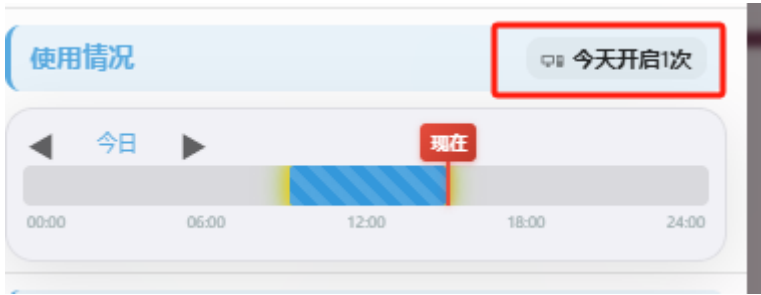
两者可同时使用，table\_per\_line 控制换行，table\_one\_width 控制按钮宽度。

## 行分组高级功能

### 行级标题右侧实体

每行可以独立配置 title\_entities，格式与弹窗顶层的

title\_entities 相同:



```
cards:
- row: 1
  title: 灯光控制
  title_entities:
  - entity: switch.all_lights
    icon: mdi:lightbulb-group
    tap_action:
      action: toggle
  items: [...]
```

行级详情链接 (show\_more\_info)

在行标题右侧显示详情链接，点击弹出更多信息:



```
cards:
- row: 1
  title: 空调
  show_more_info: popup_card # 显示"详情>>>"链接
  card: # 点击后弹出的自定义卡片
    type: entities
    entities:
    - climate.living
  items: [...]
```

|                     |                                     |
|---------------------|-------------------------------------|
| show_more_info<br>值 | 效果                                  |
| popup_card          | 在标题右侧显示"详情>>>"链接，点击弹出 card 配置的自定义卡片 |
| 不配置                 | 不显示详情链接                             |



## 完整配置示例

### 示例一：简单面板



```
buttons:
  tap_action:
    config_id: chuju
    action: call-service
    service: modern_room_card.show_free_layout_popup
    service_data:
      title: 厨具
      width: 430px
      popup_position: center
      tabs:
```

```

- name: 电饭锅
  config_id: dian_fan_guo
  entity: sensor.chunmi_cn_404489380_ehl_status_p_2_1
  entity_value:
    - Delay
    - Keep Warm
    - Busy
  rows:
    - row: 1
      per_line: 1
      title: 电饭锅
      show_title: false
      items:
        - type: picture
          show_button_background: false
          halo_entity: input_boolean.chu_fang_dian_fan_guo
          bg_image: /local/house/电饭锅.png
          bg_image_position:
            - x: 0
            - "y": 0
            - zoom: 0.8
            - rotate: 0
    - row: 2
      per_line: 1
      title: 电饭锅运行时刻
      title_entities:
        - type: sensor
          entity: input_boolean.chu_fang_dian_fan_guo
          icon: mdi:stove
          name: 今日使用次数
          show_name: false
          way: today_usage_count
          format: 今天用了{_default}次
      items:
        - entity: sensor.chunmi_cn_404489380_ehl_status_p_2_1
          type: timeline
          name: 电饭锅运行情况
          api_base_url: /api/gateway/take1943/clock_device/api
          api_device_name: 厨房-电饭锅
          icon: mdi:stove
          map_table:
            "on":
              Busy:
                text: 烹饪中
                icon: mdi:stove
                color: "#ff5959"
              Delay:
                text: 预约中
                icon: mdi:stove
                color: "#085f63"
              Keep Warm:
                text: 保温中
                icon: mdi:stove
                color: "#facf5a"
            "off":
              idle:
                text: 空闲
                icon: mdi:stove
                color: "#8b968d"
            "off":
              text: 关闭
              icon: mdi:stove
              color: "#8b968d"
    - row: 3
      per_line: 3
      title: 状态
      items:
        - type: sensor
          entity: sensor.chunmi_cn_404489380_ehl_status_p_2_1

```

```

        name: 状态
        on_card_color: true
        show_duration: false
        icon: mdi:information-variant-circle
        map_table:
            "on":
                Busy:
                    text: 烹饪中
                    icon: mdi:stove
                    color: "#ff5959"
                Delay:
                    text: 预约中
                    icon: mdi:stove
                    color: "#085f63"
                Keep Warm:
                    text: 保温中
                    icon: mdi:stove
                    color: "#facf5a"
            "off":
                idle:
                    text: 空闲
                    icon: mdi:stove
                    color: "#8b968d"
                "off":
                    text: 关闭
                    icon: mdi:stove
                    color: "#8b968d"
        - entity: sensor.chufang_dianfanguo_gonglv
          type: sensor
          name: 功率
          show_duration: false
          icon: mdi:lightning-bolt
        - type: switch
          entity:
            button.chunmi_cn_404489380_ehl_cancel_cooking_a_2_1
          name: 停止
          confirm: input_boolean.confirm
          show_ios-switch: false
          show_duration: false
          icon: mdi:stop-circle-outline
        - row: 4
          per_line: 3
          title: 用电中心
          show_more_info: popup_card
          card:
            type: custom:usage-card
            title: 电饭锅用电量日历
            show_title: false
            width: 400px
            card_background_color: rgba(0, 0, 0, 0.2)
            calendarbox: "1px solid #888084"
            daybackground: rgba(236, 240, 241, 0.05)
            host: /api/gateway/take1943
            theme: input_select.theme
            dark_light_theme: dark,light
            device_name: dianfanguo
            entity: input_boolean.chu_fang_dian_fan_guo
            calc_value_choose: value
            value_font_size: 10px
            date_font_size: 10px
            data_value_color: "#00a381"
            calc_value_color: "#c85179"
            units: °,h
            decimals: 2,2
            series: 用量,时长
          items:
            - type: sensor
              entity: sensor.chufang_dianfanguo_day
              name: 日用电

```

```

        layout: mini
        icon: mdi:calendar-today
      - type: sensor
        entity: sensor.chufang_dianfanguo_month
        name: 月用电
        layout: mini
        icon: mdi:calendar-range
      - type: sensor
        entity: sensor.chufang_dianfanguo_year
        name: 年用电
        layout: mini
        icon: mdi:calendar-month
    - row: 6
      per_line: 1
      title: 智能插座
      items:
        - type: switch
          entity: switch.chufang_dianfanguo
          confirm: input_boolean.confirm
          show_duration: input_boolean.show_duration
          name: 插座开关
          icon: mdi:power-socket-uk
          info_entity: sensor.chufang_dianfanguo_gonglv
          info_popup:
            title: 电饭锅插座
            width: 400px
            cards:
              - row: 1
                per_line: 3
                title: 信息中心
                items:
                  - entity: sensor.chufang_dianfanguo_tongzhi
                    type: sensor
                    layout: mini
                    name: 通知
                    icon: mdi:information-variant-circle
                  - entity: sensor.chufang_dianfanguo_wendu
                    type: sensor
                    layout: mini
                    name: 温度
                    icon: mdi:thermometer
                  - entity: sensor.chufang_dianfanguo_gonglv
                    type: sensor
                    name: 功率
                    layout: mini
                    icon: mdi:lightning-bolt
              - row: 2
                per_line: 3
                title: 用电中心
                items:
                  - entity: sensor.chufang_dianfanguo_day
                    type: sensor
                    name: 日用电
                    layout: mini
                    icon: mdi:calendar-today
                  - entity: sensor.chufang_dianfanguo_month
                    type: sensor
                    name: 月用电
                    icon: mdi:calendar-range
                    layout: mini
                  - entity: sensor.chufang_dianfanguo_year
                    type: sensor
                    layout: mini
                    name: 年用电
                    icon: mdi:calendar-month
              - row: 3
                title: 控制中心
                per_line: 2
                items:

```

```

- entity:
light.cuco_cn_943423267_v3_s_13_indicator_light
  type: switch
  name: 指示灯
  icon: mdi:alarm-light
- entity: switch.chufang_dianfanguo_lock
  type: switch
  name: 童锁
  icon: mdi:lock
- row: 4
  per_line: 1
  title: 实时功率
  items:
    - type: chart_line
      entity: sensor.chufang_dianfanguo_gonglv
      height: 120px
      icon: mdi:lightning-bolt
      name: 实时功率
      update_interval: 3
      line:
        min: 0
        max: 2500
        hours_to_show: 24
- name: 空气炸锅
  config_id: kong_qi_zhaguo
  entity: switch.kongqizhaguo
  entity_value: "on"
  rows:
.....

```

## 示例二：动态图标+选项卡 + 自动跳转 + 条件过滤

```

buttons:
- type: dynamic_icon
  name: 厨具
  show_badge: true
  icon: mdi:countertop
  color: "#74787c"
  loop_display: true
  display_time: 3
  rules:
    - condition:
        entity: sensor.chunmi_cn_404489380_eh1_status_p_2_1
        operator: "=="
        value:
          - Delay
          - Keep Warm
          - Busy
        icon: mdi:stove
        color: "#f58220"
        animation: breathe
        name: 电饭锅
    - condition:
        entity: switch.kongqizhaguo
        operator: "=="
        value: "on"
        icon: mdi:pot-steam
        color: "#f58220"
        animation: breathe
        name: 空气炸锅
    - condition:

```

```

        entity: switch.weibolu
        operator: "=="
        value: "on"
    icon: mdi:microwave
    color: "#f58220"
    animation: breathe
    name: 微波炉
- condition:
    entity: switch.youyanji
    operator: "=="
    value: "on"
    icon: mdi:gas-burner
    color: "#f58220"
    animation: breathe
    name: 油烟机&燃气灶
- condition:
    entity: switch.reshuiqi
    operator: "=="
    value: "on"
    icon: mdi:water-boiler
    color: "#f58220"
    animation: breathe
    name: 热水器
- condition:
    entity: input_boolean.shao_shui_hu
    operator: "=="
    value: "on"
    icon: mdi:kettle-steam
    color: "#f58220"
    animation: breathe
    name: 烧水壶
tap_action:
    config_id: chuju
    action: call-service
    service: modern_room_card.show_free_layout_popup
    service_data:
        title: 厨具
        width: 430px
        popup_position: center
        display_only:
            - "on"
            - Delay
            - Keep Warm
            - Busy
            - "off"
            - Idle
    auto_redirect: true
    tabs:
        - name: 电饭锅
          config_id: dian_fan_guo
          entity: sensor.chunmi_cn_404489380_ehl_status_p_2_1
          entity_value:
            - Delay
            - Keep Warm
            - Busy
          rows:
            - row: 1
              per_line: 1
              title: 电饭锅
              show_title: false
              items:
                - type: picture
                  show_button_background: false
                  halo_entity: input_boolean.chu_fang_dian_fan_guo
                  bg_image: /local/house/电饭锅.png
                  bg_image_position:
                    - x: 0
                    - "y": 0
                    - zoom: 0.8

```

```

- rotate: 0
- row: 2
  per_line: 1
  title: 电饭锅运行时刻
  title_entities:
    - type: sensor
      entity: input_boolean.chu_fang_dian_fan_guo
      icon: mdi:stove
      name: 今日使用次数
      show_name: false
      way: today_usage_count
      format: 今天用了[_default]次
  items:
    - entity: sensor.chunmi_cn_404489380_ehl_status_p_2_1
      type: timeline
      name: 电饭锅运行情况
      api_base_url: /api/gateway/take1943/clock_device/api
      api_device_name: 厨房-电饭锅
      icon: mdi:stove
      map_table:
        "on":
          Busy:
            text: 烹饪中
            icon: mdi:stove
            color: "#ff5959"
          Delay:
            text: 预约中
            icon: mdi:stove
            color: "#085f63"
          Keep Warm:
            text: 保温中
            icon: mdi:stove
            color: "#facf5a"
        "off":
          idle:
            text: 空闲
            icon: mdi:stove
            color: "#8b968d"
        "off":
          text: 关闭
          icon: mdi:stove
          color: "#8b968d"
- row: 3
  per_line: 3
  title: 状态
  items:
    - type: sensor
      entity: sensor.chunmi_cn_404489380_ehl_status_p_2_1
      name: 状态
      on_card_color: true
      show_duration: false
      icon: mdi:information-variant-circle
      map_table:
        "on":
          Busy:
            text: 烹饪中
            icon: mdi:stove
            color: "#ff5959"
          Delay:
            text: 预约中
            icon: mdi:stove
            color: "#085f63"
          Keep Warm:
            text: 保温中
            icon: mdi:stove
            color: "#facf5a"
        "off":
          idle:
            text: 空闲

```

```

        icon: mdi:stove
        color: "#8b968d"
    "off":
        text: 关闭
        icon: mdi:stove
        color: "#8b968d"
    - entity: sensor.chufang_dianfanguo_gonglv
      type: sensor
      name: 功率
      show_duration: false
      icon: mdi:lightning-bolt
    - type: switch
      entity:
button.chunmi_cn_404489380_ehl_cancel_cooking_a_2_1
      name: 停止
      confirm: input_boolean.confirm
      show_ios-switch: false
      show_duration: false
      icon: mdi:stop-circle-outline
- row: 4
  per_line: 3
  title: 用电中心
  show_more_info: popup_card
  card:
    type: custom:usage-card
    title: 电饭锅用电量日历
    show_title: false
    width: 400px
    card_background_color: rgba(0, 0, 0, 0.2)
    calendarbox: "1px solid #888084"
    daybackground: rgba(236, 240, 241, 0.05)
    host: /api/gateway/take1943
    theme: input_select.theme
    dark_light_theme: dark,light
    device_name: dianfanguo
    entity: input_boolean.chu_fang_dian_fan_guo
    calc_value_choose: value
    value_font_size: 10px
    date_font_size: 10px
    data_value_color: "#00a381"
    calc_value_color: "#c85179"
    units: °,h
    decimals: 2,2
    series: 用量,时长
  items:
    - type: sensor
      entity: sensor.chufang_dianfanguo_day
      name: 日用电
      layout: mini
      icon: mdi:calendar-today
    - type: sensor
      entity: sensor.chufang_dianfanguo_month
      name: 月用电
      layout: mini
      icon: mdi:calendar-range
    - type: sensor
      entity: sensor.chufang_dianfanguo_year
      name: 年用电
      layout: mini
      icon: mdi:calendar-month
- row: 6
  per_line: 1
  title: 智能插座
  items:
    - type: switch
      entity: switch.chufang_dianfanguo
      confirm: input_boolean.confirm
      show_duration: input_boolean.show_duration
      name: 插座开关

```



```

icon: mdi:power-socket-uk
info_entity: sensor.chufang_dianfanguo_gonglv
info_popup:
  title: 电饭锅插座
  width: 400px
cards:
  - row: 1
    per_line: 3
    title: 信息中心
    items:
      - entity: sensor.chufang_dianfanguo_tongzhi
        type: sensor
        layout: mini
        name: 通知
        icon: mdi:information-variant-circle
      - entity: sensor.chufang_dianfanguo_wendu
        type: sensor
        layout: mini
        name: 温度
        icon: mdi:thermometer
      - entity: sensor.chufang_dianfanguo_gonglv
        type: sensor
        name: 功率
        layout: mini
        icon: mdi:lightning-bolt
  - row: 2
    per_line: 3
    title: 用电中心
    items:
      - entity: sensor.chufang_dianfanguo_day
        type: sensor
        name: 日用电
        layout: mini
        icon: mdi:calendar-today
      - entity: sensor.chufang_dianfanguo_month
        type: sensor
        name: 月用电
        icon: mdi:calendar-range
        layout: mini
      - entity: sensor.chufang_dianfanguo_year
        type: sensor
        name: 年用电
        icon: mdi:calendar-month
  - row: 3
    title: 控制中心
    per_line: 2
    items:
      - entity:
light.cuco_cn_943423267_v3_s_l3_indicator_light
        type: switch
        name: 指示灯
        icon: mdi:alarm-light
      - entity: switch.chufang_dianfanguo_lock
        type: switch
        name: 童锁
        icon: mdi:lock
  - row: 4
    per_line: 1
    title: 实时功率
    items:
      - type: chart_line
        entity: sensor.chufang_dianfanguo_gonglv
        height: 120px
        icon: mdi:lightning-bolt
        name: 实时功率
        update_interval: 3
        line:
          min: 0

```

```

max: 2500
hours_to_show: 24
- name: 空气炸锅
  config_id: kong_qi_zhaguo
  entity: switch.kongqizhaguo
  entity_value: "on"
  rows:
.....

```

### 示例三：跨行跨列 + 传感器 + 标题实体

```

buttons:
- entity: switch.main
  name: 智能面板
  tap_action:
    action: call-service
    service: room_card.show_free_layout_popup
    service_data:
      title: 客厅
      width: auto
      title_entities:
        - entity: sensor.living_temperature
          name: 温度
          icon: mdi:thermometer
        - entities:
            - light.living_ceiling
            - light.living_floor_lamp
          name: 灯光
          icon: mdi:lightbulb-group
          tap_action:
            action: toggle
cards:
- row: 1
  per_line: 3
  title: 环境
  items:
    - type: sensor
      entity: sensor.living_temperature
      name: 温度
      unit: °C
      row_column: '1,1-1'
    - type: sensor
      entity: sensor.living_humidity
      name: 湿度
      unit: '%'
      row_column: '2,1-1'
    - type: sensor
      entity: sensor.living_pm25
      name: PM2.5
      row_column: '3,1-1'
- row: 2
  per_line: 2
  title: 控制
  items:
    - type: switch
      entity: switch.living_light
      name: 主灯
      row_column: '1,1-2'
    - type: select
      entity: select.ac_mode
      name: 空调模式

```

```
        row_column: '2,1-1'
      - type: number
        entity: number.ac_temp
        name: 温度
        min: 16
        max: 30
        step: 1
        row_column: '3,1-1'
    - row: 3
      per_line: 1
      title: 功率曲线
      items:
        - type: chart_line
          entity: sensor.living_power
          name: 功率
          row_column: '1,1-1'
```

## 6. 样式

样式为卡片级参数 (style)，该配置项可以设置该卡片内的元素的样式。主要有两种方法：第一种卡片整体样式，第二种是按选择器覆盖内部元素（高级）。

### 第一种：卡片整体样式

```
style:
  width: 350px           # 卡片宽度
  height: auto           # 卡片高度
  background: 'linear-gradient(135deg, #667eea, #764ba2)' # 渐变背景
  border-radius: 24px    # 圆角
  backdrop-filter: blur(20px) # 背景模糊
  box-shadow: 0 8px 32px rgba(0,0,0,0.2) # 阴影
```

### 第二种：按选择器覆盖内部元素（高级）

该方法可以定义卡片内部任意一个类的样式，功能很大，可以根据自己的需求将卡片定义为任意样式。

```
style:
- selector: '.room-card .grouped-btn' # CSS 选择器
  background: 'rgba(255,255,255,0.1)'
  border-radius: 16px
  box-shadow: 0 4px 12px rgba(0,0,0,0.1)
- class: 'primary-text' # 也可用 class 名
  font-size: 16px
  color: '#3498db'
  font-weight: 500
- class: 'room-name'
  font-size: 14px
  color: '#7f8c8d'
```

selector 和 class 二选一即可。selector 写完整 CSS 选择器，class 只需要写类名。

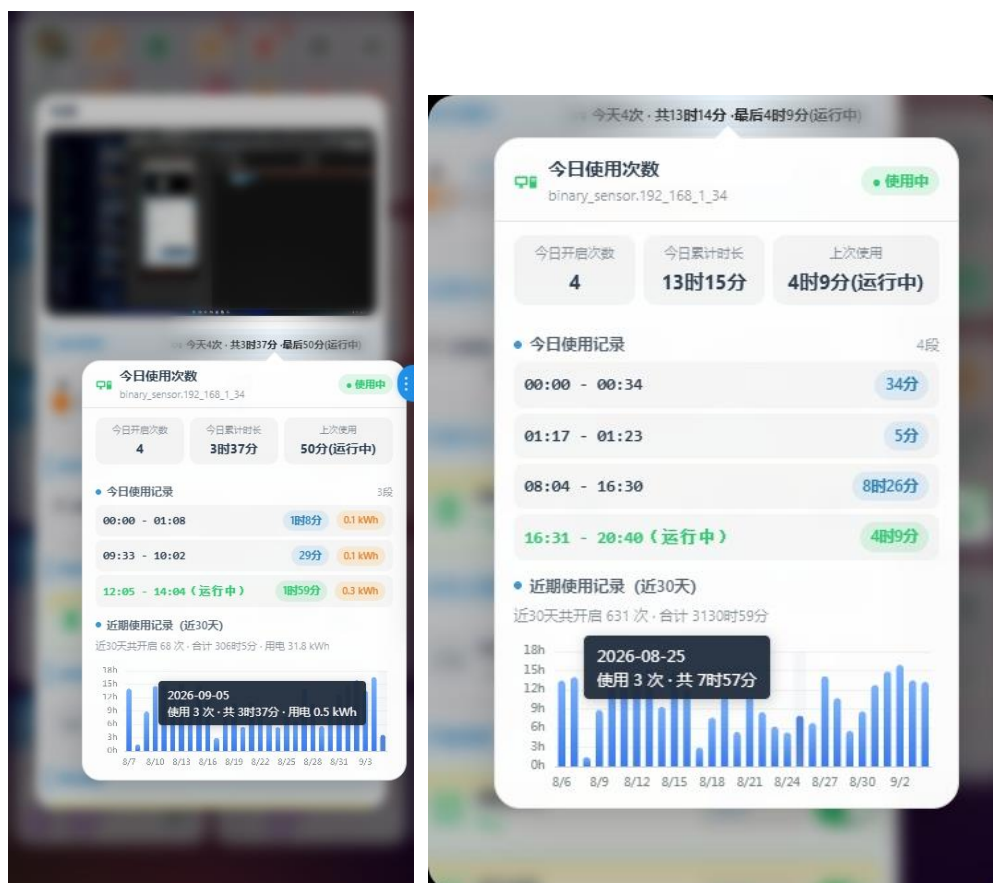
找到该 class，并根据元素 id 来设置样式，一个元素 class 一个样式。

元素 class 查找方法：浏览器开发者工具→元素选项卡，鼠标选择要设置的元素，将元素 class 复制出来，下方即可设置样式。



## 7. Way 计算引擎

Way 引擎可以为实体计算各种统计数据（如"今天开了几次"、"今天一共开了多久"等）。作为传感器的数据源使用。点击弹出气泡，无需配置杰克弹出详细信息。



```
- type: sensor
  entity: switch.youyanji
  name: 今日使用次数
  show_name: false
  way: today_usage_count
  format: 今天用了{ default}次
```

way 写的是"计算方式", format 写的是"显示格式"。

{\_default} 是单实体模式下的默认实体别名, 会被计算结果替换。

## format 占位符规则

format 模板中使用 {键名} 来引用计算结果，具体规则取决于实体数量和 way 数量：

| 场景          | 可用的占位符               | 示例                                       |
|-------------|----------------------|------------------------------------------|
| 单实体 + 单 way | {_default} 或 {way 名} | {today_usage_count}                      |
| 单实体 + 多 way | 只能用 {way 名}          | {today_usage_time} + {today_usage_count} |
| 多实体         | 只能用 {别名.way 名}       | {plug1.today_usage_time}                 |

**说明：**单实体模式下，实体别名为 \_default。单 way 时两种写法等价；多 way 时 {\_default} 无法区分不同 way 的结果，必须用 way 名引用。

## 所有可用的 way

| way 名称          | 说明                | 示例结果      |
|-----------------|-------------------|-----------|
| state           | 取原始状态值            | on / off  |
| friendly_state  | 显示中文友好状态          | 开启 / 关闭   |
| last_changed    | 距上次状态变化过了多久       | 2 时 15 分前 |
| last_updated    | 距上次更新过了多久         | 5 分前      |
| last_usage_time | 最近一次完整 on 会话的持续时长 | 5         |

## 开关型 - 今日统计

| way 名称            | 说明                  | 示例结果     |
|-------------------|---------------------|----------|
| today_usage_count | 今日开启次数              | 5        |
| today_usage_time  | 今日累计开启时长            | 3 时 20 分 |
| today_on_count    | 同 today_usage_count | 5        |
| today_off_count   | 今日关闭次数              | 3        |
| on_duration       | 当前已持续开启时长           | 1 时 30 分 |
| off_duration      | 当前已持续关闭时长           | 30 分     |
| last_on_time      | 最后一次开启的时间           | 14:30    |
| last_off_time     | 最后一次关闭的时间           | 09:15    |
| first_on_time     | 今日首次开启时间            | 07:00    |

### 开关型 - 昨日统计

| way 名称                | 说明       |
|-----------------------|----------|
| yesterday_usage_count | 昨日开启次数   |
| yesterday_usage_time  | 昨日累计开启时长 |
| yesterday_on_count    | 昨日开启次数   |
| yesterday_off_count   | 昨日关闭次数   |

### 数值型 - 今日统计

| way 名称    | 说明    | 示例结果    |
|-----------|-------|---------|
| today_max | 今日最大值 | 28.5° C |
| today_min | 今日最小值 | 22.1° C |



| way 名称         | 说明      | 示例结果    |
|----------------|---------|---------|
| today_avg      | 今日平均值   | 25.3° C |
| today_sum      | 今日累计值   | 3.5kWh  |
| today_range    | 今日波动幅度  | 6.4° C  |
| today_max_time | 最大值出现时间 | 14:30   |
| today_min_time | 最小值出现时间 | 06:00   |
| round          | 保留小数    | 0.12    |

## 数值型 - 昨日统计

| way 名称             | 说明      |
|--------------------|---------|
| yesterday_max      | 昨日最大值   |
| yesterday_min      | 昨日最小值   |
| yesterday_avg      | 昨日平均值   |
| yesterday_sum      | 昨日累计值   |
| yesterday_max_time | 昨日最大值时间 |
| yesterday_min_time | 昨日最小值时间 |

## 趋势

| way 名称 | 说明      | 示例结果                     |
|--------|---------|--------------------------|
| trend  | 对比昨日同时段 | ↑ 2.5° C 或 ↓ 1.0° C 或 持平 |

## 多实体 + 多 way 示例

```

buttons:
- type: socket
  entities:                                # 方式一：用 entities 定义多个实体
    plug1: switch.plug1
    plug2: switch.plug2
  way:
    plug1: ['today_usage_time', 'today_usage_count'] # 给插头 1 两个 way
    plug2: state                                     # 插头 2 取原始状态
  format: '插头 1: {plug1.today_usage_time} (用了 {plug1.today_usage_count}
次) | 插头 2: {plug2.state}'
  unit: ''                                           # 单位

```

## 用 entity 的单个 way（单实体时）

```

buttons:
- type: socket
  entity: switch.plug1
  way: today_usage_count                    # 今日使用次数
  format: '今天用了{_default}次'          # 显示格式（也可写
{today_usage_count}）

```

## 单实体 + 多 way

单实体配置多个 way 时，format 中必须用 way 名引用，不能用 {\_default}：

```

buttons:
- type: socket
  entity: switch.plug1
  way: ['today_usage_time', 'today_usage_count'] # 同时计算时长和次数
  format: '今日开启 {today_usage_time}，共 {today_usage_count} 次'

```

注意：单实体 + 单 way 时 {\_default} 和 {way 名} 都可以用；单实体 + 多 way 时只能用 {way 名}；多实体时用 {别名.way 名}。

## round 用法示例

round 用于将传感器值保留指定小数位数，适合温度、湿度等存在多位小数的传感器（支持 v3.1.1 及后续版本）。

```
# 默认保留 1 位小数
buttons:
  - type: sensor
    entity: sensor.temperature
    way: round
    unit: °C

# 指定保留 2 位小数
buttons:
  - type: sensor
    entity: sensor.humidity
    way: round
    decimals: 2
    unit: '%'

# 不保留小数（取整）
buttons:
  - type: sensor
    entity: sensor.power
    way: round
    decimals: 0
    unit: W
```

round 仅对数字类型的实体状态生效，非数字值（如 on/off/unavailable）会原样返回。

8. primary 文本（v3.1.3 增强功能）

按钮下方的"primary"文本支持使用模板来动态显示数据。



```
- type: socket
icon: mdi:power-socket-au
width: 400px
per_line: 1
primary: >
    {% set value = states('sensor.cha_zuo_zong_gong_lv') | float(0) %} {%
if
    value < 1000 %}
        {{ value | round(0)}}W
    {% else %}
        {{ (value / 1000) | round(2) }}kW
    {% endif %}
```

数据计算函数：

| 函数       | 说明  | 示例                       |
|----------|-----|--------------------------|
| SUM(a,b) | 求和  | {{SUM(t1,t2)}}           |
| SUB(a,b) | 相减  | {{SUB(t1,5)}}            |
| MUL(a,b) | 相乘  | {{MUL(power,0.001)}} 转千瓦 |
| DIV(a,b) | 相除  | {{DIV(used,total)}}      |
| MAX(a,b) | 最大值 | {{MAX(t1,t2,t3)}}        |
| MIN(a,b) | 最小值 | {{MIN(v1,v2)}}           |

| 函数                       | 说明               | 示例                          |
|--------------------------|------------------|-----------------------------|
| AVG(a,b)                 | 平均值              | {{AVG(t1,t2,t3)}}           |
| ABS(n)                   | 绝对值              | {{ABS(n)}}                  |
| ROUND(n,1)               | 四舍五入保留 1 位<br>小数 | {{ROUND(t1,1)}}             |
| IF(条件,true 值,false<br>值) | 条件判断             | {{IF(t1>30,'很热','还好')}}     |
| EACH_SUM(alias,'prop')   | 数组属性求和           | {{EACH_SUM(items,'value')}} |
| EACH_AVG(alias,'prop')   | 数组属性平均           | {{EACH_AVG(items,'price')}} |
| EACH_COUNT(alias)        | 数组元素个数           | {{EACH_COUNT(items)}}       |

**Javeler 风格模板（兼容 Jinja2）：**

```
primary: |
{% set total = states('sensor.power1') + states('sensor.power2') %}
{% if total > 1000 %}
    当前功率偏高: {{ total }}W
{% else %}
    当前功率正常: {{ total }}W
{% endif %}00) | round(2) }}kW
{% endif %}
```

前端内置的 `_TemplateEngine` 只支持 Jinja2 的子集（不支持 `elif`、`for` 循环、过滤器链 | `map` | `list` 等）。当你需要使用完整的 Jinja2 语法时，可以添加 `server_explain:`  
`true`，将模板发送到 Home Assistant 后端渲染。

**工作原理**

|                            |                                                        |
|----------------------------|--------------------------------------------------------|
| server_explain: false (默认) | → 前端 TemplateEngine 渲染 (同步, 零延迟)                       |
| server_explain: true       | → 后端 HA REST API 渲染 (异步, 完整 Jinja2)<br>后端不可用时自动降级到前端引擎 |

## 适用场景

- 需要 {% elif %} 多值判断
- 需要 {% for %} 循环
- 需要过滤器链, 如 | map(attribute='xxx') |  
map('float') | list | sum
- 需要复杂的 Jinja2 语法, 前端引擎无法正确解析时

## 用法示例

按钮 primary 文本:

```
buttons:
- icon: mdi:fire
  primary: >-
    {% set sensor = 'sensor.tian_ran_qi_2_new_2' %}
    {% set balance = states(sensor) | float(0) %}
    {% set daylist = state_attr(sensor, 'daylist') %}
    {% if daylist and daylist | length > 0 %}
    {% set e_gas_list = daylist | map(attribute='e_gas') | map('float')
| list %}
    {% set avg_e_gas = e_gas_list | sum / e_gas_list | length %}
    {% set days_left = (balance / avg_e_gas) | round(0) if avg_e_gas > 0
else 0 %}
    可用 {{ days_left }} 天
    {% else %}
    暂无历史数据
    {% endif %}
  server_explain: true    # ← 加这个字段, 走后端渲染
```

## 配置说明

| 参数             | 类型      | 默认值   | 说明                                                                     |
|----------------|---------|-------|------------------------------------------------------------------------|
| server_explain | boolean | false | true 时将模板发送到 HA 后端渲染，使用完整 Jinja2 语法；false 或不配置时使用前端 _TemplateEngine 渲染 |

### 注意事项：

- server\_explain: true 的模板通过 HA REST API (POST /api/template) 渲染，存在约 10-50ms 的网络延迟。
- 首次渲染时可能短暂显示为空（异步获取结果后会自动更新），后续更新有 10 秒缓存，不会频繁请求后端。
- 后端不可用时（如网络断开），会自动降级到前端 \_TemplateEngine 渲染。
- 简单模板（如 {{ states('sensor.xxx') }}℃）不需要开启 server\_explain，前端引擎即可处理。
- server\_explain 可用于任何包含 primary 或 content 模板的配置项（按钮、公告栏等）。
- primary 文本默认每 10 秒更新一次，可通过顶层参数 primary\_update\_interval 自定义更新频率（秒），设为 0 表示实时更新。

## 9. 配置共享（多个卡片共用配置）

如果你有多个房间卡片，它们可以共享按钮配置。这样改一个地方，所有卡片都生效。

为了解决卡片配置存在重复配置的情况，采用外部 Store (NanoStores) 方法实现穿透卡片配置，一处定义多处引用，卡片采用 RoomElvesStore 方法，该方法是挂在 window 上的全局单例，允许跨卡片实例、跨自定义元素类型共享配置，实现一处定义、多处引用。

同时支持参数替换，也就是说，当引用过来后，部分参数替换后作为新的卡片配置。

### 第一步：在"源卡片"中注册配置

在按钮的 tap\_action 中添加 config\_id:

```
# 卡片 A: 定义共享配置
buttons:
  - type: lights
    tap_action:
      action: call-service
      service: light.turn_on
      service_data:
        entity_id: light.living
        config_id: living_light_config      # 给这个动作起个名字
```

### 第二步：在"目标卡片"中引用

```
# 卡片 B: 引用卡片 A 的配置
buttons:
  - type: lights
    tap_action:
      action: call-service
      card_config:      # 配置引用
        from_config_id: living_light_config # 引用哪个配置
```



|                                       |                           |
|---------------------------------------|---------------------------|
| get_type: action                      | # 提取类型: all/entity/action |
| replace_config:                       | # 可以替换部分字段                |
| service_data.entity_id: light.bedroom | # 把灯换成卧室的                 |

get\_type 有三种: all 取完整配置, entity 只取第一个实体 ID, action 只取交互动作。

### 高级引用: tabs\_config 组合引用 — 从多个配置中组合新的弹窗

除了在 card\_config.from\_config\_id 中引用整个 action 配置外, 还可以使用 tabs\_config 将多个已注册的配置组合成一个新的选项卡弹窗。这对于将不同设备(如冰箱、微波炉、电饭锅)的独立弹窗配置合并为一个"厨具"弹窗非常实用。

### 核心概念

| 概念          | 说明                                                                       |
|-------------|--------------------------------------------------------------------------|
| config_id   | 在 tap_action 或 tabs 数组元素上声明的唯一标识, 配置会被自动注册到全局 Store                      |
| tabs_config | 在 service_data.tabs 中使用, 通过 from_config_id 数组引用多个已注册的配置, 展开为选项卡          |
| 展开规则        | 被引用的配置如果有 tabs → 展开为多个选项卡; 有 rows → 作为一个选项卡; 有 cards → 自动转为 rows 作为一个选项卡 |
| 混用          | tabs_config 和手动定义的选项卡可以在同一个 tabs 数组中混用, 按顺序排列                            |

### 注册配置的三种位置

config\_id 可以写在以下位置, 都会被自动扫描并注册到

Store:

### 位置 1: tap\_action 内部 (原有功能)

注册的是整个 action 对象

(含 config\_id + service\_data 等), 适合  
被 card\_config.from\_config\_id 引用。

```
buttons:
  - name: 冰箱
    entity: switch.bingxiang
    tap_action:
      config_id: bingxiang          # 注册 ID
      action: call-service
      service: modern_room_card.show_free_layout_popup
      service_data:
        title: 冰箱
        width: 360px
      cards:
        - row: 1
          items:
            - type: picture
              bg_image: /local/house/冰箱.png
```

### 位置 2: tabs 数组元素内部 (新增, 专用于 tabs\_config 引用)

注册的是该 tab 对象

(含 config\_id + rows + entity 等), 适合  
被 tabs\_config.from\_config\_id 引用为单个选项卡。

```
buttons:
  - type: dynamic_icon
    name: 厨具
    tap_action:
      action: call-service
      service: modern_room_card.show_free_layout_popup
      service_data:
        title: 厨具
      tabs:
        - name: 电饭锅
          config_id: dian_fan_guo    # 注册 ID (在 tabs 数组内)
          entity: sensor.chunmi_cn xxx status
```

```

        entity_value:
            - Delay
            - Keep Warm
            - Busy
        rows:
            - row: 1
              items:
                - type: picture
                  bg_image: /local/house/电饭锅.png
- name: 空气炸锅
  config_id: kongqizhaguo      # 注册 ID
  entity: switch.kongqizhaguo
  entity_value: "on"
  rows:
    - row: 1
      items:
        - type: picture
          bg_image: /local/house/空气炸锅.png
- name: 微波炉
  config_id: weibolu          # 注册 ID
  entity: switch.weibolu
  entity_value: "on"
  rows:
    - row: 1
      items:
        - type: picture
          bg_image: /local/house/微波炉.png

```

**注意：**tabs 数组内的 config\_id 和 tap\_action 内的 config\_id 互不冲突，因为它们注册的内容类型不同（tab 级 vs action 级），同一 ID 不应重复使用。

### 使用 tabs\_config 组合引用

在另一个按钮的 service\_data.tabs 中，使用 tabs\_config 将上述已注册的配置组合为新的弹窗：

```

buttons:
  - name: 全部厨具
    icon: mdi:countertop
    tap_action:
      action: call-service
      service: modern_room_card.show_free_layout_popup
      service_data:
        title: 全部厨具
        width: 430px
        popup_position: center
        display_only:
          - "on"
          - Delay
          - Keep Warm
          - Busy
          - "off"

```

|              |                               |                    |
|--------------|-------------------------------|--------------------|
|              | - Idle                        |                    |
|              | auto_redirect: true           |                    |
|              | tabs:                         |                    |
|              | - tabs_config:                | # 组合引用             |
|              | from_config_id:               |                    |
| 配置           | - config_id: bingxiang        | # 引用冰箱的 tap_action |
|              | name: 冰箱                      | # 覆盖选项卡名称          |
|              | icon: mdi:fridge              | # 覆盖选项卡图标          |
|              | entity: switch.bingxiang      | # 覆盖选项卡 entity     |
| entity_value | entity_value: "on"            | # 覆盖选项卡            |
|              | replace_config:               | # 替换配置中的指定键        |
|              | entity_value: "on"            |                    |
|              | - config_id: dian_fan_guo     | # 引用电饭锅的 tab 配置    |
|              | name: 电饭锅                     |                    |
| 置            | - config_id: kongqizhaguo     | # 引用空气炸锅的 tab 配置   |
|              | - config_id: weibolu          | # 引用微波炉的 tab 配置    |
|              | - name: 其他设备                  | # 混用手动定义的选项卡       |
|              | entity: switch.other          |                    |
|              | entity_value: "on"            |                    |
|              | rows:                         |                    |
|              | - row: 1                      |                    |
|              | items:                        |                    |
|              | - type: picture               |                    |
|              | bg_image: /local/house/其他.png |                    |

## from\_config\_id 引用项的字段说明

| 字段             | 必填 | 类型           | 说明                                               |
|----------------|----|--------------|--------------------------------------------------|
| config_id      | ✓  | string       | 要引用的已注册配置 ID                                     |
| name           | ✗  | string       | 覆盖选项卡显示名称（不指定则沿用原配置的 name）                       |
| icon           | ✗  | string       | 覆盖选项卡图标（如 mdi:fridge，不指定则沿用原配置的 icon）            |
| entity         | ✗  | string       | 覆盖选项卡实体 ID（用于 display_only 过滤和 auto_redirect 跳转） |
| entity_value   | ✗  | string/array | 覆盖选项卡实体匹配值（用于 display_only 和 auto_redirect 判断）   |
| replace_config | ✗  | object       | 替换解析结果中的指定键值对（详见下文）                              |

## 展开规则详解

tabs\_config 根据被引用配置的结构自动决定如何展开：

| 被引用配置结构                                    | 展开方式                             | 典型场景                        |
|--------------------------------------------|----------------------------------|-----------------------------|
| 有 tabs 数组（action 级注册，service_data.tabs 存在） | 展开为多个选项卡，每个子 tab 独立展示            | 引用一个含多个 tabs 的弹窗配置，拆分为独立选项卡 |
| 有 tabs 数组（tab 级注册，顶层 tabs 存在）              | 展开为多个选项卡                         | 同上                          |
| 有 rows 数组                                  | 作为一个选项卡                          | 引用单个 tab 配置，直接作为选项卡         |
| 有 cards 数组（无 tabs）                         | 自动将 cards 转为 rows，作为一个选项卡        | 引用使用 cards 布局的弹窗配置          |
| 仅有 entity                                  | 构造最小选项卡（含 entity + entity_value） | 引用简单实体配置                    |

示例：被引用配置含 tabs 时展开为多个选项卡

假设 chuju 注册的配置含 3 个 tabs（电饭锅、空气炸锅、微波炉），则：

```
tabs:
- tabs_config:
  from_config_id:
  - config_id: chuju      # 展开为 3 个选项卡
```

等效于：

```
tabs:
- name: 电饭锅
  entity: sensor.chunmi_cn_xxx_status
  entity value: [Delay, Keep Warm, Busy]
```

```

    rows: [...]
  - name: 空气炸锅
    entity: switch.kongqizhaguo
    entity_value: "on"
    rows: [...]
  - name: 微波炉
    entity: switch.weibolu
    entity_value: "on"
    rows: [...]

```

如果只想引用其中一个，直接使用子 tab 的 config\_id:

```

tabs:
  - tabs_config:
      from_config_id:
        - config_id: dian_fan_guo  # 只引用电饭锅

```

## replace\_config 替换配置

每个引用项都可以使用 replace\_config 对展开后的选项卡配置进行**整项替换**（不做深度合并）。支持两种方式：

**方式 1：简单键名** — 直接替换顶层键，如果顶层不存在则递归搜索并替换首个匹配。

```

- config_id: bingxiang
  name: 冰箱
  replace_config:
    entity_value: "on"  # 替换 entity_value 字段

```

**方式 2：点号路径** — 精确定位嵌套对象中的属性。

```

- config_id: bingxiang
  replace_config:
    service_data.display_only:  # 精确替换 service_data.display_only
      - "on"
      - "off"

```

replace\_config 是**整项替换**而非深度合并。例如替

换 entity\_value 时，新值完全覆盖旧值（数组替换数组）。

## 混用 tabs\_config 和手动选项卡

tabs\_config 引用项和手动定义的选项卡可以自由混排，按出现顺序组合：

```
tabs:
  - tabs_config:
      from_config_id:
        - config_id: bingxiang
          name: 冰箱
  - name: 烤箱 # 手动定义的选项卡
    entity: switch.oven
    entity_value: "on"
    rows:
      - row: 1
        items:
          - type: picture
            bg_image: /local/house/烤箱.png
  - tabs_config:
      from_config_id:
        - config_id: weibolu
  - name: 洗碗机 # 又一个手动选项卡
    entity: switch.dishwasher
    rows: [...]
```

最终渲染的选项卡顺序为：冰箱 → 烤箱 → 微波炉 → 洗碗机。

## 覆盖字段的优先级

当引用项指定了覆盖字段（如 name、icon、entity、entity\_value）时：

- 引用项中的覆盖字段 **优先级最高**，会替换原配置中的同名值。
- 未指定的字段 **沿用原配置的值**。。
- replace\_config 在覆盖字段之后应用，可以替换任意层级

的键值。

优先级: `replace_config` > 引用项覆盖字段 > 原配置值

`tabs_config` 自身的 `config_id` (可选)

如果组合后的弹窗本身也想被其他地方引用, 可以

在 `tabs_config` 上添加 `config_id`:

```
tabs:
- tabs_config:
  config_id: chuju_all          # 可选: 注册组合后的配置
  from_config_id:
  - config_id: bingxiang
    name: 冰箱
  - config_id: weibolu
```

## 循环引用保护

系统内置了循环引用检测。如果 A 引用 B, B 又引用 A, 会自动跳过并输出警告日志, 不会导致无限循环。

## 完整示例: 厨房场景

### 步骤 1: 在厨具按钮中注册各设备配置

```
buttons:
- type: dynamic_icon
  name: 厨具
  icon: mdi:countertop
  color: "#74787c"
  show_badge: true
  loop_display: true
  display_time: 3
  rules:
  - condition:
    entity: sensor.chunmi_cn_404489380_ehl_status_p_2_1
    operator: "=="
    value: [Delay, Keep Warm, Busy]
  icon: mdi:stove
  color: "#f58220"
  animation: breathe
  name: 电饭锅
  tap_action:
```



```

    config_id: chuju                                # 注册厨具弹窗 (action 级, 含
tabs)
    action: call-service
    service: modern_room_card.show_free_layout_popup
    service_data:
      title: 厨具
      width: 430px
      popup_position: center
      display_only: ["on", Delay, Keep Warm, Busy, "off", Idle]
      auto_redirect: true
    tabs:
      - name: 电饭锅
        config_id: dian_fan_guo                    # 注册电饭锅 tab (tab 级)
        entity: sensor.chunmi_cn_404489380_ehl_status_p_2_1
        entity_value: [Delay, Keep Warm, Busy]
        rows:
          - row: 1
            per_line: 1
            title: 电饭锅
            show_title: false
            items:
              - type: picture
                show_button_background: false
                halo_entity: input_boolean.chu_fang_dian_fan_guo
                bg_image: /local/house/电饭锅.png
                bg_image_position: [0, 0, 0.8, 0]
      - name: 空气炸锅
        config_id: kongqizhaguo                    # 注册空气炸锅 tab
        entity: switch.kongqizhaguo
        entity_value: "on"
        rows:
          - row: 1
            per_line: 1
            title: 空气炸锅
            show_title: false
            items:
              - type: picture
                show_button_background: false
                bg_image: /local/house/空气炸锅 1.png
                bg_image_position: [0, 0, 0.8, 0]
      - name: 微波炉
        config_id: weibolu                          # 注册微波炉 tab
        entity: switch.weibolu
        entity_value: "on"
        rows:
          - row: 1
            per_line: 1
            title: 微波炉
            show_title: false
            items:
              - type: picture
                show_button_background: false
                halo_entity: switch.weibolu
                bg_image: /local/house/微波炉.png
                bg_image_position: [0, 0, 0.8, 0]

```

## 步骤 2: 在另一个按钮中组合引用

```

- name: 冰箱
  entity: switch.canting_bingxiang_chazuo
  on_icon: mdi:fridge
  off_icon: mdi:fridge
  on_color: "#f58220"

```

```

off_color: "#74787c"
tap_action:
  config_id: bingxiang          # 注册冰箱弹窗 (action 级)
  action: call-service
  service: modern_room_card.show_free_layout_popup
  popup_position: clone_button_left
  service_data:
    title: 冰箱
    width: 360px
  cards:
    - row: 1
      per_line: 1
      title: 冰箱
      show_title: false
      items:
        - type: picture
          show_button_background: false
          bg_image: /local/house/冰箱.png
          halo_entity: switch.canting_bingxiang_chazuo
          bg_image_position: [0, 7, 0.8, 0]

```

### 步骤 3: 创建组合弹窗 (全厨房设备)

```

- name: 全部厨房
  icon: mdi:fridge-outline
  tap_action:
    action: call-service
    service: modern_room_card.show_free_layout_popup
    service_data:
      title: 全部厨房设备
      width: 430px
      popup_position: center
      auto_redirect: true
      display_only: ["on", Delay, Keep Warm, Busy, "off", Idle]
  tabs:
    - tabs_config:
        from_config_id:
          - config_id: bingxiang          # 引用冰箱弹窗 (有 cards → 转为
1 个 tab)
            name: 冰箱
            icon: mdi:fridge
            entity: switch.canting_bingxiang_chazuo
            entity_value: "on"
          - config_id: dian_fan_guo      # 引用电饭锅 tab
          - config_id: kongqizhaguo     # 引用空气炸锅 tab
          - config_id: weibolu          # 引用微波炉 tab
    - name: 其他                        # 混入手动选项卡
      entity: switch.other_kitchen
      entity_value: "on"
      rows:
        - row: 1
          items:
            - type: picture
              bg_image: /local/house/其他.png

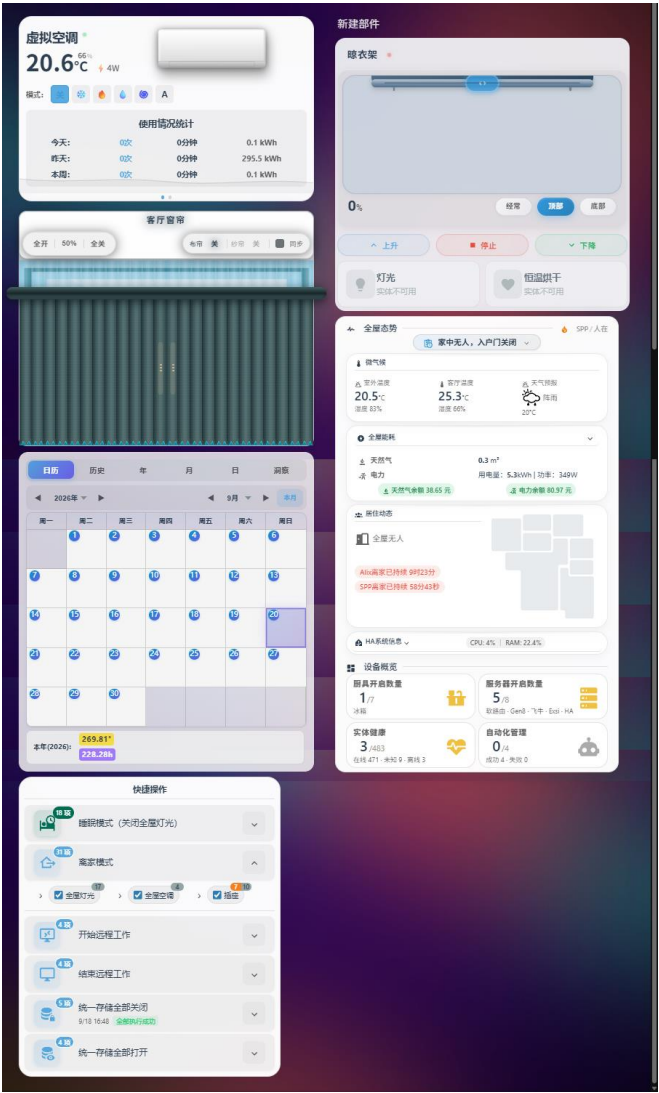
```

## 10. 外部 API 历史数据

如果你的设备历史数据不在 HA 数据库中，而是在一个外部服务器上，可以配置 API 地址，同时改功能是可选功能，如你没有搭建对应的服务端，则删除改配置项：

```
buttons:
- type: socket
  api_base_url: http://192.168.1.100:8080
  api_device_name: 客厅-插座1          # 格式：房间名-设备名
```

## 八、 独立卡片模式 (standalone) v5.0.0 新增



除了将多种设备聚合到一张卡片的常规用法，房间精灵卡片还支持独立卡片模式——将单个设备控制面板直接作为一张卡片使用，无需配置按钮网格。

可以独立使用的卡片有：窗帘卡片、空调卡片、晾衣架卡片、概览卡片、使用量卡片、情景模式卡片。

### 1. 适用场景

- 只需要控制单个设备（如一台空调、一扇窗帘），不需

要房间聚合视图；

- 在仪表盘中混合使用其他卡片，只需某一种设备的专属控制面板；
- 想把空调/窗帘等控制面板放到不同位置，而非集中在一个房间卡片内。

## 2. 基本配置

只需添加 `standalone_type` 字段，卡片会自动跳过按钮渲染，直接显示对应设备的控制界面：

```
type: custom:room-elves-card
standalone_type: ac          # 设备类型
entity: climate.living      # 实体 ID
name: 客厅空调              # 显示名称（可选）
```

## 3. 支持的设备类型

常规设备类型：

| standalone_type | 说明                       | 实体域            |
|-----------------|--------------------------|----------------|
| ac              | 空调控制面板（温度/模式/风速/摆风/使用统计） | climate        |
| curtain         | 窗帘控制面板（百分比滑块/双层/开合模式）    | cover          |
| clothes_dryer   | 晾衣架控制面板（拖拽/收藏位置/灯光联动）    | cover / switch |
| scene_mode      | 把情景模式气泡的内容直接渲染成一张常显卡片    | -              |

| standalone_type | 说明                                                                    | 实体域 |
|-----------------|-----------------------------------------------------------------------|-----|
| usage_calendar  | 使用量日历卡片                                                               | -   |
| overview        | 把「全屋态势」概览弹窗的内容（微气候 / 全屋能耗 / 人在 / 设备概览 / 实体健康等）直接渲染成一张常显卡片，配置沿用概览弹窗同一套 | -   |

### 图表类型（基于 ECharts / SVG）：

| standalone_type    | 说明      | 渲染引擎    |
|--------------------|---------|---------|
| chart / chart_line | 曲线趋势图   | ECharts |
| chart_gauge        | 仪表图     | 纯 SVG   |
| chart_progress     | 进度条图    | 纯 CSS   |
| chart_bar          | 柱状图     | 纯 CSS   |
| chart_pie          | 圆角环形图   | ECharts |
| chart_pie_full     | 完整饼图    | ECharts |
| chart_heatmap      | 日历热力图   | ECharts |
| chart_calendar     | 日历图     | ECharts |
| chart_mixed        | 混合图表    | ECharts |
| chart_nightingale  | 南丁格尔玫瑰图 | ECharts |
| chart_scatter      | 散点图     | ECharts |

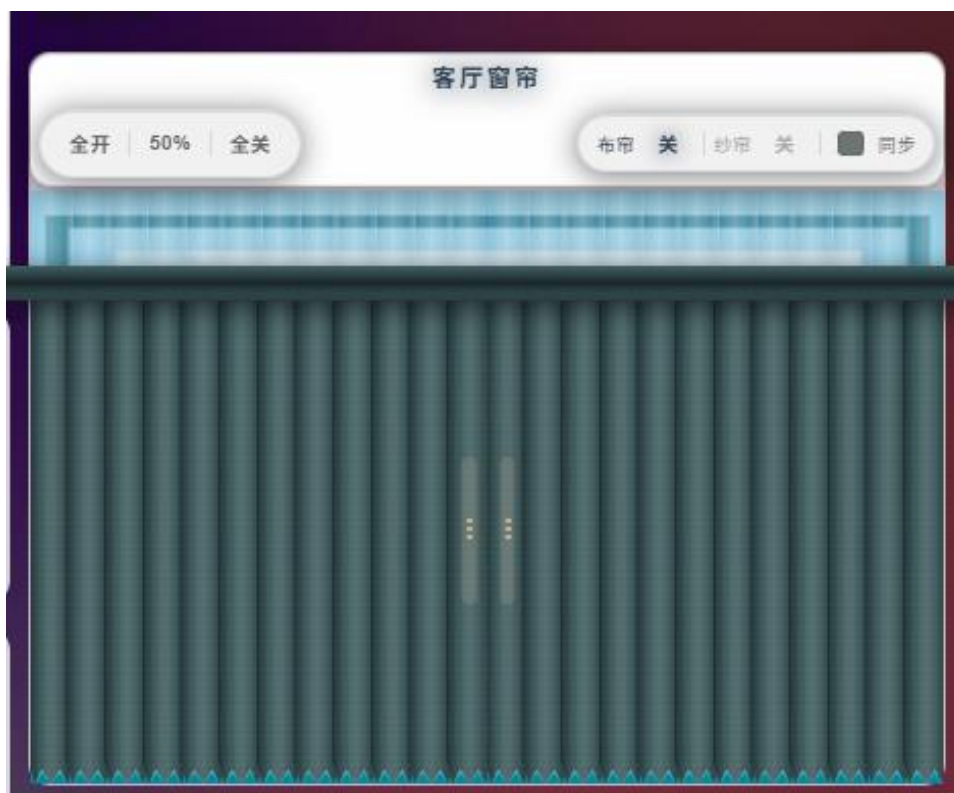
## 4. 配置示例

### 空调独立卡片



```
type: custom:room-elves-card
standalone_type: ac
name: 虚拟空调
entity: climate.ceshi
humidity_entity: sensor.keting_shidu
power_entity: sensor.ke_ting_kong_diao_nian_yong_dian_liang
power_display_entity: sensor.esp_meter_power
width: 100%
```

### 窗帘独立卡片



```

type: custom:room-elves-card
standalone_type: curtain
name: 客厅窗帘
fabric_entity: cover.living_room_fabric
sheer_entity: cover.living_room_sheer
layer_mode: double
open_mode: double
width: 430px
background_image: /local/window_bg_1.png

```

## 晾衣架独立卡片



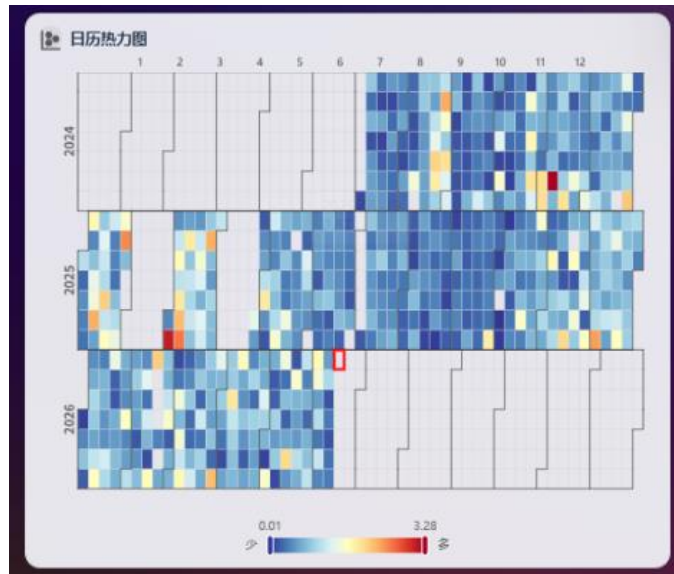
```

type: custom:room-elves-card
standalone_type: clothes_dryer
name: 晾衣架
entity: cover.ceshi
light_entity: input_boolean.ce_shi
on_icon: mdi:hanger
off_icon: mdi:tshirt-crew
on_color: "#f58220"
off_color: "#74787c"
width: 400px
locker: false
collect_position:
  - name: 经常
    position: 70
  - name: 顶部
    position: 0
  - name: 底部
    position: 100
button:
  - type: switch
    entity: input_boolean.ce_shi
    name: 恒温烘干
    icon: mdi:heart

```

## 图表类





```

type: custom:room-elves-card
standalone_type: chart_heatmap
entity: sensor.tian_ran_qi_2_new_2
height: 380px
icon: mdi:chart-scatter-plot-hexbin
name: 日历热力图
update_interval: 360
entity_array:
  attr: daylist
  day: day
  value: f_gas

```

图表不在一一列举，配置与房间内部配置方法一致，只是增加了 standalone\_type 配置项。

## 使用量独立卡片



```

type: custom:room-elves-card
standalone_type: usage_calendar
api:
  entity: binary_sensor.192_168_1_34
  api_base_url: /api/ha_data_store/
  key: 27d8e81de2f84b3eb225689b4d5d3db2
title: 客厅空调用电量日历
show_title: false
width: 100%
show_popup: true
calc_value_choose: value
units: °,h
series: 用量,时长

```

该卡片建议于“HA 统一数据存储系统”配合使用，因为数据基本都来自于后端。

### 情景模式独立卡片



```

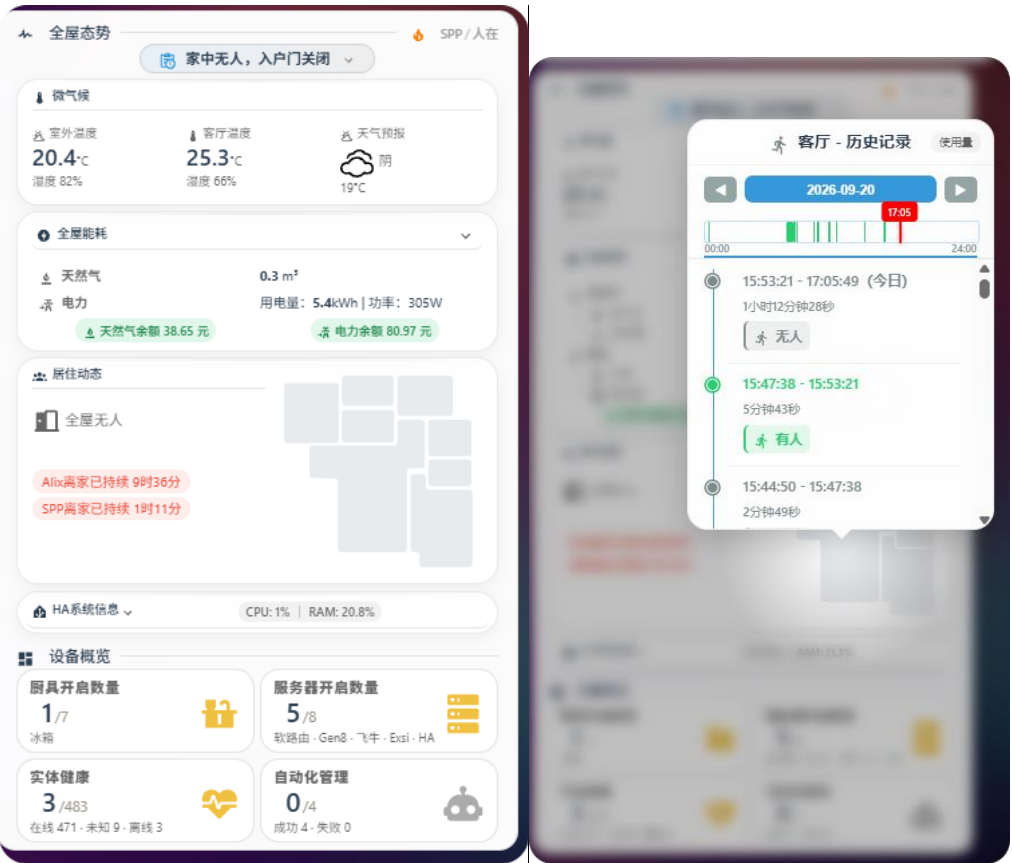
type: custom:room-elves-card
standalone_type: scene_mode
name: 快捷操作
width: 400px
entity: input_text.scene_mode
verify_timeout: 10
scenes:
- name: 睡眠模式（关闭全屋灯光）
  icon: mdi:bed-clock
  icon_color: '#006e54'
  confirm: true
  actions:
    - entities:
      - 客厅_大灯: light.keting_dadeng
      - 客厅_玄关灯: light.keting_xuanguan
      - 客厅_灯带: light.keting_dengdai
      - 厨房_大灯: light.chufang
      - 主卧_吸顶灯: light.zhuwo_xidingdeng
      - 主卧_床头南灯: light.zhuwo_chuangtounan
      - 主卧_床头北灯: light.zhuwo_chuangtoubel
      - 次卧_吸顶灯: light.ciwo_xidingdeng
      - 儿童房_吸顶灯: light.ertongfang_xidingdeng
      - 餐厅_吊灯: light.canting_diaodeng
      - 餐厅_壁灯: light.canting_bideng
      - 餐厅_灯带: light.canting_dengdai
      - 餐厅_过道灯: light.canting_guodao
      - 大卫_门口灯: switch.zimi_cn_1121229342_dhkg05_on_p_2_1
      - 大卫_浴霸灯: switch.zimi_cn_1121229342_dhkg05_on_p_3_1
      - 大卫_镜前灯: switch.zimi_cn_1121229342_dhkg05_on_p_4_1
      - 小卫_浴霸灯: light.lxzn_cn_847198179_ben_s_2
    value: 'off'
    name: 全屋灯光
    - entities:
      - 小卫插座: switch.xiaoweishengjian_chazuo
    name: 小卫插座
    select: true
    value: 'off'
- name: 离家模式
  icon: mdi:home-export-outline
  confirm: true
  fold: true
  expand_all: true
  actions:
    - entities:
      - 客厅_大灯: light.keting_dadeng
      - 客厅_玄关灯: light.keting_xuanguan
      - 客厅_灯带: light.keting_dengdai
      - 厨房_大灯: light.chufang
      - 主卧_吸顶灯: light.zhuwo_xidingdeng
      - 主卧_床头南灯: light.zhuwo_chuangtounan
      - 主卧_床头北灯: light.zhuwo_chuangtoubel
      - 次卧_吸顶灯: light.ciwo_xidingdeng
      - 儿童房_吸顶灯: light.ertongfang_xidingdeng
      - 餐厅_吊灯: light.canting_diaodeng
      - 餐厅_壁灯: light.canting_bideng
      - 餐厅_灯带: light.canting_dengdai
      - 餐厅_过道灯: light.canting_guodao
      - 大卫_门口灯: switch.zimi_cn_1121229342_dhkg05_on_p_2_1
      - 大卫_浴霸灯: switch.zimi_cn_1121229342_dhkg05_on_p_3_1
      - 大卫_镜前灯: switch.zimi_cn_1121229342_dhkg05_on_p_4_1
      - 小卫_浴霸灯: light.lxzn_cn_847198179_ben_s_2
    value: 'off'
    name: 全屋灯光
    - entities:
      - 儿童房_空调: climate.xiaomi_m6_00c5_air_conditioner
      - 次卧_空调: climate.lumi_mcn02_f61f_air_conditioner
      - 主卧_空调: climate.dc330d3430f6_climate
      - 客厅_空调: climate.keting_ac_keting_ac
    name: 全屋空调

```

```
value: 'off'
- entities:
  - 小卫插座: switch.xiaoweishengjian_chazuo
  - 儿童房床头: switch.ertongfang_chuangtouchazuo
  - 次卧床头: switch.ciwo_chuangtouchazuo
  - 厨房烧水壶: switch.chufang_chazuo_shaoshuihu
  - 厨房热水器: switch.chufang_reshuiqi
  - 厨房空气炸锅: switch.chufang_kongqizhaguo
  - 厨房油烟机: switch.chufang_youyanji
  - 厨房电饭锅: switch.chufang_dianfangguo
  - 厨房微波炉: switch.chufang_weibolu
  - 厨房微波炉: switch.chufang_weibolu
name: 插座
value: 'off'
```

情景模式与以前的情景模式（快捷操作）使用方法一致。

概览独立卡片



```
type: custom:room-elves-card
width: 400px
api_base_url: /api/ha_data_store/
key: 27d8e81de2f84b3eb225689b4d5d3db2
standalone_type: overview
overview:
  - recently-used:
    sensor.hashu_ju_tong_yi_cun_chu_xi_tong_jin_qi_shi_yong_she_bei
    width: 400px
```

```

- automations: sensor.ha_data_store_automation
  width: 400px
  name: 自动化管理
- overview_info:
sensor.hashu_ju_tong_yi_cun_chu_xi_tong_today_family_status
- type: environment
  name: 环境
  icon: mdi:thermometer-water
  rooms:
    户外:
      temperature: sensor.ketingwaiyangmian_wendu
      humidity: sensor.ketingwaiyangmian_shidu
    客厅:
      temperature: sensor.keting_wendu
      humidity: sensor.keting_shidu
- type: consumables
  name: 耗材
  icon: mdi:battery
  condition:
    - <10
- type: person
  name: 人在
  icon: mdi:run
  rooms:
    客厅: switch.giot_cn_1126348886_v64ksm_on_p_10_1
    餐厅: binary_sensor.canting_renzai
    厨房: binary_sensor.chufang_you_ren
    儿童房: binary_sensor.ertongfang_you_ren
    主卧: binary_sensor.zhuwo_renzai
    次卧: binary_sensor.ciwo_renzai
    大卫生间: binary_sensor.daweishengjian_renzai
    小卫生间: binary_sensor.xiaoweishengjian_renzai
  person:
    - name: Alix
      entity: device_tracker.ip15_94
    - name: SPP
      entity: device_tracker.spp_39
- automatic_folding_ha_info: true
- type: energy
  name: 能耗
  fold: true
  icon: mdi:gas-burner
  utilities:
    天然气:
      entity: sensor.gas_0814832321_today_usage
      icon: mdi:gas-burner
      cost_entity: sensor.gas_0814832321_day_data
      tap_action:
        card_config:
          from_config_id: ele
          get_type: all
          card_params:
            initial_tab: '2'
    燃气灶:
      icon: mdi:water-boiler
      level: 2
      content: >-
        今日: {{ states('sensor.gas_0814832321_youyanji') | float(0) |
round(2)
        }}m³
    热水器:
      icon: mdi:gas-burner
      level: 2
      content: >-
        今日: {{ states('sensor.gas_0814832321_reshuiqi') | float(0) |
round(2)
        }}m³
    电力:
      entity: sensor.ele

```

```

        icon: mdi:transmission-tower-import
        tap_action:
          card_config:
            from_config_id: ele
            get_type: all
          card_params:
            initial_tab: '1'
        power: sensor.quanwu_zongglv
        power_onsumption: sensor.quan_wu_ri_yong_dian_liang
        hide_room: 儿童房外,次卧外,楼道,客厅外
    空调:
        icon: mdi:air-conditioner
        level: 2
        content: >-
        float(0) {% set power = states('sensor.kong_diao_zong_gong_lu') |
        {% if state_attr('sensor.kong_diao_zong_gong_lu',
        'unit_of_measurement') == 'W' %}
        {% set power_kw = power / 1000 %}
        {% else %}
        {% set power_kw = power %}
        {% endif %} 今日: {{
        states('sensor.suo_you_kong_diao_ri_yong_dian_liang') | float(0) |
        round(1) }}kWh | 功率: {{ power_kw | round(1) }}kw
        tap_action:
          card_config:
            from_config_id: ac
            get_type: all
    服务器:
        icon: mdi:server
        level: 2
        content: >-
        float(0) {% set value =
        states('sensor.cuco_cn_944196247_v3_electric_power_p_11_2') |
        round(1) }}kWh 今日: {{ states('sensor.fuwuqi_day') | float(0) |
        round(1) }}kWh | 功率:
        {%- if value < 1000 -%}
        {{ value | round(0) }}W
        {%- else -%}
        {{ (value / 1000) | round(1) }}kW
        {%- endif -%}
        tap_action:
          card_config:
            from_config_id: fuwuqi
            get_type: all
- type: weather
  height: 300px
  weather:
    - name: 未央区
      entity: weather.qweather_pro_wei_yang_weather
    - name: 西乡县
      entity: weather.qweather_pro_xi_xiang_weather
- type: overview_device
  icon: mdi:countertop
  name: 厨具开启数量
  condition:
    - 'on'
    - Delay
    - Keep Warm
    - Busy
  entities:
    - entity: sensor.chunmi_cn_404489380_eh1_status_p_2_1
      name: 电饭锅
    - entity: switch.kongqizhaguo
      name: 空气炸锅
    - entity: switch.weibolu
      name: 微波炉
    - entity: switch.youyanji

```

```

    name: 油烟机
  - entity: switch.reshuiqi
    name: 热水器
  - entity: input_boolean.shao_shui_hu
    name: 烧水壶
  - entity: switch.canting_bingxiang_chazuo
    name: 冰箱
tap_action:
  action: call-service
  service: modern_room_card.show_free_layout_popup
  service_data:
    title: 全部厨具
    width: 430px
    popup_position: center
    display_only:
      - 'on'
      - Delay
      - Keep Warm
      - Busy
  auto_redirect: true
  tabs:
    - tabs_config:
        from_config_id:
          - config_id: dian_fan_guo
            name: 电饭锅
          - config_id: kong_qi_zhaguo
            name: 空气炸锅
          - config_id: wei_bo_lu
            name: 微波炉
          - config_id: you_yan_ji
            name: 油烟机
          - config_id: re_shui_qi
            name: 热水器
          - config_id: shao_shui_hu
            name: 烧水壶
          - config_id: bingxiang
            name: 冰箱
- type: overview_device
  icon: mdi:server
  name: 服务器开启数量
  condition: 'on'
  entities:
    - entity: binary_sensor.192_168_1_1
      name: 软路由
    - entity: binary_sensor.192_168_1_223
      name: Gen8
    - entity: binary_sensor.192_168_1_37
      name: 备份
    - entity: binary_sensor.192_168_1_12
      name: 飞牛
    - entity: binary_sensor.192_168_1_90
      name: Exsi
    - entity: binary_sensor.192_168_1_61
      name: 下载机
    - entity: binary_sensor.192_168_1_62
      name: 微信
    - entity: binary_sensor.192_168_1_102
      name: HA
  tap_action:
    card_config:
      from_config_id: fuwuqi
      get_type: all
- type: overview_device
  show_card: false
  show_badge: false
  icon: mdi:video
  name: C700 摄像头
  condition: 'on'
  entities:

```

```

    - entity: switch.giot_cn_1124641761_v6shsm_on_p_2_1
      name: C700 摄像头
    tap_action:
      card_config:
        from_config_id: c700
        get_type: all
  - type: overview_device
    show_card: false
    show_badge: false
    icon: mdi:run
    name: 全屋人在
    condition: 'on'
    entities:
      - entity: switch.giot_cn_1126348886_v64ksm_on_p_10_1
        name: 客厅
      - entity: binary_sensor.canting_renzai
        name: 餐厅
      - entity: binary_sensor.chufang_you_ren
        name: 厨房
      - entity: binary_sensor.ertongfang_you_ren
        name: 儿童房
      - entity: binary_sensor.zhuwo_renzai
        name: 主卧
      - entity: binary_sensor.ciwo_renzai
        name: 次卧
      - entity: binary_sensor.daweishengjian_renzai
        name: 大卫生
      - entity: binary_sensor.xiaoweishengjian_renzai
        name: 小卫生
  - type: entities_health
    entity:
      sensor.hashu_ju_tong_yi_cun_chu_xi_tong_frontend_card_entities_health
    hide_room: 次卧外, 儿童房外, 楼道, 客厅外
    name: 实体健康
    icon: mdi:heart-pulse
    width: 400px
    show_card: true

```

概览与以前在头部模式中的使用方法一致。



## 九、 配置速查表

### 1. 顶层配置速查表

| 顶层配置                         | 说明                         | 必须                                  |
|------------------------------|----------------------------|-------------------------------------|
| type: custom:room-elves-card | 卡片类型标识                     | <input checked="" type="checkbox"/> |
| room_name                    | 房间名称                       | 建议                                  |
| head                         | 是否头部模式                     | 否                                   |
| mode                         | 显示模式                       | 否                                   |
| entities                     | 传感器列表                      | 否                                   |
| buttons                      | 按钮配置                       | 否                                   |
| automation                   | 自动化列表                      | 否                                   |
| person                       | 人员配置                       | 否                                   |
| overview                     | 概览栏配置                      | 否                                   |
| theme                        | 主题                         | 否                                   |
| dark_light_theme             | 暗/亮主题对                     | 否                                   |
| show_animation               | 动画开关                       | 否                                   |
| style                        | 自定义样式                      | 否                                   |
| heartbeat_packet             | 心跳包                        | 否                                   |
| entities_tap_action          | 全局 entities 点击动作配置         | 否                                   |
| prohibit_homepage_scroll     | 禁止滚动                       | 否                                   |
| performance_mode             | 性能模式 (auto/desktop/mobile) | 否                                   |

| 顶层配置         | 说明                | 必须 |
|--------------|-------------------|----|
| head_columns | head 模式列数，默认 6    | 否  |
| head_rows    | head 模式行数，默认 auto | 否  |

## 2. 按钮配置速查表

| 按钮配置          | 说明           | 必须  |
|---------------|--------------|-----|
| type          | 按钮类型         | 否   |
| entity        | 实体 ID        | 视类型 |
| name          | 显示名称         | 否   |
| icon          | 图标           | 否   |
| on_icon       | 开启图标         | 否   |
| off_icon      | 关闭图标         | 否   |
| on_color      | 开启颜色         | 否   |
| off_color     | 关闭颜色         | 否   |
| card          | 设备列表         | 视类型 |
| tap_action    | 点击动作         | 否   |
| primary       | 主文本          | 否   |
| show_badge    | 角标           | 否   |
| badge_entity  | 角标实体         | 否   |
| badge         | 条件角标数组（独立按钮） | 否   |
| confirm       | 需要确认         | 否   |
| show_duration | 显示持续时长       | 否   |
| width         | 弹窗宽度         | 否   |
| per_line      | 每行几个         | 否   |

| 按钮配置                 | 说明                     | 必须               |
|----------------------|------------------------|------------------|
| row_column           | 网格位置 (head 按钮/弹窗 item) | 否                |
| tabs_by              | 聚合弹窗按字段分组              | 否                |
| group_lights_by_room | 灯光弹窗按房间分组              | 否                |
| way                  | 计算方式                   | 否                |
| format               | 显示格式                   | 否                |
| rules                | 动态图标规则                 | dynamic_icon 专用  |
| global_exception     | 批量操作时跳过此实体             | 否                |
| light_entity         | 灯光实体                   | clothes_dryer 专用 |

### 3. 图表配置速查表

| 图表卡片类型             | 渲染方式    | 数据源            | 说明                   |
|--------------------|---------|----------------|----------------------|
| chart_gauge        | SVG     | 单实体            | 仪表图，支持 severity 颜色分段 |
| chart_progress     | CSS     | 多实体<br>(cards) | 多进度条，自动颜色分级          |
| chart_bar          | CSS     | 单实体历史          | 柱状图，支持聚合计算           |
| chart_line / chart | ECharts | 单实体历史          | 平滑曲线趋势图              |
| chart_pie          | ECharts | 多实体<br>(cards) | 环形图，中心显示总计           |
| chart_pie_full     | ECharts | 多实体<br>(chart) | 完整饼图，带图例和标签          |
| chart_nightingale  | ECharts | 多实体<br>(chart) | 南丁格尔玫瑰图              |
| chart_heatmap      | ECharts | 实体属性/API       | 年度日历热力图              |
| chart_calendar     | ECharts | 实体属性/API       | 月历+年/月/日柱状图，双系列      |
| chart_mixed        | ECharts | 多实体属性<br>/API  | 柱+线+面积组合图，三级下钻       |

#### 4. 配置共享速查表

| 配置共享                                        | 说明                                | 必须             |
|---------------------------------------------|-----------------------------------|----------------|
| config_id                                   | 配置注册 ID（写在 tap_action 或 tabs 元素内） | 注册时必须填         |
| card_config.from_config_id                  | 引用已注册的配置 ID                       | 引用时必须填         |
| card_config.get_type                        | 提取类型：<br>all / entity / action    | 否，默认 all       |
| card_config.replace_config                  | 替换解析结果中的指定键                       | 否              |
| tabs_config.from_config_id                  | 组合引用多个配置为选项卡                      | tabs_config 必填 |
| tabs_config.from_config_id[].config_id      | 引用的配置 ID                          | ✔              |
| tabs_config.from_config_id[].name           | 覆盖选项卡名称                           | 否              |
| tabs_config.from_config_id[].icon           | 覆盖选项卡图标                           | 否              |
| tabs_config.from_config_id[].entity         | 覆盖选项卡实体                           | 否              |
| tabs_config.from_config_id[].entity_value   | 覆盖选项卡实体匹配值                        | 否              |
| tabs_config.from_config_id[].replace_config | 替换展开后选项卡的指定键                      | 否              |
| tabs_config.config_id                       | 为组合后的配置注册 ID（可选）                  | 否              |

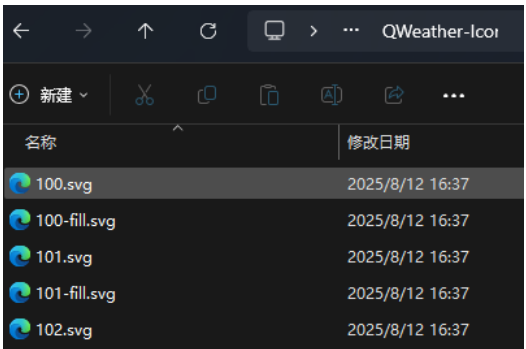
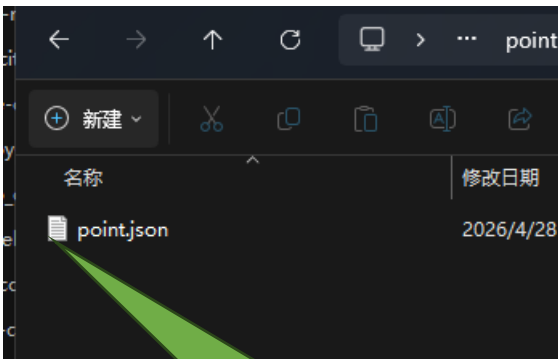
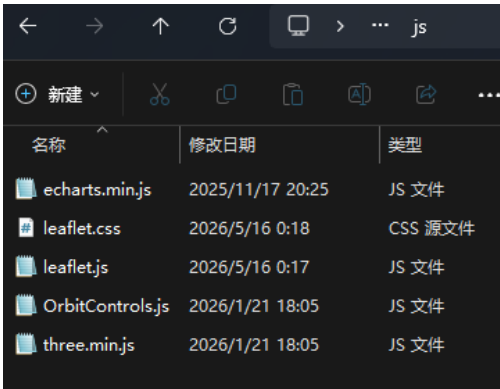
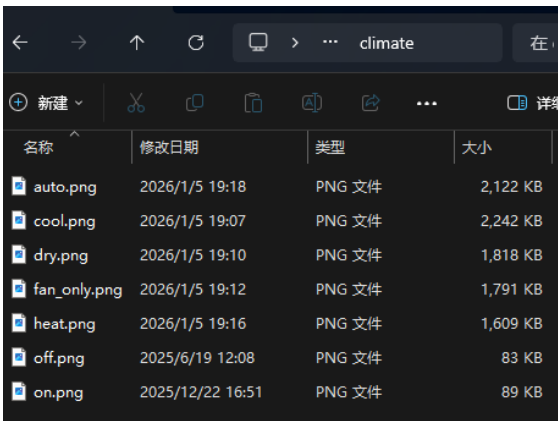
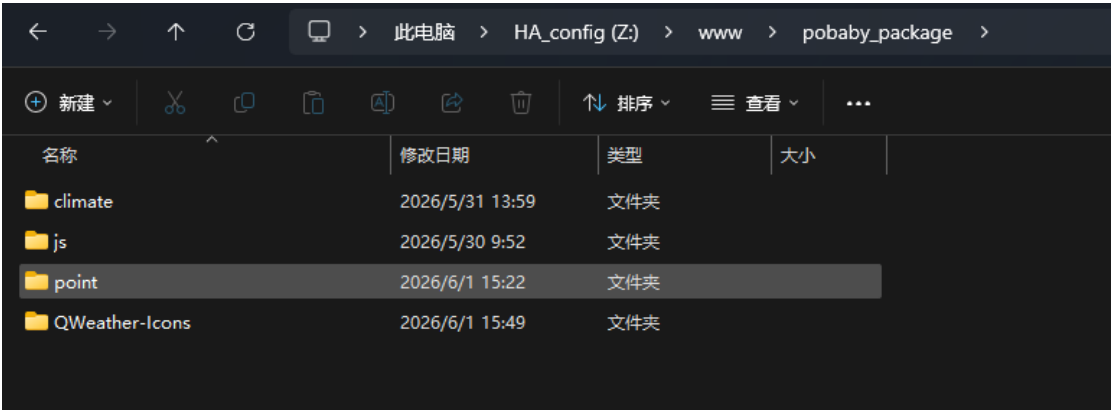
# 十、 外部组件及文件



pobaby\_package  
.rar

PDF 附件中，也可以  
单独下载

## 1. 外部组件及其他文件



Point.json 使用你自  
己的坐标文件

# 十一、配置示例

为了方便用户配置，全量配置了一份示例文件，用户可以根据自己的实际情况，将不需要的部分删除。即使没有实体，直接放置到仪表盘也会渲染 UI。

配置示例：

  
v5.9.2.yaml

配置示例文件（双击图标即可打开）。

```
1. kiosk_mode:
2.   hide_header: '[[[ return screen.width < 768 && is_state("input_boolean.full", "on") ]]]'
3.   hide_sidebar: '[[[ return screen.width < 768 && is_state("input_boolean.full", "on") ]]]'
4.   background: center / cover no-repeat url("/local/ha_background.jpg") fixed
5. views:
6.   - type: sections
7.     title: Home
8.     path: Home
9.     sections:
10.     - type: grid
11.       cards:
12.       - type: custom:room-elves-card
13.         primary_text_variant: minimal
14.         theme: input_select:theme
15.         dark_light_theme: dark,light
16.         head_columns: 7
17.         primary: true
18.         head: true
19.         show_animation: input_boolean.show_animation
20.         style:
21.         - class: room-card
22.           height: 190px
23.           width: 100%
24.       overview:
25.       - type: environment
26.         name: 环境
27.         icon: mdi:thermometer-water
28.         rooms:
29.         户外:
30.           temperature: sensor.ketingvaisyangnian_wendu
31.           humidity: sensor.ketingvaisyangnian_shidu
32.         客厅:
33.           temperature: sensor.keting_wendu
34.           humidity: sensor.keting_shidu
35.       - type: consumables
36.         name: 耗材
37.         icon: mdi:battery
38.         condition:
39.         - <10
40.       - type: person
41.         name: 人在
42.         icon: mdi:room
43.         rooms:
44.         客厅: switch.giot_cn_1126348880_v04kcm_cn_p_10_1
45.         餐厅: binary_sensor.canting_renzai
46.         厨房: binary_sensor.chufang_you_ren
47.         儿童房: binary_sensor.ertongfang_you_ren
48.         主卧: binary_sensor.zhuwo_renzai
49.         次卧: binary_sensor.ciwu_renzai
50.         大卫生间: binary_sensor.daweishengjian_renzai
51.         小卫生间: binary_sensor.xiaoweishengjian_renzai
52.         person:
53.         - name: Aliz
54.           entity: device_tracker.ip15_94
55.         - name: SFP
56.           entity: device_tracker.spp_39
57.         room_point_from: /local/point.json
58.         automatic_folding_ha_info: true
59.       - type: energy
60.         name: 能源
```



十二、配置成果

